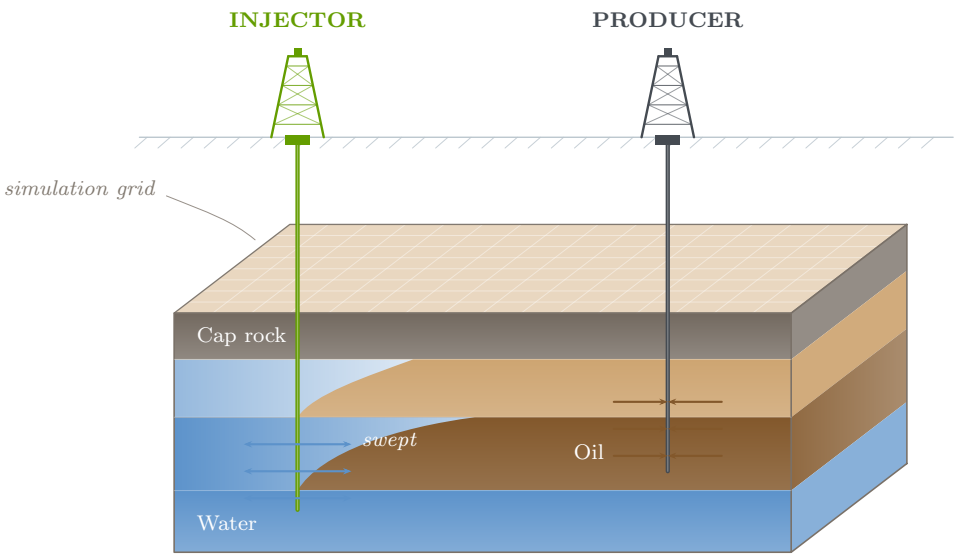
NVRS-CUDA

GPU-Native Reservoir Simulator



User Reference Manual

Manual version 1.2

Clement Etienam Tsubasa Onishi Oleg Ovcharenko
Nick Luiken Issam Said

NVIDIA

Documents NVRS-CUDA release version 2.

Document information

Manual version	1.2
Applies to	NVRS-CUDA release version 2
Prepared by	NVIDIA

Every default value quoted in this manual is the value compiled into the release named above. Every performance figure is taken from a run whose logs and reports are committed in the repository under `benchmark/`.

Contents

1 Introduction **11**
- 1.1 Purpose of this manual . . . 11
- 1.2 Overview of the simulator . . . 11
- 1.3 Supported capabilities . . . 12
- 1.4 Scope and limitations . . . 12
- 1.5 Typographic conventions . . . 13
- 1.6 Units . . . 14
- 1.7 Related documents . . . 15

2 Installation **16**
- 2.1 Hardware requirements . . . 16
- 2.2 Software requirements . . . 16
- 2.3 Building with Make . . . 17
- 2.4 Building with CMake . . . 17
- 2.5 Build options . . . 18
- 2.6 Building for a GPU newer than the toolkit . . . 18
  - 2.6.1 Consequences at run time . . . 19
- 2.7 Installing the dependencies without root . . . 19
- 2.8 Installing everything in one command . . . 20
- 2.9 One command for the whole workflow . . . 20
  - 2.9.1 Commands . . . 20
  - 2.9.2 Options . . . 21
  - 2.9.3 The help screen . . . 21
  - 2.9.4 What it checks before running . . . 22
- 2.10 The browser interface . . . 22
- 2.11 Reproducing a result . . . 23
- 2.12 Verifying the installation . . . 23

3 Running the simulator **24**
- 3.1 Command-line syntax . . . 24
- 3.2 Device binding . . . 24
- 3.3 The run banner . . . 24
- 3.4 Progress output . . . 25
- 3.5 Timing summary . . . 25
- 3.6 Selecting the number of ranks . . . 26
- 3.7 The MPI launcher . . . 26
- 3.8 Environment variables . . . 26
- 3.9 Utility scripts . . . 27
- 3.10 Exit status . . . 28

4 Deck structure **29**
4.1 File format . . . 29
4.2 Comments and whitespace . . . 29
4.3 Keyword ordering . . . 29
4.4 A minimal deck . . . 30
4.5 Deck sections . . . 30

5 Keyword reference **31**
5.1 Model definition . . . 31
5.1.1 NCOMP . . . 31
5.1.2 MODEL . . . 31
5.1.3 TITLE . . . 32
5.1.4 CASENAME . . . 32
5.2 Grid geometry . . . 32
5.2.1 NX, NY, NZ . . . 32
5.2.2 DX, DY, DZ . . . 32
5.2.3 TOPS . . . 32
5.2.4 CONNLIST . . . 32
5.3 Rock properties . . . 33
5.3.1 PERMFILE . . . 33
5.3.2 POROFILE . . . 34
5.3.3 PERMZFILE . . . 34
5.3.4 KZMULT . . . 34
5.3.5 PERMMIN, PERMMAX . . . 35
5.3.6 POROMIN, POROMAX . . . 35
5.4 Fluid properties . . . 35
5.4.1 UW, UO, UG . . . 35
5.4.2 BW, BO, BG . . . 35
5.4.3 RHOW, RHOO, RHOG . . . 35
5.4.4 CFW, CFO, CT . . . 36
5.4.5 RS . . . 36
5.4.6 VAPOIL . . . 36
5.4.7 PBUB . . . 37
5.4.8 PATM . . . 37
5.4.9 PVTCONST . . . 37
5.4.10 TEMP . . . 37
5.4.11 ZFACTOR . . . 38
5.5 Saturation functions . . . 38
5.5.1 SWI . . . 38
5.5.2 SWR . . . 38
5.5.3 SWOF . . . 38
5.5.4 SGOF . . . 39
5.6 Initial conditions . . . 39
5.6.1 PINIT . . . 39
5.6.2 SWINIT, SGINIT, SOINIT . . . 39
5.6.3 GRAVITY . . . 40
5.7 Wells . . . 40
5.7.1 WELLS . . . 40
5.8 Field decks . . . 41
5.8.1 Geometry and the active grid . . . 41
5.8.2 Rock properties and the array operators . . . 42

5.8.3 Faults . . . 42
5.8.4 Equilibration . . . 43
5.8.5 Wells and the schedule . . . 43
5.8.6 Keywords that are not interpreted . . . 44
5.8.7 Reading a deck before running it . . . 44
5.8.8 The Norne field model . . . 45
5.8.9 What a field deck may not contain . . . 46
5.9 Time stepping . . . 47
5.9.1 DTREPORT . . . 47
5.9.2 TEND . . . 47
5.9.3 DTMAX . . . 47
5.9.4 DTCONTROL . . . 48
5.9.5 DTKEEP . . . 48
5.9.6 OPMCONV . . . 49
5.9.7 NEWTONSTALLMAX . . . 50
5.9.8 LINESEARCH . . . 50
5.9.9 DTRELAX . . . 51
5.9.10 DTGROW . . . 51
5.9.11 MAXCHOP . . . 52
5.9.12 SUBSTEPS . . . 52
5.10 Non-linear solution control . . . 52
5.10.1 NEWTONMAX . . . 52
5.10.2 NEWTONTOL . . . 52
5.10.3 LINESEARCH . . . 53
5.10.4 DPMAX . . . 53
5.10.5 DSMAX . . . 53
5.10.6 PROPSARRAYS . . . 53
5.10.7 USEENDSCALE . . . 54
5.10.8 FIELDSATFUNC . . . 54
5.10.9 APPLEYARD . . . 54
5.10.10 JACOBIAN . . . 55
5.10.11 TCPRECISION . . . 55
5.10.12 TCVALIDATE . . . 56
5.10.13 FDEPS . . . 56
5.10.14 SCHEME . . . 56
5.10.15 AIM . . . 56
5.10.16 IMPLICIT . . . 57
5.10.17 AIMCFL . . . 57
5.11 Linear solver control . . . 58
5.11.1 KRYLOV . . . 58
5.11.2 RESTART . . . 58
5.11.3 ORTHOPASSES . . . 58
5.11.4 LINTOL . . . 58
5.11.5 LINMAXIT . . . 59
5.11.6 LINSOLVER . . . 59
5.11.7 GINKGOPREC . . . 60
5.11.8 PRECOND . . . 60
5.11.9 STAGE2 . . . 61
5.11.10 MIXED . . . 61
5.11.11 EMULATE . . . 61
5.11.12 BLOCKSPMV . . . 62

5.11.13 CUDAGRAPH . . . 62
5.11.14 AMGREUSE . . . 63
5.11.15 AMGREUSEGROWTH . . . 63
5.11.16 AMGFAST . . . 63
5.11.17 AMGMAXCOARSE . . . 64
5.11.18 AMGCOARSERELAX . . . 64
5.11.19 AGGLEVELS, AGGTRUNC, AGGPMAX . . . 64
5.11.20 AMGSEQTHRESH . . . 65
5.11.21 AMGNONGALERKIN . . . 65
5.11.22 AMGPRINT . . . 65
5.11.23 MGRRELAX, MGRSWEEPS, MGRINTERP, MGRSMOOTH, MGRSMOOTHITS . . . 65
5.12 Output control . . . 66
5.12.1 OUTDIR . . . 66
5.12.2 ECLOUT . . . 66
5.12.3 MAKECSV . . . 66
5.12.4 MAKESUMMARY . . . 66
5.12.5 MAKEVTK . . . 66
5.12.6 MAKEGASSMANN . . . 67
5.12.7 AUTOPLOT . . . 67
5.13 Compositional keywords . . . 67
5.13.1 ZMF . . . 67
5.13.2 TCRIT, PCRIT, ACF, VMOL . . . 67
5.14 Physical constants . . . 68
5.14.1 DARCY . . . 68
5.14.2 GRAVGRAD . . . 68
5.14.3 REFACTOR . . . 68

6 Numerical formulation **69**
6.1 Governing equations . . . 69
6.2 Spatial discretisation . . . 69
6.3 Well model . . . 69
6.4 Temporal discretisation and the Newton method . . . 70
6.4.1 Update limiting . . . 70
6.4.2 Convergence and failure . . . 70
6.5 The Tensor Core Jacobian . . . 70
6.6 Timestep control . . . 71
6.7 Adaptive implicit . . . 72
6.7.1 Motivation . . . 72
6.7.2 Formulation . . . 72
6.7.3 The switching criterion . . . 72
6.7.4 When the criterion is evaluated . . . 73
6.7.5 Measured behaviour . . . 73
6.8 The linear solver . . . 73
6.8.1 The character of the system . . . 73
6.8.2 Constrained pressure residual . . . 73
6.8.3 Hierarchy reuse . . . 74
6.9 Mixed precision . . . 74
6.9.1 Principle . . . 74
6.9.2 Emulated double precision . . . 74
6.9.3 Measured budget . . . 75

6.9.4 Mixed precision against all-FP64 . . . 75
6.10 Multigrid hierarchies . . . 75
6.10.1 The two hierarchies . . . 75
6.10.2 Automatic selection . . . 76
6.11 The ZLINE second stage . . . 76

7 Parallel execution **78**
7.1 Domain decomposition . . . 78
7.2 The halo exchange . . . 78
7.3 Scaling behaviour . . . 78
7.4 Multi-node execution . . . 79

8 Output files and post-processing **80**
8.1 ECLIPSE-format output . . . 80
8.2 Native output . . . 80
8.3 Reporting scripts . . . 80
8.4 Locating the output of a benchmark study . . . 81

9 Verification and benchmarks **82**
9.1 Approach . . . 82
9.2 Matching the formulations . . . 82
9.3 The former three-phase residual difference . . . 82
9.4 Performance on the verification case . . . 83
9.5 The size ladder . . . 83
9.6 SPE10 model 2 . . . 83
9.6.1 Results . . . 83
9.6.2 Configurations that were rejected . . . 84
9.6.3 Why the case is reported on one GPU . . . 84
9.7 Reproducing the benchmarks . . . 84

10 Norne validation and OPM parity **86**
10.1 Purpose and acceptance boundary . . . 86
10.2 Well responses against OPM Flow . . . 86
10.3 Reference case and reproducibility identity . . . 86
10.4 Cluster installation runbook . . . 90
10.4.1 Get the code . . . 90
10.4.2 Build or restore HYPRE . . . 90
10.4.3 Build or restore AmgX . . . 91
10.4.4 Configure and compile . . . 91
10.4.5 Fetch, run and compare Norne . . . 91
10.5 Schedule time semantics . . . 91
10.6 Metric Norne and the internal unit contract . . . 92
10.7 Connection model and reported phase rates . . . 92
10.7.1 Hydrostatic completion pressure . . . 93
10.7.2 Closed-form BHP for a rate target . . . 93
10.8 History RESV conversion: derivation from OPM . . . 93
10.8.1 Representative region state . . . 93
10.8.2 Effective dissolved and vaporised ratios . . . 93
10.8.3 Surface-to-reservoir matrix . . . 94
10.9 Why a zero phase target stops the whole well . . . 94
10.10PVTNUM parity across physics, controls and reporting . . . 94
10.11Group injection ceilings . . . 95

10.12 Derived well quantities and denominator policy . . . 95
10.13 How the report computes and displays R^2 . . . 95
10.14 Measured Norne result, version 2 . . . 96
10.14.1 What version 2 changed . . . 96
10.14.2 Water inventory . . . 97
10.14.3 Field-state agreement . . . 97
10.15 Per-well agreement . . . 97
10.16 Three-phase saturation mathematics used in Norne . . . 98
10.16.1 ECLIPSE-default oil relative permeability . . . 98
10.16.2 Endpoint scaling . . . 99
10.16.3 Carlson gas hysteresis . . . 99
10.17 Control modes and their connection coefficients . . . 99
10.17.1 Worked ORAT example . . . 100
10.17.2 Worked zero-target example . . . 100
10.17.3 Worked history RESV example . . . 100
10.18 A staged decomposition of simulator disagreement . . . 101
10.18.1 Level 0: identity and static model . . . 101
10.18.2 Level 1: prescribed schedule volumes . . . 101
10.18.3 Level 2: field conservation . . . 101
10.18.4 Level 3: field state . . . 101
10.18.5 Level 4: well mode and total rate . . . 101
10.18.6 Level 5: completion allocation . . . 101
10.18.7 Level 6: residual and Jacobian . . . 101
10.19 A robust well-validation scorecard . . . 102
10.19.1 Normalised RMSE . . . 102
10.19.2 Integral and cumulative error . . . 102
10.19.3 Active-time mask . . . 102
10.19.4 Event timing . . . 102
10.19.5 Field-weighted aggregation . . . 102
10.20 Reading nonlinear and runtime diagnostics . . . 102
10.21 Release promotion and branch discipline . . . 103
10.22 Provenance of the parity implementation . . . 103
10.23 Per-well triage matrix for the current report . . . 104
10.24 Planned experiments beyond the stable release . . . 105
10.24.1 Experiment A: active-period metrics only . . . 105
10.24.2 Experiment B: injection ledger . . . 105
10.24.3 Experiment C: selected completion ledger . . . 105
10.24.4 Experiment D: reference-depth audit . . . 105
10.24.5 Experiment E: group replacement modes . . . 105
10.24.6 Experiment F: PINCH and transmissibility . . . 106
10.24.7 Experiment G: timestep parity as a diagnostic . . . 106
10.24.8 Experiment H: component mass-balance audit . . . 106
10.25 Published-result record template . . . 106
10.26 One-page Norne diagnosis decision tree . . . 107
10.27 Performance safeguards for parity work . . . 108
10.28 Regression and review matrix . . . 108
10.29 Code map for the parity implementation . . . 108
10.30 Limitations after the parity release . . . 109
10.31 Checklist before publishing a new Norne result . . . 109

11 Profiling **111**
11.1 Two tools, two questions . . . 111
11.2 Running a profile . . . 111
11.3 What reaches the report . . . 111
11.4 When profiling does not work . . . 112
11.4.1 nsys runs and collects no CUDA . . . 112
11.4.2 ncu fails to initialise . . . 112
11.4.3 Reading the diagnostics . . . 112

12 Tuning guide **113**
12.1 Method . . . 113
12.2 Interpreting the phase breakdown . . . 113
12.3 Reducing Krylov iterations per Newton . . . 113
12.4 Reducing the number of timesteps . . . 114
12.5 A worked example . . . 114

13 Troubleshooting **115**
13.1 The run stops . . . 115
13.1.1 Report step failed to converge after n timestep reductions . . . 115
13.1.2 Non-finite residual . . . 115
13.1.3 PERMFILE has fewer values than grid cells . . . 115
13.1.4 Compositional model requires a ZMF feed record . . . 115
13.2 The results are not as expected . . . 115
13.2.1 Water does not move . . . 115
13.2.2 Disagreement with another simulator in three phases . . . 116
13.2.3 Results changed after upgrading . . . 116
13.2.4 Material balance error grows . . . 116
13.3 Performance problems . . . 116
13.3.1 Many Krylov iterations per Newton iteration . . . 116
13.3.2 Several GPUs are slower than one . . . 116
13.3.3 The multigrid setup dominates . . . 116
13.3.4 The timestep collapses . . . 116
13.4 Library errors . . . 116
13.4.1 CUDA failure: out of memory during coarse point selection . . . 116
13.4.2 Incorrect AmgX configuration provided . . . 116
13.4.3 Invalid device ordinal from within the multigrid library . . . 116
13.5 Build problems . . . 117
13.5.1 HYPRE_DIR is empty . . . 117
13.5.2 No HYPRE.h under the given prefix . . . 117
13.5.3 Deck sets MIXED but the binary disagrees . . . 117
13.6 Every ablation and study . . . 117

A Example decks **119**
A.1 Two-phase waterflood . . . 119
A.2 Three-phase with tabulated saturation functions . . . 120
A.3 Reading an external rock model . . . 121

B Keyword index **122**

C Glossary **123**

D References **125**
D.1 Solver libraries 125
D.2 Method 125
D.3 Mixed precision and emulation 126
D.4 Cases and inversion 126

Chapter 1

Introduction

1.1 Purpose of this manual

This manual is the reference for using NVRS-CUDA. It describes the input deck keyword by keyword, the numerical methods those keywords control, the execution model on single and multiple GPUs, the files a run produces, and the diagnosis of runs that do not behave as expected.

The manual is organised so that it can be used in three ways. Chapters 2 and 3 are read once, when installing the simulator and preparing a first run. Chapters 4 and 5 are consulted repeatedly while building models; the keyword reference is the largest part of this document and is arranged by function, with an alphabetical index in Appendix B. Chapters 6 through 9 document the methods and the measurements behind the defaults, and are intended to be read before adjusting solver settings.

Every default value quoted in this manual is the value compiled into the simulator. Every performance figure quoted is taken from a run whose logs and reports are committed in the repository under `benchmark/`, and the case that produced it is named alongside the figure.

1.2 Overview of the simulator

NVRS-CUDA solves the equations of multiphase flow in porous media on structured Cartesian grids. Its distinguishing characteristic is that the dominant numerical work executes on the GPU: residual and Jacobian assembly, the linear solve, phase behaviour and property evaluation are implemented as CUDA kernels operating on device-resident state. The host coordinates the schedule and performs the small global well-control reduction after a Newton update. That control path reads the current state and completion records; it does not alter the reservoir residual or Jacobian kernels. Expensive immutable PVT flattening is cached for a run, and the field-region state used by history RESV is evaluated once per report step rather than once per Newton iteration.

This is a different design from a simulator that accelerates the linear solve and retains the remaining stages on the host. The consequence is measurable. On the verification case of Chapter 9, the entire non-linear side of NVRS-CUDA, comprising assembly, well handling, property updates and output, accounts for 2.5 s of wall clock where the corresponding host-side work in a reference simulator accounts for 85.6 s.

The linear solver is a two-stage constrained pressure residual (CPR) preconditioner applied within a flexible Krylov method, executed in mixed precision: the elliptic pressure stage runs in single precision, the coupled operator is applied in emulated double precision, and the state, residual and Krylov recurrences are held in native double precision. Chapter 6 develops this arrangement and the reasoning behind it.

1.3 Supported capabilities

Fluid models Two-phase black oil (oil and water), three-phase black oil (oil, water and gas) with dissolved gas, and compositional flow with an n_c-component equation of state.

Discretisation Structured Cartesian grids and corner-point geometry, with a two-point flux approximation, single-point upstream weighting of phase mobilities, and gravity in the phase potentials.

Field decks Corner-point grids are read directly in the format the industry's simulators share, with faults and the non-neighbour connections their throws create, region arrays, the deck array operators, equilibration from a datum by region, and well schedules whose controls vary with date. See Section 5.8.

Wells Peaceman inflow performance, with the well index formed from the directions perpendicular to the wellbore so that horizontal and vertical completions are treated correctly. Wells carry a control mode together with their limits and move between them as the limits bind.

Time integration Fully implicit or adaptive implicit, with automatic timestep selection, chopping on non-convergence and controlled growth.

Linear solution Mixed-precision CPR with two interchangeable algebraic multigrid libraries, hierarchy selection automated by rank count, and hierarchy reuse across Newton iterations.

Parallel execution MPI domain decomposition in three dimensions, one GPU per rank, with GPU-direct halo exchange, scaling from a single device to multiple nodes.

Output The ECLIPSE file family (EGRID, INIT, UNRST, SMSPEC, UNSMRY, RSM), comma-separated well responses, plain-text and JSON run summaries, and optional VTK field snapshots.

1.4 Scope and limitations

The following capabilities lie outside the current scope of the simulator. They are documented here so that they can be taken into account during model design and when selecting cases for comparison against other simulators.

Grid geometry Corner-point geometry, faults and the non-neighbour connections they create are supported (Section 5.8). Local grid refinement, radial grids and unstructured grids are not.

Reservoir models Dual-porosity and dual-permeability formulations, thermal effects, polymer and surfactant models, and geomechanical coupling are not implemented.

Well management Time-varying controls are read from a field deck's schedule, including opening and shutting wells and individual completions. Group control is implemented for producers: a group rate target stated by `GCONPROD` is shared among the wells that defer to it in proportion to their productivity, with the group hierarchy taken from `GRUPTREE`. Group injection ceilings from `GCONINJE` are implemented for surface-rate (`RATE`) control, separately by phase. Reservoir-rate (`RESV`) records are parsed but are not yet enforced. Reinjection (`REIN`) and voidage replacement (`VREP`) remain outside scope because their targets depend on production accumulated by the group. Tubing-head control and the VFP tables it requires, general multi-segment wells, and network balancing are not implemented. Completion cross-flow is represented by the signed Peaceman connection rates; an active zero rate target follows OPM and stops the whole well, so it cannot create cancelling inter-completion flow.

Saturation regions `SATNUM` is read but only the first `SWOF/SGOF` region is used. On a deck whose `SATNUM` varies spatially, every cell runs region 1's curves and the remaining regions are discarded. A run whose `SATNUM` takes more than one value now says so at start-up. Spatial variation of the curves is instead available through endpoint scaling, which is implemented and which most field decks use: see `ENDSCALE` and `SCALECRS`.

Relative permeability hysteresis Not implemented. The drainage curves are used throughout. `SATOPTS`, `EHYSTR`, `IMBNUM` and the imbibition endpoint arrays such as `ISGCR` are parsed and stored but nothing consumes them, and no per-cell saturation history is kept. A deck carrying `IMBNUM` now says so at start-up.

An earlier edition of this manual stated that Carlson non-wetting gas hysteresis was implemented. It was not, and never has been. Where a deck's imbibition table differs from its drainage table, this moves water cut and gas/oil ratio, so a comparison against a simulator that models hysteresis is not like for like on those quantities.

Norne is unaffected, which is worth recording because its deck looks as though it should be: `TABDIMS` declares two saturation tables and `SCAL_NORNE.INC` carries two `SWOF` and two `SGOF` blocks, but `SATNUM` is 1 in every cell, the second block is the imbibition table selected by `IMBNUM 2`, and that block is numerically identical to the first in all four columns.

Capillary pressure Capillary pressure *is* included in the phase potentials and in the Jacobian: the fourth column of `SWOF` gives $P_{cow} = p_o - p_w$ and that of `SGOF` gives $P_{cog} = p_g - p_o$, so $p_w = p_o - P_{cow}(S_w)$ and $p_g = p_o + P_{cog}(S_g)$. A table whose fourth column is absent, or is identically zero, costs nothing: the terms vanish and every expression reduces to what it was. Capillary *hysteresis* is not modelled.

Three-phase relative permeability Oil relative permeability in three phases uses the ECLIPSE/OPM saturation-weighted default. Stone's first and second models are not implemented; see Section 9.3.

Equilibration Hydrostatic equilibration from a datum is performed per region, with the phase columns anchored at the contacts. The contacts are sharp: a capillary transition zone is not spread across them. Phase gradients must be supplied, for the reason given in Section 5.8.

Connate water and the choice of equilibration record are taken per cell from the field deck's `SWL` and `EQLNUM` arrays only when `PROPSARRAYS` is set; otherwise every cell uses `SWCONNATE` and the first `EQUIL` record. The distinction is material rather than cosmetic (on Norne it decides whether three thousand cells carry a gas cap) which is why it is a stated choice and not a silent default.

Items in this list are limitations of the current release rather than consequences of the design, and several are under development.

1.5 Typographic conventions

`KEYWORD` Bold monospace denotes a deck keyword.

`value` Plain monospace denotes a keyword argument, a file name, a command, or an environment variable.

⟨ ⟩ Angle brackets in a syntax description enclose a quantity to be supplied by the user.

[ ] Square brackets in a syntax description enclose an optional element.

Syntax descriptions appear in a rule-marked block:

```
KEYWORD  <value>
```

1.6 Units

NVRS-CUDA solves in field units throughout, consistent with the ECLIPSE `FIELD` convention. What that means for input depends on which deck is being written.

The *run deck* (this simulator's own format, the file named on the command line) may be written in either, and says which with a bare `METRIC` or `FIELD` keyword on a line of its own. Field units are assumed if neither appears. A metric run deck is converted once on load: lengths and well radii to feet, pressures and capillary pressures to psi, compressibilities to reciprocal psi, densities from kg/m^3 to g/cc, solution gas-oil ratio from sm^3/sm^3 to scf/stb, well rates from sm^3/day to stb/day, and any equilibration gradients from bar/m to psi/ft.

What does *not* convert is worth stating, because it is the part that looks like an omission: permeability is millidarcy in both systems, viscosity is centipoise in both, formation volume factors are numerically the same (rm^3/sm^3 and rb/stb), times are days, and saturations, relative permeabilities and mole fractions are dimensionless. The solver's own constants (`DARCY`, `GRAVGRAD`, `REFACTOR`) are field constants by definition and are applied after the conversion, so they are left alone.

A rate of -1 in a `WELLS` record is the sentinel for "no rate target, run on bottom-hole pressure" and is not scaled; scaling it would turn a producer into an injector of -6.29 stb/day.

A *field deck* named by `FIELDDECK` may be in either system. Its `METRIC` or `FIELD` keyword is read, and a metric deck is converted on load: cell volumes and pore volumes to barrels, lengths and depths to feet, transmissibilities and well connection factors by m$^3 \to$ bbl over bar $\to$ psi, pressures to psi, and the PVT and saturation tables with them.

Well control rates are converted too, and by phase, which is the part worth stating because getting it wrong is silent. A rate of 12811 in a metric deck is 12811 sm^3/day; read as 12811 stb/day it is a sixth of the intended injection. Liquids convert to stb/day. Gas converts to *scf*/day and not Mscf/day, because the gas equation is written in scf (b_g is scf/rb and R_s is scf/stb) so a source term in Mscf/day enters a thousand times too small. Norne's C-4H asks for 392181 sm^3/day, which is 1.385×10^7 scf/day; entered as 1.385×10^4 it sits five orders of magnitude below the gas residual and the well does nothing at all.

Everything the run reports (the summary vectors, the restart files, the figures) is in field units regardless of what the field deck was written in. The deck's original system is remembered and stated in the run header.

Quantity	**Unit**	**Appears in**
Length	ft	`DX`, `DY`, `DZ`, `RW`
Pressure	psia	`PINIT`, `PBUB`, `PATM`, well BHP
Permeability	mD	`PERMFILE`, `PERMMIN`, `PERMMAX`
Porosity	fraction	`POROFILE`, `POROMIN`, `POROMAX`
Viscosity	cP	`UW`, `UO`, `UG`
Time	days	`DTREPORT`, `TEND`, `DTMAX`
Liquid rate	STB/day	injector rate, well output
Gas rate	MSCF/day	well output
Density	g/cm^3	`RHOW`, `RHOO`, `RHOG`
Oil, water FVF	RB/STB	`BO`, `BW`
Gas FVF	RB/MSCF	`BG`
Solution GOR	MSCF/STB	`RS`
Compressibility	1/psi	`CFW`, `CFO`, `CT`
Transmissibility	mD·ft	internal

Two dimensional constants follow from this system and are exposed as keywords rather than compiled in, so that a deck can document the convention it assumes: the Darcy transmissibility

constant $C = 1.127 \times 10^{-3}$ (`DARCY`) and the gravity gradient 0.4335 psi/ft per g/cm^3 (`GRAVGRAD`). The latter is the same constant as 1/144 psi/ft per lb/ft^3, since $62.428/144 = 0.4335$.

1.7 Related documents

`docs/cuda_reservoir_simulation.pdf` The mathematical development of the discretisation, the CPR preconditioner and the mixed-precision analysis.

`README.md` Repository overview, quick start and the measured results with the commands that produce them.

`benchmark/README.md` The verification procedure against a reference simulator, in full detail, including the decisions that make the two decks describe the same physical problem.

Chapter 2

Installation

2.1 Hardware requirements

NVRS-CUDA requires an NVIDIA GPU of compute capability 7.0 (Volta) or newer. Development, validation and all performance figures in this manual are on NVIDIA GB200 Grace-Blackwell hardware (`sm_100`).

Device memory determines the largest model that can be run on a given number of GPUs. As a working estimate, a three-phase black-oil model requires approximately 1.4 kB of device memory per active cell for the coupled system, its single-precision mirror, the multigrid hierarchy and the Krylov basis at the default restart length. A five-million-cell model therefore requires on the order of 7 GB per GPU when run on a single device, and correspondingly less per device when decomposed.

2.2 Software requirements

CUDA Toolkit Version 12.0 or newer, providing `nvcc`, cuSPARSE, cuBLAS and cuSOLVER.

MPI Any MPI-3 implementation. A CUDA-aware implementation is strongly recommended: without one, every halo exchange and every multigrid communication stages through host memory, and multi-GPU runs may be slower than single-GPU runs. The run banner reports whether the MPI in use is CUDA-aware.

HYPRE Built with CUDA support and, for the mixed-precision configuration, in single precision. The relevant configure options are listed below.

AmgX Optional but recommended, and the source of the default stage-1 multigrid. Without it the simulator builds and runs using HYPRE and reports this at start-up.

Python Version 3.8 or newer with NumPy and Matplotlib, for the deck generators, reporting and plotting scripts.

The HYPRE configuration required by the mixed-precision build is:

```
--enable-single                     # single-precision library
--with-cuda                         # CUDA support
-DHYPRE_ENABLE_GPU_AWARE_MPI=ON     # device-pointer MPI
```

The last of these is important on more than one GPU. Without it every multigrid halo exchange is staged through host memory, at every level of the hierarchy and on every cycle; the cost is invisible on a single rank, where no exchange occurs, and can make a multi-GPU run slower than a single-GPU run. The run banner reports the setting that the linked library was built with.

2.3 Building with Make

The dependency installers write environment scripts that export the paths required by the build. Source both, then build:

```
source ../../amgx-install/amgx_env.sh                        # sets AMGX_DIR
source ../../../mixed_precision_emmulation/hypre_env.sh      # sets HYPRE_FP32_DIR
make -j
```

The build reports the AmgX installation it located, or states that none was found and that the simulator will use HYPRE only. The executable is written to `./nvrs_cuda`.

If `HYPRE_DIR` is unset the build stops with an explanatory message rather than proceeding to a link error; this is the usual symptom of a new shell in which the environment scripts have not been sourced.

2.4 Building with CMake

```
cmake -S . -B build \
      -DHYPRE_DIR=$HYPRE_FP32_DIR \
      -DAMGX_DIR=$AMGX_DIR \
      -DCMAKE_CUDA_ARCHITECTURES=100 \
      -DNVRS_MIXED_PRECISION=ON
cmake --build build -j
```

The executable is written to `build/nvrs_cuda`. Where both build systems have been used, the helper scripts select the more recently built executable and report which one they are running, so that a rebuilt binary is never silently ignored.

> **Note. Make and CMake produce different binaries.** `make` writes `cuda/nvrs_cuda`; `cmake --build build` writes `cuda/build/nvrs_cuda`. Every script in `benchmark/` runs the first, and prints `note : ignoring older .../cuda/build/nvrs_cuda` when the second exists. A CMake-only build therefore leaves the harness measuring whatever `make` produced last, or nothing at all.
>
> The two also locate AmgX differently: CMake is handed `-DAMGX_DIR` on the command line, while the Makefile reads `AMGX_DIR` from the environment. One build can have AmgX and the other not, from the same source tree. If you build both, build with `make` last.

Whichever is used, one line of the build output decides whether the timings that follow mean anything:

```
AmgX: /path/to/amgx-install          # good
AmgX: not found -- HYPRE only        # stop here
```

A missing AmgX is not an error. The build continues, the run prints one warning, and the benchmark then measures BoomerAMG under a label that says AmgX. The usual cause is a prefix holding `lib/libamgxsh.so` without `include/amgx_c.h`: AmgX's own install step has historically omitted the headers, which is why `install_amgx.sh` copies them itself and refuses to finish without them. A prefix in that state was never completed, and `FORCE_AMGX=1 ./benchmark/install_all.sh` rebuilds it.

2.5 Build options

Option	Default	Effect
`NVRS_MIXED_PRECISION`	`ON`	Builds the CPR preconditioner in single precision. Must match the precision of the HYPRE library being linked.
`CMAKE_CUDA_ARCHITECTURES`	detected	Target architecture: 70 for V100, 80 for A100, 90 for H100, 100 for GB200, 103 for VR72. Detected from the device when not given; see Section 2.6 for what happens when the toolkit has no target for the device.
`CUDA_ARCH`	`auto`	Same, for the Make build. A number pins the architecture and skips detection, which is what to use when cross-building for a machine you are not sitting on.
`FP64_PRECOND`	`0`	Make build only. `1` builds the all-FP64 binary `nvrs_cuda_fp64` (preconditioner in FP64, AmgX mode `dDDI`, linked against the FP64 HYPRE), with its objects in `build_fp64/`, beside the default mixed-precision `nvrs_cuda`.
`AMGX_DIR`	*unset*	AmgX installation prefix. When absent, AmgX support is compiled out.
`HYPRE_DIR`	*unset*	HYPRE installation prefix. Required.

Precision is a build-time property because it determines the storage type of the preconditioner throughout the code and the HYPRE library it links against. The installer therefore builds two binaries: `nvrs_cuda`, the mixed-precision default (FP32 preconditioner, AmgX `dFFI`), and `nvrs_cuda_fp64`, the all-FP64 build (`make FP64_PRECOND=1`; `SKIP_FP64=1` skips it). AmgX needs no separate build: one library contains every mode, and the binary selects the mode. HYPRE does need one build per precision; the installer builds both.

The deck keyword `MIXED` chooses between them. Every launcher (`nvrs.sh`, `run_comparison.sh`, `scripts/run.sh` and the scaling and SPE10 scripts) reads the last `MIXED` line of the deck and starts `nvrs_cuda_fp64` for `MIXED 0` and `nvrs_cuda` otherwise, stopping with the build command if the requested binary is missing. A binary started by hand with a deck that disagrees with it reports the discrepancy, names the binary to use, and proceeds with the compiled behaviour.

2.6 Building for a GPU newer than the toolkit

A GPU can report a compute capability that no released CUDA toolkit is able to target. VR72 reports 10.7, and the toolkits available at the time of writing list 100, 103, 110, 120 and 121, with nothing at 107. Passing the reported capability straight to the compiler produces:

```
nvcc fatal : Unsupported gpu architecture 'compute_107'
```

This is not a defect in the driver or the device, and it is not worked around by choosing the newest architecture the toolkit knows. Two rules govern what will run:

- A *cubin*, produced by `code=sm_NN`, runs at an equal or higher *minor* revision within the same major capability, and never at a lower one. An `sm_103` cubin therefore loads on a 10.7 device.

- *PTX*, produced by `code=compute_NN`, is compiled by the driver when the module loads, targeting the actual device. It is compiled forward onto newer hardware only, never back onto older.

Both rules point the same way: the architecture to build for is the newest one the toolkit knows that is *not newer than* the device. Taking the toolkit's newest instead (`compute_121` for the example above) produces a binary that cannot run on the device at all.

The build resolves this automatically, in the Make and CMake paths alike and in every dependency install script. It intersects `nvcc --list-gpu-arch` with the capability `nvidia-smi` reports. On an exact match it emits a cubin for that architecture and nothing else. Otherwise it emits *both* a cubin and PTX for the newest architecture at or below the device:

```
-gencode arch=compute_103,code=sm_103       # loads directly, no compilation
-gencode arch=compute_103,code=compute_103  # fallback, compiled at load
```

The cubin is what avoids the load-time cost; the PTX is retained so that a device whose compatibility rules do not cover the gap still runs, by the driver compiling it. If the device is *older* than every architecture the toolkit supports, no target can run and the build stops with a message naming both lists, rather than producing a binary that fails at its first kernel launch.

2.6.1 Consequences at run time

Where PTX is used, every kernel is compiled by the driver the first time it is launched. This is a one-off cost, but a substantial one: a Norne report step that averages 1.5 s took 63.5 s when it was the first. The driver caches the result, and the benchmark scripts point `CUDA_CACHE_PATH` at `runs/.jitcache` with a 4 GiB `CUDA_CACHE_MAXSIZE` so the cache sits on the mounted filesystem and survives the container. A cache that is too small evicts and recompiles indefinitely; `CUDA_CACHE_DISABLE` set to `1` in the environment disables it entirely, and the scripts report this rather than obeying it silently.

Timings taken where code is compiled at load are not comparable with timings from a natively built binary. The kernel *mix* in a profile remains valid; the wall clock does not.

2.7 Installing the dependencies without root

Inside a container under a batch scheduler the user is frequently not root and cannot become one: `dpkg` refuses with *requested operation requires superuser privilege*, and `sudo` is often not installed setuid. Package installation is therefore unavailable, but package *retrieval* is not. `apt-get download` needs no privilege, and `dpkg-deb -x` unpacks into any directory the user owns.

`install_opm_gpu.sh` uses this for everything OPM Flow needs that the image lacks (Boost, SuiteSparse, ZOLTAN, METIS, ParMETIS, Scotch, SuperLU, GMP) resolving the dependency closure with `apt-cache depends --recurse` and unpacking only what `dpkg` reports as not already installed. DUNE is built from source into the same prefix OPM installs to, because the packaged version is older than OPM requires. `install_nsys.sh` does the same for Nsight Systems.

Two rules make this safe, and both were learned by breaking them:

- **Never unpack the MPI stack.** A source-built Open MPI is normal on these machines, and `dpkg` has no record of it, so a dependency resolver will conclude that `libopenmpi`, `libpmix`, `libhwloc` and `libevent` are missing and fetch the distribution's copies. Those then shadow the real ones and the launcher loads a runtime that is not its own, after which it rejects every argument it has, including `--version`. The symptom resembles a launcher with unusual syntax and is nothing of the kind.

- **Append such a prefix to `LD_LIBRARY_PATH`, never prepend it.** It exists to supply libraries nothing else provides, so the end of the search path is the correct place for it. Prepending can only shadow.

2.8 Installing everything in one command

```
cd benchmark && ./install_all.sh
```

This runs HYPRE, then AmgX, then OPM Flow with its dependencies, then Nsight Systems, then the simulator itself, in that order. The order matters: each installer writes an environment file naming the prefix it created, and the script sources that file before the next step. `hypre_env.sh` provides `HYPRE_FP32_DIR` and puts the matching `mpicxx` on the path, which AmgX is then configured against; `amgx_env.sh` provides `AMGX_DIR`. Both are sourced whether or not their step actually ran, so a resumed run has the same environment as a fresh one.

A step whose output already exists is skipped, and a step that fails does not stop the others: a missing AmgX still leaves a working HYPRE build. `SKIP_HYPRE`, `SKIP_AMGX`, `SKIP_OPM`, `SKIP_NSYS` and `SKIP_BUILD` disable individual steps; the corresponding `FORCE_...` variables rebuild something already present.

2.9 One command for the whole workflow

`install_all.sh` installs. Everything after it was a sequence of commands to be remembered in the right order: fetching a dataset, running both simulators, profiling each backend, drawing the figures, folding them into the report, with the step most easily forgotten being the one that differs between cases. `benchmark/nvrs.sh` is that sequence behind one entry point. It replaces none of the scripts below; it calls them, and knows which one a given case needs.

```
cd benchmark
./nvrs.sh                  # what is installed, what is ready, what to type
./nvrs.sh install          # HYPRE, AmgX, OPM Flow, Nsight, nvrs_cuda and nvrs_cuda_fp64
./nvrs.sh all NORNE        # fetch it, run it, profile it, report it
```

The last command ends at `runs/compare_NORNE.html`: NVRS-CUDA against OPM Flow, the field images, the animation, and the Nsight figures for both multigrid backends, on one page.

2.9.1 Commands

Command	What it does
`install`	HYPRE, AmgX, OPM Flow, Nsight Systems and `nvrs_cuda`. Each step is skipped when its output is present, so it resumes after a failure rather than starting over.
`cases`	Every case, whether its deck is ready, whether it can be run against OPM Flow, and how its deck is prepared.
`get <CASE>`	Fetches or generates the deck and any data it needs.
`run <CASE>`	Both multigrid backends, and OPM Flow where a case has an ECLIPSE deck.
`profile <CASE>`	Nsight Systems, once per backend.
`report <CASE>`	The figures, folded into the HTML report.
`all <CASE>`	All four of the above, in order. Several cases may be named and each is run in turn.

Preparing a deck means three different things and that is the distinction the driver exists to absorb: Norne is downloaded, SPE10 is downloaded and then converted, and the `BENCH` family is generated from parameters. A case already sitting in `decks/` needs nothing.

2.9.2 Options

Every option is a flag and also an environment variable, the variables having come first. A flag is what `--help` can list, and each command prints the options it accepts as it starts, so the list below is a reference rather than something to memorise.

Option	Applies to	Meaning
`--no-opm`	run, all	NVRS-CUDA only; do not run OPM Flow.
`--backend <b>`	run, profile	`amgx`, `hypre` or `both`; the default is both.
`--ranks <n>`	run, profile	MPI ranks for NVRS-CUDA.
`--no-scaling`	run, all	One GPU only; skip the strong-scaling sweep.
`--scaling`	run, all	Force the sweep on.
`--no-profile`	all	Skip the profiling step.
`--ncu`	profile, all	Add Nsight Compute counters, roughly 100× slower per kernel.
`--fraction <f>`	profile, all	How much of the history to profile: `0.25`, or `all`. The default is a tenth.
`--tend <days>`	profile, all	Profile exactly this many days instead.
`--steps <n>`	profile, all	Norne only: report steps, or `all`.
`--heartbeat <s>`	profile, all	Progress interval; `0` silences it.
`--deck <path>`	get, run, all	A deck of your own, for a case not under `decks/`.

`--steps` is refused outside Norne rather than accepted and ignored. A report step is a fortnight there and something else in every other deck, so it cannot mean anything general; `--fraction` and `--tend` are the case-independent forms.

The strong-scaling sweep repeats a run on 2, 4 and however many GPUs are present. That is the point of the `BENCH` family, which is small and exists partly to measure how the decomposition holds up. It is the wrong default for SPE10. That case is 1.12 million cells chosen to stress the solver, and each extra rank repeats a long run to answer a question it was never selected to ask, so SPE10 defaults to a single GPU and says so when it starts.

2.9.3 The help screen

Typed without arguments, or with `--help`, the driver prints everything above and then what it can see on this machine: whether `nvrs_cuda` is built, how many decks are ready, and the most recent report. That last block is omitted here because it describes a particular machine rather than the software.

```
==========================================================================
 NVRS-CUDA  --  GPU reservoir simulation, against OPM Flow
==========================================================================

  USAGE
    ./nvrs.sh <command> [CASE] [options]

  COMMANDS
    install                HYPRE, AmgX, OPM Flow, Nsight and nvrs_cuda
    cases                  every case, and whether its deck is ready
    all <CASE>             get + run + profile + report, in one command
    get <CASE>             fetch or generate the deck and its data
```

```
    run <CASE>              the backends, against OPM Flow
    profile <CASE>          Nsight Systems, once per backend
    report <CASE>           the figures, folded into the HTML

  OPTIONS
    OPTION              APPLIES TO       MEANING
, , , , , -, , -
    --no-opm            run, all         NVRS only; do not run OPM Flow
    --backend <b>       run, profile     amgx, hypre, or both (default both)
    --ranks <n>         run, profile     MPI ranks for NVRS
    --no-profile        all              skip the profiling step
    --no-scaling        run, all         one GPU only; skip the 2/4-GPU sweep
    --scaling           run, all         force the sweep on (SPE10 is off)
    --ncu               profile, all     add Nsight Compute counters (~100x slower)
    --fraction <f>      profile, all     history to profile: 0.25, or all (default 0.1)
    --tend <days>       profile, all     profile exactly this many days instead
    --steps <n>         profile, all     NORNE only: report steps, or all
    --heartbeat <s>     profile, all     progress interval, 0 to silence
    --deck <path>       get, run, all    your own .DATA, for a case not under decks/
    -h, --help          anywhere         this

    Every command prints the options it takes when it starts, so this
    list is a reference rather than something to memorise.

  FROM NOTHING TO A REPORT
    ./nvrs.sh install
    ./nvrs.sh all NORNE
    ./nvrs.sh all NORNE SPE10 BENCH3P    (several, in order)

  COMMON VARIATIONS
    ./nvrs.sh all NORNE --no-opm              skip Flow, NVRS only
    ./nvrs.sh all NORNE --backend amgx        one backend, faster
    ./nvrs.sh all NORNE --steps all           the whole 247-step history
    ./nvrs.sh all SPE10 --fraction 0.5        half of SPE10's history
    ./nvrs.sh all MYFIELD --deck /path.DATA   a deck of your own
    ./nvrs.sh profile NORNE --ncu             add the kernel counters
    ./nvrs.sh report NORNE                    rebuild the HTML only
```

2.9.4 What it checks before running

Two failures used to arrive only after a deck had been fetched and OPM Flow had run, and both are now caught first.

The binary must be able to load. `hypre_env.sh` deliberately keeps HYPRE off `LD_LIBRARY_PATH`, because the FP32 and FP64 builds share a soname and a search path could load the wrong precision; the link uses an rpath instead. When that rpath does not resolve the run dies at the dynamic linker, so `ldd` is consulted first, the FP32 prefix the build was configured with is added if it helps, and the run stops with the missing libraries named if it does not.

The binary must contain the backend that was asked for. A build without AmgX falls back to HYPRE with a warning, which is a silent change to what is being measured: on SPE10 that fallback turned a 100-day timestep into 571 Newton iterations and 60,262 Krylov iterations. When the deck names `LINSOLVER AMGX`, or `--backend amgx` is given, and the binary has no AmgX, the run stops rather than producing numbers under the wrong label.

2.10 The browser interface

`gui/server.py` is a local server, Python standard library only, and `gui/index.html` is its page. From the project folder:

```
python3 gui/server.py
```

and open `http://127.0.0.1:8765`. Inside the container the same port is published to the host. Over SSH, forward it first with `ssh -L 8765:127.0.0.1:8765 user@machine`. The Build tab runs `./nvrs.sh install`. Run a deck takes a deck from `decks/`, adds the keywords you choose, shows the exact command, then runs it through `mpirun` with the log streamed to the page. Benchmark cases drives `./nvrs.sh`. Results shows the GB200 and VR72 reports and movies. Runs are written under `benchmark/runs/gui/`. Opened directly from disk, the page shows the method, results and manual but cannot start work.

2.11 Reproducing a result

One command per case fetches or generates the deck, builds if needed, runs OPM Flow and the simulator, profiles with NSIGHT SYSTEMS and writes the report to `benchmark/runs/compare_CASE.html`.

```
cd benchmark
./nvrs.sh all NORNE
./nvrs.sh all SPE10
./nvrs.sh all BENCH3P
```

The reports measured on GB200 and VR72 are in `results/GB200/` and `results/VR72/`.

Note. A run reports the work it threw away. Every run now ends with a line like

```
discarded: 476 Newton and 7936 linear iteration(s) on 71 of 247 report
step(s) that were retried; this work is in the wall clock below but not
in the per-step counts
```

This is not double counting. The per-step table reports the attempt that succeeded, which is what a per-step table should show; the wall clock beside it contains every attempt, including the ones that were retried shorter. Before this line existed the two quantities were reported side by side with nothing saying they measured different things, and any figure derived by dividing one by the other mixed them.

2.12 Verifying the installation

```
mpirun -np 1 ./nvrs_cuda decks/SMOKE.DATA
```

`SMOKE.DATA` is a small two-phase case that exercises the complete path, from deck parsing through assembly, linear solution, timestep control and output, and completes in a few seconds. A successful run ends with a timing summary and a non-zero set of output files under the directory named by `OUTDIR`.

The analytic Jacobian can be verified against a finite-difference approximation of the same residual:

```
mpirun -np 1 ./nvrs_cuda decks/SMOKE.DATA --jac-check
```

This assembles both forms, reports the maximum relative difference between them, and exits without advancing the simulation. The two must agree to the order of the finite-difference truncation error; a larger discrepancy indicates an inconsistency between the residual and its derivatives.

Chapter 3

Running the simulator

3.1 Command-line syntax

```
mpirun -np <ranks> ./nvrs_cuda <deck> [--jac-check]
```

`<deck>` Path to the input deck. Required.

`--jac-check` Compare the analytic and numerical Jacobians and exit, as described in Section 2.6.

Relative paths inside the deck, such as `PERMFILE`, are resolved against the current working directory rather than the deck location. Run from the directory the deck was written for; the supplied benchmark decks assume the `cuda/` directory.

3.2 Device binding

Each rank binds to one GPU according to its node-local rank. The binding is established before any library initialises a CUDA context, which is necessary because both HYPRE and the CUDA runtime cache device state at first use. The banner records the binding for every rank:

```
[rank 0/4] host local_rank=0/4 -> GPU 0 of 4 (NVIDIA GB200, sm_100)
```

Requesting more ranks than there are visible devices causes ranks to share devices. This is reported as a warning; it is supported for debugging but will not perform well.

3.3 The run banner

The banner is the authoritative record of the configuration a result was produced with. Several properties are compile-time or determined automatically, and the banner is the only place they are stated. A typical banner for a single-GPU run reads:

```
  Launched : <date and time of the run>
  Host     : gpu-node
  Parallel : 1 MPI rank, one GPU per rank
  Precision: FP64 outer Krylov, FP32 CPR preconditioner
  AmgX     : classical PMIS hierarchy (automatic: strongest on one rank)
  ========================================================
    NVRS-CUDA  -  SPE10
, , , , , , , , , , , , , , , , , , , ,
    Model        : BLACK-OIL (2-phase)
    Grid         : 60 x 220 x 85  (1122000 cells)
    Cell size    : 20 x 10 x 2 ft
    Solver       : CPR  +  FGMRES  [mixed FP32 AMG, df64-emulated operator]
```

```
  Wells        : 1 injectors, 4 producers
  Time         : 0 -> 2000 days, report every 100 days
=========================================================
  Newton : tol 5.00e-03 (relative), max 20 iterations/step
  Linear : tol 1.00e-02, max 200 iterations, FGMRES restart 30
  Stage 2: block-Jacobi on each cell block (STAGE2 JACOBI)
  Scheme : ADAPTIVE IMPLICIT (AIM) - a cell runs explicit in saturation
           below throughput CFL 0.5; wells and fronts stay implicit.
```

The banner describes only the multigrid library actually in use. A run using AmgX reports the AmgX hierarchy and the reason it was selected, and reports nothing about HYPRE; a run using HYPRE reports the HYPRE build configuration and its kernel settings, and reports nothing about AmgX.

3.4 Progress output

One line is written per report step:

```
  step  time[d]    |R|_start      |R|_end       rel  newt krylov wall[s]
, , , , , , , , , , , , , , , , , , , , , , , , , , , , , , , , , , , ,
  1       100.0   2.3203e+01   5.0246e-02  2.17e-03   354  14163    46.0
```

`|R|_start` Residual norm at the first Newton iteration of the report step.

`|R|_end` Residual norm at convergence of its final sub-step.

`rel` Ratio of the two, to be compared against `NEWTONTOL`.

`newt`, `krylov` Newton and Krylov iterations accumulated over the report step, including any discarded attempts.

`wall` Cumulative wall-clock time since the start of the run.

Two classes of informational message are interleaved with this table:

`[chop]` The report step did not converge at the current timestep; the state has been restored and the step is being retried at half the length.

`[dt]` The report step converged, with the timestep at which it did so and the length the next step will start from.

`[aim]` The proportion of cells running explicit in saturation for the coming sub-step, with the timestep and threshold that produced it.

3.5 Timing summary

At the end of a run:

```
simulation finished in 342.12 s (17.11 s per report step)
AMG setup: 538 of 1989 rebuilt (1451 reused, 471 cut short by the growth
           guard), 25.5 ms each, 13.74 s total
Linear   : AMG V-cycles 191.34 s (199985 applies, 0.96 ms each),
           operator+halo 40.60 s, stage-2 24.57 s, CGS2 67.37 s
```

The linear-solver breakdown separates the multigrid application, the operator application including halo exchange, the second CPR stage and the Krylov orthogonalisation. These are the quantities to consult before adjusting solver settings; Section 12 describes how to read them.

3.6 Selecting the number of ranks

Additional GPUs do not automatically reduce run time. Decomposition introduces halo exchanges and, more significantly, distributes the multigrid hierarchy, whose coarse levels become dominated by communication latency rather than arithmetic.

The practical guideline is to maintain at least 5×10^5 cells per GPU. Below that threshold a single GPU is frequently the fastest configuration. Chapter 7 presents the measurements supporting this guideline and explains the mechanism.

3.7 The MPI launcher

`nvrs_cuda` is always started under MPI, including when profiling. A bare launch does not merely fail: a singleton `MPI_Init` under Slurm's PMIx can terminate the entire job step,

```
pmixp_client_v2.c:225: Error handler invoked: status = -61
*** STEP ... CANCELLED ... DUE to SIGNAL Killed ***
```

which takes anything else running in that allocation with it.

Which `mpirun` is used is not a formality either. `--oversubscribe` and `--allow-run-as-root` are Open MPI spellings and are rejected outright by other implementations, and the first `mpirun` on `PATH` is not necessarily the MPI the binaries were linked against, a module system can place its own ahead of the container's. The benchmark scripts therefore prefer `$OMPI_HOME/bin/mpirun`, fall back to `/opt/openmpi` and then to `PATH`, and add the Open MPI options only after confirming that the launcher reports itself as Open MPI. `MPIRUN` overrides the choice entirely.

3.8 Environment variables

Deck keywords describe the physical model and the numerical method. Environment variables cover machine-level configuration and experiments that should not require modifying a deck.

Variable	Effect
`NVRS_DECOMP`	Forces the process grid, given as `"px py pz"`. Ignored unless the product equals the rank count and each factor is no greater than the corresponding grid dimension.
`NVRS_AMGX_ALGO`	`CLASSICAL` or `AGGREGATION`; overrides the automatic hierarchy selection described in Section 6.10.
`NVRS_AMGX_CONFIG`	Path to an AmgX configuration file, replacing the built-in configuration entirely. Takes precedence over `NVRS_AMGX_ALGO`.
`NVRS_AMGX_COMM`	AmgX communicator, default `MPI_DIRECT`. Set to `MPI` to use host-buffered exchanges.
`NVRS_RESLOG`	A directory. Rank 0 writes one CSV row per Newton evaluation (`newton.csv`: step, substep, iteration, $\|R\|$, relative residual, MB, CNV, accepted) and per linear solve (`linear.csv`: Krylov iterations, final relative residual, converged, $\|\Delta u\|$). The values are already on the host, so the cost is one line per iteration. Used by `precision_fp64_vs_mixed.sh` to compare convergence.
`NVRS_GINKGO_SWEEPS`	Ginkgo pre- and post-smoothing sweeps, default `1`.
`NVRS_GINKGO_RELAX`	Ginkgo Jacobi relaxation factor, default `0.9`.
`NVRS_GINKGO_COARSE_ITERS`	Jacobi sweeps on the coarsest Ginkgo level, default `10`.

Variable	Effect
NVRS_GINKGO_MAXLEVELS	Maximum Ginkgo hierarchy depth, default 24.
NVRS_GINKGO_MINCOARSE	Ginkgo stops coarsening below this many rows, default 32.
NVRS_GINKGO_CYCLE	V (default), W or F.
NVRS_GINKGO_S2_ACCURACY	Relative accuracy that drives the per-block precision choice of STAGE2 GINKGO, default 0.1.
NVRS_GINKGO_S2_PREC	64, 32 or 16 stores every STAGE2 GINKGO block in that precision instead of choosing per block.
NVRS_GINKGO_NVTX	1 attaches Ginkgo's NVTX ranges, so an Nsight Systems trace names its operations.
NVRS_HYPRE_SPMV	Set to 1 to use cuSPARSE for HYPRE's sparse matrix-vector products. The default, 0, uses HYPRE's own kernels, which are faster in this application.
NVRS_HYPRE_SPGEMM	As above, for the Galerkin triple products.
NVRS_HYPRE_GPUMPI	Set to 0 to disable HYPRE's GPU-aware MPI at run time.
NVRS_VERBOSE	Enables a per-iteration trace: linear residual, iteration count and update norm for every Newton iteration.
NVRS_DEVICE	Pins all ranks to a single device. Intended for debugging.
NVRS_PYTHON	Python interpreter used by the plotting hooks.
NVRS_PLOT_SCRIPT	Alternative plotting script.
NVRS_PROFILE	1 for an Nsight Systems timeline alongside the benchmark, both to add Nsight Compute's counters, ncu for the counters alone. See Chapter 11.
MPIRUN	Launcher to use, overriding the search described in Section 3.7.
NSYS, NCU	Paths to specific Nsight binaries, when the copy bundled with the toolkit is not the one wanted.
CUDA_ARCH	Pins the GPU architecture for the install scripts; see Section 2.6.
CUDA_CACHE_PATH	Where the driver keeps compiled PTX. Defaults to runs/.jitcache so that it survives the container.

3.9 Utility scripts

Script	Purpose
scripts/run.sh	Runs one deck with the standard output layout and post-processing.
scripts/run_scaling.sh	Sweeps rank counts for one deck, writes scaling.json and prints a per-phase timing table. Controlled by GPUS, STEPS, GRID and EXTRA.
scripts/backend_study.sh	The complete benchmark suite across cases, backends and rank counts, regenerating the HTML report.
scripts/spe10_tune.sh	Baseline-relative A/B harness for solver options, one variable per run.
scripts/plot_results.py	Renders well curves, field slices and animations from a completed run.
scripts/viz3d.py	Three-dimensional field rendering.

Script	Purpose
`benchmark/make_decks.py`	Generates matched NVRS-CUDA and ECLIPSE decks for the verification cases from a single parameter set.
`benchmark/make_spe10.py`	Retrieves the public SPE10 model 2 dataset and generates its deck.
`benchmark/compare.py`	Compares two ECLIPSE-format result sets and renders a self-contained HTML report.
`benchmark/nvrs.sh`	The whole workflow behind one entry point (§2.9): install, get, run, profile, report.
`benchmark/profile_case.sh`	Profiles any deck with Nsight Systems, one output directory per backend.
`benchmark/profile_norne.sh`	The Norne wrapper over the above, keeping `PROF_STEPS` for the one deck whose step length is known.
`benchmark/profile_plots.py`	Draws the profile figures: the CPR split, the Nsight timeline, the share of GPU time, and the Nsight Compute roofline and limiter planes when counters exist.
`benchmark/profile_topk.py`	Prints the kernels that dominate, straight after a profile.

3.10 Exit status

Status	Meaning
0	The simulation completed to `TEND`.
1	An error was raised during set-up or execution; the message precedes the exit.
2	A report step failed to converge within the permitted number of timestep reductions.

Under MPI a non-zero status on any rank aborts the job, so a failure is reported once by the failing rank and the remaining ranks are terminated.

Chapter 4

Deck structure

4.1 File format

An input deck is a plain text file. Each keyword appears at the start of a line and is written in upper case. Two forms exist.

Scalar keywords take a single value on the same line:

```
NX       60
DTREPORT 100
LINSOLVER AMGX
```

An equals sign between keyword and value is accepted and ignored, so `NX = 60` is equivalent. Quoted values have their quotes removed.

Record keywords introduce a list of records, terminated by a line containing a single slash. A trailing slash on the final record is also accepted:

```
WELLS
  30 110   1   5000   1.0   0.0  10000   all INJ
   1   1  -1     -1   1.0   0.0   4000   all P1
/
```

The record keywords are `WELLS` (and its synonym `WELSPECS`), `SWOF`, `SGOF`, `ZMF`, `TCRIT`, `PCRIT`, `ACF` and `VMOL`.

4.2 Comments and whitespace

A double hyphen introduces a comment, which extends to the end of the line. Comments may follow a value on the same line. Blank lines are ignored. Leading whitespace is not significant.

```
-- Grid definition
NX  60        -- cells in the i direction
```

4.3 Keyword ordering

Keyword order is not significant, with one exception: `NCOMP` must appear before `MODEL`, because the model resolves to the two-phase or three-phase black-oil formulation according to the component count. Placing `NCOMP` first in the deck, as in the supplied examples, avoids the question entirely.

Unrecognised keywords are ignored without a diagnostic. This permits a deck written for a later version of the simulator to be read by an earlier one, and it means that a misspelled

keyword takes its default value silently. The run banner reports the values in effect for the principal settings, and is the recommended check after editing a deck.

4.4 A minimal deck

The smallest deck that runs specifies the model, the grid, the schedule and at least one well. All other keywords take their defaults.

```
NCOMP    2
MODEL    BLACKOIL
NX       40
NY       40
NZ       3
DTREPORT 100
TEND     1000
WELLS
  10 10   1   500   0.5  0.0  4000  all I1
  30 30  -1    -1   0.5  0.0  2800  all P1
/
```

4.5 Deck sections

NVRS-CUDA does not require the `RUNSPEC`, `GRID`, `PROPS`, `SOLUTION` and `SCHEDULE` section headers used by ECLIPSE-format decks. Such headers, if present, are ignored as unrecognised keywords. The supplied decks group keywords under comment banners in the same order for readability, and this manual documents the keywords in that grouping.

Chapter 5

Keyword reference

This chapter documents every keyword recognised by the simulator, grouped by function. Each entry gives the syntax, the default value, a description of the effect, and, where the behaviour warrants it, an example and cross-references. An alphabetical index appears in Appendix B.

5.1 Model definition

5.1.1 NCOMP

```
NCOMP  <integer>
```

Default: 3.

Values:

`2` Two-phase black oil: oil and water. Unknowns per cell are $[p, S_w]$.

`3` Three-phase black oil: oil, water and gas. Unknowns per cell are $[p, S_w, S_g]$.

n_c Compositional: the number of hydrocarbon components. Unknowns per cell are $[p, N_1 \dots N_{n_c}]$.

Number of components. For `MODEL BLACKOIL`, a value of 2 selects the two-phase oil and water formulation and 3 selects three-phase black oil with dissolved gas. For `MODEL COMPOSITIONAL` this is the number of hydrocarbon components in the equation of state, and the block size becomes $n_c + 1$.

This keyword must precede `MODEL`.

5.1.2 MODEL

```
MODEL  BLACKOIL | COMPOSITIONAL
```

Default: `BLACKOIL`.

Values:

`BLACKOIL` Black-oil formulation. Resolves to two or three phases according to `NCOMP`. `BO` is accepted as an abbreviation.

`COMPOSITIONAL`
Equation-of-state compositional formulation. `COMP` is accepted as an abbreviation.

Selects the fluid formulation. `BO` and `COMP` are accepted abbreviations. The primary unknowns are $[p, S_w]$ for two-phase black oil, $[p, S_w, S_g]$ for three-phase black oil, and $[p, N_1 \dots N_{n_c}]$ for compositional flow, where N_i is the overall molar density of component i per unit pore volume.

5.1.3 TITLE

```
TITLE  <text>
```

Default: `NVRS-CUDA reservoir model`.

Values: Any text. Not interpreted.

Descriptive title reproduced in the run banner and in the output summary.

5.1.4 CASENAME

```
CASENAME  <name>
```

Default: `NVRS`.

Values: Any name valid as a file-name stem on the target filesystem. Avoid spaces.

Base name for the ECLIPSE-format output files, which are written as `<name>.EGRID`, `<name>.UNRST` and so on.

5.2 Grid geometry

5.2.1 NX, NY, NZ

```
NX  <integer>
NY  <integer>
NZ  <integer>
```

Default: 1 for each.

Values: Integers ≥ 1. The product is the total cell count; there is no fixed upper bound beyond available device memory. A dimension of 1 gives a two-dimensional model.

Number of cells in the i, j and k directions of the global grid. The total cell count is their product. Cell indices in well specifications are one-based.

5.2.2 DX, DY, DZ

```
DX  <real>
DY  <real>
DZ  <real>
```

Default: 50, 50, 20 ft.

Values: Positive reals, in feet. Cells need not be cubic, and strongly non-cubic cells are common in practice; `DZ` $\ll$ `DX` produces the vertical anisotropy discussed in Sections 6.10 and 6.11.

Cell dimensions. The grid is uniform: all cells share these dimensions. Face areas and cell pore volumes follow from them, the latter also requiring porosity.

> **Note.** Cell geometry has a strong influence on the anisotropy of the discrete system. A grid with `DZ` $\ll$ `DX` produces vertical transmissibility much larger than horizontal, which affects the choice of multigrid and of CPR second stage; see Sections 6.10 and 6.11.

5.2.3 TOPS

Not a keyword. Depth enters through the cell index in the k direction and `DZ`; the datum is the top of layer one.

5.2.4 CONNLIST

```
CONNLIST  <integer>
```

Default: 0 on a Cartesian grid; forced to 1 on a corner-point grid.

Values:
0 Assemble over the fixed six-face stencil.
1 Assemble over the general connection list.

Selects how the assembly kernels traverse cell connectivity.

The default path assumes every cell has at most six neighbours, at fixed positions, which is true of the uniform grid this keyword section describes. A corner-point grid read through `GRDECL` does not have that property: a fault throw juxtaposes one cell against parts of several, so the number of neighbours varies from cell to cell. On the Norne model it ranges from one to seventeen, with a mean of 5.81. The connection-list path makes no assumption about that number, and is therefore selected automatically whenever the grid did not come from `NX`/`NY`/`NZ`.

On a Cartesian grid both paths are available and describe the same connectivity, so either may be used. They are not merely expected to agree: the check in `tests/test_conn_assembly.cu` runs both on the same state and compares the residual, every entry of the coupled Jacobian, the CPR pressure block, the block-diagonal inverse and the quasi-IMPES weights. Agreement is to machine precision, the largest relative difference over 40,824 Jacobian entries of a three-phase problem with gravity and wells is 1.3×10^{-15}.

That residual difference is not error in either path. The two form the gravity head differently: the fixed stencil computes $(p_j - g\rho z_j) - (p_i - g\rho z_i)$, while the connection list carries the depth difference and computes $(p_j - p_i) - g\rho\,(z_j - z_i)$. These are equal in exact arithmetic; in floating point the second is the better conditioned, because it never forms the difference of two large $g\rho z$ terms. Setting `CONNLIST` 1 on a Cartesian grid is therefore a marginal improvement in the gravity term rather than a compromise.

> **Note.** The connection-list path also changes the cost of assembly. Because a neighbour's Jacobian block is complete as soon as its own connection has been processed, blocks are written directly to the sparse matrix inside the loop and only the self block accumulates. Register use is then independent of how many neighbours a cell has, which is what allows a degree of seventeen without spilling to local memory.

5.3 Rock properties

5.3.1 PERMFILE

```
PERMFILE  <path>
```

Default: none.

Values: A path, absolute or relative to the working directory. The file must contain at least `NX` × `NY` × `NZ` numbers. Omit the keyword to use the synthetic field bounded by `PERMMIN` and `PERMMAX`.

Path to a text file containing one horizontal permeability value per global cell, in natural order: the i index varies fastest, then j, then k. Values are in mD and may be separated by any whitespace.

When `PERMFILE` is absent, a deterministic smooth synthetic field spanning `PERMMIN` to `PERMMAX` is generated. This is a placeholder that allows the simulator to run without external data; it is not a geological model.

The path is resolved relative to the working directory. A file containing fewer values than the grid has cells is an error.

> **Note.** K_y is always equal to K_x. Anisotropy in the horizontal plane is not represented.

Example.

```
NX        60
NY        220
```

```
NZ        85
PERMFILE  dataset/SPE10/perm.txt
```

The file is read once at start-up and distributed to the ranks by global cell index, so the same field is obtained irrespective of how the domain is partitioned. A file of 10^6 values is a few tens of megabytes of text and takes a second or two to parse; this is not repeated during the run.

See also. `POROFILE` and `PERMZFILE` for the other two per-cell fields; `PERMMIN` and `PERMMAX` for the synthetic alternative.

Common errors. A file written for a different grid, which fails with a count mismatch; and a file in k-fastest order, which is accepted silently and produces a transposed field. Natural order means the i index varies fastest.

5.3.2 POROFILE

```
POROFILE  <path>
```

Default: none.

Values: A path, as for `PERMFILE`. Omit the keyword to derive porosity from permeability by a linear map onto [`POROMIN`, `POROMAX`].

Path to a text file containing one porosity value per global cell, in the same order as `PERMFILE`. When absent, porosity is derived from permeability by a linear map onto the interval [`POROMIN`, `POROMAX`].

An independent porosity field is required for datasets in which porosity is a separate measurement rather than a function of permeability; SPE10 model 2 is such a dataset.

Values read from the file are floored at `POROMIN`. This is significant for published datasets that substitute a very small value for zero porosity to avoid inactive cells: an accumulation term of order 10^{-7} conditions the linear system poorly while representing no storage.

Published datasets frequently substitute a very small positive number for zero porosity, so that no cell is inactive. Such a value gives an accumulation term some seven orders of magnitude smaller than its neighbours, which conditions the linear system poorly while representing no storage. `POROMIN` is applied as a floor for this reason.

Example.

```
POROFILE  dataset/SPE10/poro.txt
POROMIN   1e-4        -- floor under the file's dead-cell stand-in
```

See also. `POROMIN` for the floor; `PERMFILE` for the file format.

5.3.3 PERMZFILE

```
PERMZFILE  <path>
```

Default: none.

Values: A path, as for `PERMFILE`. Omit the keyword to use $K_z = \texttt{KZMULT} \times K_x$.

Path to a text file containing one vertical permeability value per global cell. When absent, $K_z = \texttt{KZMULT} \times K_x$.

Vertical face transmissibility uses the harmonic mean of the vertical permeabilities of the two cells sharing the face, in the same way as the horizontal directions.

5.3.4 KZMULT

```
KZMULT  <real>
```

Default: 0.2.

Values: A positive real. Values of 0.01 to 0.5 are usual; 1.0 makes the medium vertically isotropic. Ignored when `PERMZFILE` is supplied.

Ratio of vertical to horizontal permeability, applied uniformly. Ignored when `PERMZFILE` is given.

5.3.5 PERMMIN, PERMMAX

```
PERMMIN  <real>
PERMMAX  <real>
```

Default: 5 and 500 mD.

Values: Positive reals in mD, with `PERMMIN` < `PERMMAX`. Ignored when `PERMFILE` is supplied.

Bounds of the synthetic permeability field used when `PERMFILE` is absent. Ignored otherwise.

5.3.6 POROMIN, POROMAX

```
POROMIN  <real>
POROMAX  <real>
```

Default: 0.05 and 0.30.

Values: Reals in $(0, 1)$ with `POROMIN` < `POROMAX`. Producible reservoirs are typically 0.05 to 0.35. `POROMIN` also acts as a floor on values read from `POROFILE`, and should be small but non-zero for that purpose (for example 10^{-4}).

Bounds of the porosity mapping applied when `POROFILE` is absent. `POROMIN` additionally serves as a lower bound on porosity values read from `POROFILE`.

5.4 Fluid properties

5.4.1 UW, UO, UG

```
UW  <real>
UO  <real>
UG  <real>
```

Default: 1.0, 2.5, 0.02 cP.

Values: Positive reals in cP. Water is typically 0.3 to 1.0, oil 0.5 to 10, gas 0.01 to 0.05.

Water, oil and gas viscosities. Constant throughout the run.

5.4.2 BW, BO, BG

```
BW  <real>
BO  <real>
BG  <real>
```

Default: 1.0 RB/STB, 1.2 RB/STB, 0.005 RB/MSCF.

Values: Positive reals. B_w is near 1.0; B_o is typically 1.0 to 1.5; B_g is small, of order 10^{-3} to 10^{-2} RB/MSCF. A value of exactly 1.0 makes surface and reservoir volumes coincide, which is convenient when matching another simulator.

Formation volume factors. These are the values used directly when `PVTCONST` is set; otherwise they are the reference values from which the correlations are evaluated.

5.4.3 RHOW, RHOO, RHOG

```
RHOW  <real>
RHOO  <real>
RHOG  <real>
```

Default: 1.0, 0.8, 0.01 g/cm^3.

Values: Positive reals in g/cm^3. Water near 1.0, oil 0.7 to 0.9, gas 0.001 to 0.05. Setting all three equal removes buoyancy without disabling the hydrostatic gradient.

Phase densities at reservoir conditions, used to form the gravity term of the phase potentials. Note that these are reservoir densities, whereas ECLIPSE-format decks specify surface densities and divide by the formation volume factor; when constructing a matched pair of decks, choose the surface densities so that ρ_{surface}/B equals the value given here.

5.4.4 CFW, CFO, CT

```
CFW  <real>
CFO  <real>
CT   <real>
```

Default: 10^{-5}, 10^{-5}, 2×10^{-5} 1/psi.

Values: Non-negative reals in 1/psi, typically 10^{-6} to 10^{-5}. Zero gives an incompressible fluid, which is the setting used by the verification decks so that the accumulation term reduces to $V\phi/\Delta t$ exactly.

Water, oil and total compressibilities.

5.4.5 RS

```
RS  <real>
```

Default: 500 MSCF/STB.

Values: A non-negative real in MSCF/STB. Zero removes dissolved gas, making a three-phase run immiscible.

Solution gas-oil ratio. Consulted when `PVTCONST` is set; otherwise the value is obtained from a correlation at the field mean pressure. A value of zero removes dissolved gas, making a three-phase run immiscible.

5.4.6 VAPOIL

```
VAPOIL  0 | 1
```

Default: 1 (vaporised oil kept).

Values: `1` uses `PVTG`'s saturated R_v; `0` sets R_v to zero everywhere, so the gas phase carries no stock-tank oil.

Whether the gas phase carries vaporised oil.

R_v is not a primary unknown in this formulation, so with `VAPOIL 1` every cell holding free gas is credited with the *saturated* R_v at its pressure, whether or not that gas has ever been in contact with oil. On a field re-injecting dry gas this books stock-tank oil into the injected stream. On Norne the field gives up 371.1 M STB while the wells produce 286.1 M STB, an 85.0 M STB discrepancy, and the vaporised-oil term is the leading candidate.

`VAPOIL 0` is the opposite extreme and is equally wrong as physics: a live gas condensate really does carry oil. It exists to *bracket* that discrepancy in one run. If the balance closes with R_v removed, the term is the cause and the fix is R_v as a lagged per-cell state advanced between accepted steps; if the balance does not move, the cause is elsewhere.

The switch scales R_v inside `dev_rv_sat`, the one function through which the accumulation, the fluxes, the well rates and the in-place report all reach the tables, so none of them can disagree about whether it is on.

See also. `RS`, `PVTTABLES`.

5.4.7 PBUB

```
PBUB  <real>
```

Default: 3000 psia.

Values: A positive real in psia. Commonly set equal to `PINIT` for an undersaturated initial state.

Bubble-point pressure, the reference for the PVT correlations.

5.4.8 PATM

```
PATM  <real>
```

Default: 14.6959 psia.

Values: A positive real in psia. The default is one standard atmosphere; there is rarely a reason to change it.

Atmospheric pressure, used as the surface reference in the PVT correlations.

5.4.9 PVTCONST

```
PVTCONST  0 | 1
```

Default: 0.

Values:

`0` Fluid properties are re-evaluated once per report step from the built-in correlations, at the field mean pressure.

`1` Fluid properties are held at the deck values `BO`, `BG`, `RS` and `UG` for the entire run. Required when the same fluid must be described to another simulator.

When set to 1, the fluid properties B_o, B_g, R_s and μ_g are held at the values given in the deck for the whole run, rather than being re-evaluated once per report step from the built-in correlations.

This keyword should be set whenever the same fluid must be described to another simulator. The built-in correlations are a direct port of a reference implementation and are not a dimensionally consistent field-unit property set, so they cannot be transcribed into an ECLIPSE PVT table with the same meaning. With `PVTCONST` the fluid is exactly the deck constants, which transfer without ambiguity.

The built-in correlations are a direct port of a reference implementation and are not a dimensionally consistent field-unit property set: they produce a gas formation volume factor of order unity in RB/SCF and a solution gas-oil ratio of several thousand SCF/STB. They are adequate for a self-contained run and cannot be transcribed into another simulator's PVT tables with the same meaning.

Setting `PVTCONST` makes the fluid exactly the deck constants, which transfer without ambiguity. Every deck in the verification suite sets it for this reason.

See also. `BO`, `BG`, `RS`, `UG`; Chapter 9.

5.4.10 TEMP

```
TEMP  <real>
```

Default: 663 °R.

Values: A positive real in degrees Rankine. Compositional runs only; ignored otherwise. 663°R is approximately 200°F.

Reservoir temperature. Compositional runs only.

5.4.11 ZFACTOR

```
ZFACTOR  <real>
```

Default: 0.9.

Values: A real in $(0, 1]$. Compositional runs only.

Gas compressibility factor. Compositional runs only.

5.5 Saturation functions

5.5.1 SWI

```
SWI  <real>
```

Default: 0.1.

Values: A real in $[0, 1)$, and less than $1 - \texttt{SWR}$. Typically 0.1 to 0.3.

Connate (irreducible) water saturation. Together with `SWR` it defines the normalised saturation used by the two-phase analytic curves, and it is the lower bound applied to water saturation in the Newton update.

5.5.2 SWR

```
SWR  <real>
```

Default: 0.1.

Values: A real in $[0, 1)$, and less than $1 - \texttt{SWI}$. Typically 0.1 to 0.3.

Residual oil saturation to water.

5.5.3 SWOF

```
SWOF
  <Sw>  <Krow>  <Krw>  [<Pcow>]
  ...
/
```

Default: none.

Values: Two or more records. S_w must increase strictly down the table; k_{rw} must not decrease and must be zero in the first record; k_{row} must not increase and must be zero in the last. A fourth column, if present, is the oil-water capillary pressure $P_{cow} = p_o - p_w$ and is used.

Water-oil saturation table, read for three-phase runs. Each record gives a water saturation, the oil relative permeability in the presence of water, and the water relative permeability. A fourth column, if present, is the oil-water capillary pressure $P_{cow} = p_o - p_w$ and is used in the water phase potential and in the Jacobian; a column of zeros is equivalent to no column at all.

> **Note.** The column order is (S_w, k_{row}, k_{rw}), which is the reverse of the ECLIPSE convention (S_w, k_{rw}, k_{row}). A table transcribed from an ECLIPSE deck without exchanging the two relative permeability columns will be accepted and will produce incorrect results, since water will be given the relative permeability that begins at unity and decreases.

For two-phase runs the table is not read at all. The two-phase kernel evaluates analytic Corey curves,

$$S = \frac{S_w - \texttt{SWI}}{1 - \texttt{SWI} - \texttt{SWR}}, \qquad k_{rw} = S^2, \qquad k_{ro} = (1 - S)^2,$$

which is relevant when constructing a matched deck for another simulator: tabulating exactly these curves on the other side makes the two formulations identical.

Example.

```
-- Sw       Krow      Krw       (NVRS order: oil column SECOND)
SWOF
  0.20    1.0000    0.0000
  0.30    0.6944    0.0278
  0.50    0.2500    0.2500
  0.70    0.0278    0.6944
  0.80    0.0000    1.0000
/
```

The same table in ECLIPSE order would place k_{rw} in the second column. Both are accepted by the parser, and only one is correct: a transposed table gives water a relative permeability of unity at connate water, and the flood does not develop.

See also. `SGOF`, `SWI`, `SWR`, and Section 9.3 for the three-phase oil relative permeability convention.

5.5.4 SGOF

```
SGOF
  <Sg>  <Krog>  <Krg>  [<Pcog>]
  ...
/
```

Default: none.

Values: Two or more records, subject to the same monotonicity rules with k_{rg} increasing from zero and k_{rog} decreasing to zero. The maximum S_g should equal $1 - S_{wco}$, and k_{rog} at $S_g = 0$ should equal k_{row} at connate water.

Gas-oil saturation table, read for three-phase runs. The same column-order note applies as for `SWOF`. The oil relative permeability at zero gas saturation should equal the value from `SWOF` at connate water, and the maximum gas saturation should equal $1 - S_{wco}$.

5.6 Initial conditions

5.6.1 PINIT

```
PINIT  <real>
```

Default: 3000 psia.

Values: A positive real in psia.

Initial pressure, applied uniformly to every cell. The initial state is not equilibrated; see Section 1.4.

5.6.2 SWINIT, SGINIT, SOINIT

```
SWINIT  <real>
SGINIT  <real>
SOINIT  <real>
```

Default: 0.2, 0.02, 0.78.

Values: Reals in $[0, 1]$ summing to 1. `SWINIT` should be at least `SWI`. For two-phase runs `SGINIT` is ignored and `SOINIT` is $1 -$ `SWINIT`.

Initial saturations, applied uniformly. They should sum to unity. For two-phase runs `SGINIT` is ignored.

5.6.3 GRAVITY

```
GRAVITY  0 | 1
```

Default: 1.

Values:

`0` Gravity is omitted: phase potentials are pressures alone. Used to reproduce formulations that carry no gravity term.

`1` Gravity is included: $\Phi_p = p - \rho_p g z$, with the gradient set by `GRAVGRAD`.

When set to 0, the gravity term is removed from the phase potentials, so that flow is driven by pressure differences alone. This is used when reproducing formulations that omit the gravity head.

Note that NVRS-CUDA interprets `RHOW`, `RHOO` and `RHOG` as densities at reservoir conditions, whereas ECLIPSE-format decks give surface densities and divide by the formation volume factor. When constructing a matched pair of decks, choose the surface densities so that ρ_{surface}/B equals the value given here; otherwise the two models have different hydrostatic gradients and will disagree for reasons unconnected with the solver.

See also. `GRAVGRAD`, `RHOW`, and Chapter 9.

5.7 Wells

5.7.1 WELLS

```
WELLS
  <I> <J> <TYPE> <RATE> <RW> <SKIN> <PWF> <PERF> <NAME>
  ...
/
```

Default: none. `WELSPECS` is an accepted synonym.

Values: One record per well; at least one well is required for flow to occur. `TYPE` is 1 or -1; `PERF` is `all`; the remaining fields are reals as described above.

One record per well.

`I, J` Areal cell indices of the well, one-based.

`TYPE` 1 for an injector, -1 for a producer.

`RATE` Water injection rate in STB/day for an injector. For a producer the field is not used and is conventionally written as -1.

`RW` Wellbore radius in ft.

`SKIN` Skin factor, dimensionless.

`PWF` Bottom-hole pressure in psia. For a producer this is the control; for an injector it is an upper limit.

`PERF` Perforation interval. `all` perforates every layer.

`NAME` Well name, used in the output files and reports.

The connection transmissibility factor is computed from the Peaceman model with a fixed equivalent radius $r_e = \texttt{REFACTOR} \times \texttt{DX}$:

$$\mathrm{WI} = \frac{2\pi C k h}{\ln(r_e/r_w) + s}.$$

This is the quantity that ECLIPSE-format decks specify explicitly in item 8 of `COMPDAT`, so it may be transferred between the two without conversion. If $\ln(r_e/r_w) + s \le 0$ the well index

would be negative, which is reported as an error rather than allowed to produce a well that injects when it should produce.

Example.

```
-- WELLS:  I  J  TYPE  RATE  RW  SKIN  PWF  PERF  NAME
WELLS
  30 110   1   5000  1.0  0.0  10000  all INJ    -- injector, 5000 STB/d
   1   1  -1     -1  1.0  0.0   4000  all P1     -- producer, 4000 psi BHP
/
```

The rate field is meaningful only for injectors. For producers it is conventionally written as -1 to make the distinction visible in the deck; any value is ignored.

5.8 Field decks

The keywords described so far define a model in the simulator's own compact format: a uniform box, one number per property, wells stated once and held fixed. That format is convenient for benchmarks and scaling studies, and it is not what a field model looks like.

A field deck is written in the corner-point format that the industry's simulators share. It describes geometry as pillars and corner depths rather than as cell dimensions; it deactivates the parts of the box that are not reservoir; it declares faults that displace one side of the grid against the other; it builds one property array out of another by copying and scaling; and it changes well controls on dated steps over the producing life of the field.

The simulator reads such a deck directly. There is no conversion step and no intermediate file: the deck given on the command line is the same file the originating simulator was given.

The remainder of this section lists the deck keywords that are read. Section 5.8.6 states, equally explicitly, which are not, a keyword that is present in a deck and silently ignored is more dangerous than one that is rejected, because the run still produces an answer.

5.8.1 Geometry and the active grid

Keyword	Read as
`SPECGRID`, `DIMENS`	Grid dimensions.
`COORD`	Pillar endpoints, $6(n_x{+}1)(n_y{+}1)$ values.
`ZCORN`	Corner depths, $8n_xn_yn_z$ values.
`DX`, `DY`, `DZ`	Cell sizes, one value per cell.
`DXV`, `DYV`, `DZV`	Cell sizes, one value per index along that axis.
`TOPS`	Depth of the top face. Accepted for the first layer alone, n_xn_y values, which is the usual spelling, or for every cell.
`ACTNUM`	Active flag per cell.
`MINPV`, `MINPVV`	Minimum pore volume; cells below it are deactivated.
`INCLUDE`	Followed, resolved relative to the including file.
`METRIC`, `FIELD`	Unit system.

Cell volumes and centroids are computed by decomposing each cell into twenty-four tetrahedra, so that cells whose faces are not planar (the normal case in a corner-point grid) are integrated exactly rather than approximated by a box of equivalent dimensions.

A deck describes its grid one of two ways, and both are read. Corner-point gives `COORD` and `ZCORN` directly. Block-centred gives cell sizes and the depth of the first layer and leaves the corners implied; SPE1, SPE3 and SPE9 are all written that way.

The second form is converted into the first on reading, rather than carried through the simulator as a separate kind of grid. A block-centred deck describes bricks, whose corner-point form is vertical pillars at the accumulated widths with each cell's corners at its own top

and bottom depth, and that is what is built. Everything downstream is then identical for the two: faults, non-neighbour connections, MULTREGT, the transmissibility calculation and end-point scaling need no knowledge of which form the deck used.

Layers stack. TOPS states the depth of the first layer and each layer below begins where the one above ended, which is what makes a column continuous. A deck giving neither form is refused, naming both, rather than reporting COORD as though corner-point were the only possibility.

5.8.2 Rock properties and the array operators

Keyword	Read as
PORO, NTG	Porosity and net-to-gross.
PERMX, PERMY, PERMZ	Permeability; PERMY and PERMZ default to PERMX.
MULTX, MULTY, MULTZ	Directional transmissibility multipliers.
MULTX-, MULTY-, MULTZ-	The same on the negative faces.
MULTPV	Pore volume multiplier.
SATNUM, PVTNUM, EQLNUM, FIPNUM	Region arrays.
FLUXNUM, MULTNUM	Region arrays used by MULTREGT.
BOX, ENDBOX	Restrict subsequent operations to a range.
COPY	Copy one array to another over a box.
EQUALS	Set an array to a constant over a box.
MULTIPLY	Scale an array over a box.
ADD	Offset an array over a box.

The four operators are applied in the order they appear, because that order is the deck's meaning. This is not a convenience: on the Norne model, vertical permeability is never stated. It exists only as PERMX copied to PERMZ and then multiplied by a different factor in each of the twenty-two layers, ranging from 0.004 to 1.0. A reader that ignored COPY and MULTIPLY would take vertical permeability to equal horizontal, overstate it by up to 250× in the tightest layers, and give no indication that anything was wrong.

5.8.3 Faults

Keyword	Read as
FAULTS	Named sets of cell faces, with a face direction.
MULTFLT	Transmissibility multiplier for a named fault.
MULTREGT	Multiplier between region pairs; the region array is selected by the letter code F (FLUXNUM) or M (MULTNUM).

Fault geometry is not treated as a special case anywhere in the simulator. For each pair of adjacent columns the reader sweeps both columns and connects every pair of cells whose faces overlap in depth, weighting each half-transmissibility by the fraction of its own face that the overlap covers.

On an unfaulted grid every cell overlaps exactly its own layer and the result is the ordinary seven-point stencil. Where a fault has thrown one side down, the same sweep produces the several partial connections that the throw actually creates, which is what a fault is, geometrically. MULTFLT then scales those faces. On Norne this yields 9,526 connections that join cells in different layers, out of 125,233 in total.

A multiplier of zero removes a connection rather than leaving one that carries nothing. This matters to the preconditioner as much as to the flow: a structurally zero entry would still be an edge in the graph the algebraic multigrid coarsens, across a barrier that the deck intends to be closed.

5.8.4 Equilibration

Keyword	Read as
`EQUIL`	Per region: datum depth and pressure, the water-oil and gas-oil contact depths, and the capillary pressure at each.
`DENSITY`	Surface densities of oil, water and gas.
`EQLNUM`	Which equilibration region each cell belongs to.

Within a region each phase occupies its own hydrostatic column. The oil column is anchored at the datum, where the deck states the pressure; the water and gas columns are anchored at their contacts, where the deck states the capillary pressure. Anchoring at the contacts rather than at the datum is what makes the result an equilibrium: the phase pressures then differ at each contact by exactly the stated capillary pressure, whatever the gradients are and wherever the datum happens to sit. Anchoring all three at the datum would look almost identical on a plot and would leave a pressure discontinuity at every contact, which the first timestep would immediately begin to flatten.

Phase gradients require reservoir densities, which are surface densities divided by formation volume factors, and those come from PVT tables that this reader does not interpret (Section 5.8.6). The gradients must therefore be supplied by the caller. They are never guessed: a wrong gradient is invisible in the output, since the resulting pressure field still looks like a pressure field, so initialisation refuses to proceed without them rather than substituting a plausible default.

5.8.5 Wells and the schedule

Keyword	Read as
`WELSPECS`	Well name, group, head location, reference depth, preferred phase.
`COMPDAT`	Completions: cell range, open or shut, connection factor, diameter, Kh, skin, and the penetration direction.
`WCONPROD`	Producer control mode and limits.
`WCONHIST`	Observed producer rates.
`WCONINJE`, `WCONINJH`	Injector control and limits.
`WELOPEN`	Open or shut a well, or individual completions.
`WEFAC`	Well efficiency factor.
`DATES`, `TSTEP`	Advance the schedule and record a report step.

Penetration direction. The direction field of `COMPDAT` distinguishes a vertical completion from a horizontal one, and it changes the well index. Peaceman's formula is usually written for a vertical well, and the two permeabilities and two cell extents that appear in it are those *perpendicular* to the wellbore. For a well running along i those are (k_y, k_z) and (d_y, d_z), with d_x as the exposed length, not (k_x, k_y) and (d_x, d_y). Applying the vertical form to a horizontal completion does not fail; it returns a productivity that is wrong by whatever factor the anisotropy supplies. Norne has twenty-four wells with horizontal sections and twelve that are purely vertical.

Control is a mode plus limits. A well is not simply on rate or on pressure. It carries a requested mode together with every limit the deck stated, and which one binds is decided from what the well achieved. An injector given a rate and a bottom-hole pressure ceiling injects at the rate until the formation will not take it and holds the ceiling thereafter. A producer given an oil target and a pressure floor holds the target until it can no longer sustain it, then drops to the floor. A producer held at a pressure with a rate ceiling does the reverse. A simulator that fixes one mode per well will run, and will diverge from the deck at the first limit, which for an injector is usually within the first few steps. Norne's history contains 230 control changes across 247 report steps.

Completions that fall in dead cells are reported rather than discarded. A well losing perforations to `ACTNUM` or `MINPV` is a material fact about what the model can produce, not a detail of the reading.

5.8.6 Keywords that are not interpreted

The following are recognised where they appear, and their data is consumed so that the rest of the deck continues to parse, but they do not affect the run. Every one encountered is listed in the run log and by the `deckinfo` utility, so that a deck depending on one of them is visible in review rather than at the end of a study:

- Hysteresis, Carlson non-wetting hysteresis (`EHYSTR` model 0) is implemented in both gas/oil and oil/water, driven by `IMBNUM` and the I-prefixed imbibition endpoints. Other `HYSTMOD` models are not; for those the drainage curve remains active and the run says so at startup.
- Tubing tables and tubing-head control, `VFPPROD`, `VFPINJ`, and therefore `THP` as a control mode.
- Group *injection* control, `GCONINJE RATE` is implemented as a phase-specific ceiling. `RESV`, `REIN` and `VREP` records are parsed but are not enforced. Group *production* control is implemented; see below.
- Vaporised-oil rate limits, `DRVDT`. `DRSDT` and `VAPPARS` are enforced on the accepted sub-step state; see Section 5.8.9.
- Threshold pressures (`THPRES`), tracers, `PINCH`, and multiple saturation regions (`SATNUM`: a second region is parsed and the first is used).

Three entries that appeared in this list in earlier releases have since been implemented, and are recorded here because a reader may have designed around their absence:

- **PVT tables.** `PVTO`, `PVTG` and `PVTW` are read from the field deck and used on the device, with `PVTNUM` regions. The scalar keywords of the Fluid properties section apply only when `PVTCONST` is set or no tables are present.
- **Saturation functions and endpoint scaling.** `SWOF` and `SGOF` are read from the field deck by default (`FIELDSATFUNC`); `ENDSCALE`'s `SWL`, `SWCR`, `SWU`, `SGL`, `SGCR`, `SGU`, `SOWCR`, `SOGCR`, `KRW`, `KRO` and `KRG` are read under `PROPSARRAYS` and applied under `USEENDSCALE`, honouring `SCALECRS`.
- **Group production and surface-rate injection control.** `GRUPTREE` builds the hierarchy and `GCONPROD`'s rate target is shared among the wells that defer to it in proportion to their productivity, the guide-rate default. A phase-specific `GCONINJE RATE` target acts as a ceiling on the stated rates of the group's injectors.

5.8.7 Reading a deck before running it

The `deckinfo` utility loads a deck and reports what each reader made of it, cell and connection counts, faults, wells and their completions, equilibration regions, and the full list of keywords not interpreted. It links against no GPU code and runs anywhere:

```
./scripts/build_deck_tests.sh dataset/NORNE/NORNE_ATW2013.DATA
./build/deck/deckinfo dataset/NORNE/NORNE_ATW2013.DATA \
     --gradients 0.0706,0.1001,0.0186
```

To run the model, name the corner-point deck from a run deck through `FIELDDECK` and launch it as any other case:

```
./benchmark/get_norne.sh
NGPU=1 ./scripts/run.sh decks/NORNE.DATA
```

Note. Always launch through `scripts/run.sh`. The simulator is an MPI program; started directly it initialises no transport, and the run ends with a message from UCX that says nothing about the deck. `run.sh` also selects the rank count, and knows not to apply the K-slab rank limit to a field deck, whose `NZ` is a placeholder rather than a layer count.

Note. The Norne model is not distributed with the simulator. Run `benchmark/get_norne.sh` to fetch it; the arrangement is the same one `make_spe10.py` uses for SPE10, and files already present are left alone so that re-running is inexpensive.

The voidage balance deserves a check before a long run. Injected volume over the schedule should be a sensible fraction of the pore volume: enough to move a front past the producers, not so much that the model fills. The deck generators print this fraction, and a run that fails to converge late in the schedule is often a model that has been over-injected.

See also. `REFACTOR` for the equivalent radius, and Section 6.3 for the well model.

5.8.8 The Norne field model

Norne is the reference field for this simulator: the case every corner-point feature was built against, and the one the comparison with OPM Flow is run on. Equinor released the full model (grid, faults, PVT, saturation functions, and nine years of measured well history) and it is consequently the closest thing the industry has to a shared, open, non-trivial benchmark.

Quantity	Value
Grid	$46 \times 112 \times 22 = 113{,}344$ cells
Active	44,431 (68,417 inactive, 496 below `MINPV`)
Faults	61 named, 13,479 connections scaled
Connections	121,167, of which 7,617 non-neighbour
Wells	36, 27 producers, 9 injectors, 504 completions
Schedule	247 report steps, 06 Nov 1997 to 01 Dec 2006
Fluid	Three-phase black oil, live oil and wet gas

Norne exercises nearly everything that distinguishes a field deck from a generated one, which is why it is worth running before trusting a change: corner-point geometry with fault throws, the non-neighbour connections those throws create, a connection list whose degree runs from one to seventeen, region arrays, per-region equilibration, deck-supplied connection factors on horizontal wells, and a history schedule in which controls, and the wells themselves, come and go.

Running it

```
./benchmark/get_norne.sh                    % 61 files, about 26 MB
NGPU=1 ./scripts/run.sh decks/NORNE.DATA
```

The comparison against OPM Flow, and the report it produces, are driven from the benchmark directory:

```
cd benchmark
./run_norne.sh                              % both simulators
SKIP_OPM=1 ./run_norne.sh                   % NVRS only, reusing Flow output
python compare.py --case NORNE              % rebuild compare_NORNE.html alone
```

Comparing like with like

Two simulators reading the same deck are not thereby solving the same problem, and the ways they can differ are quiet ones.

The saturation functions must come from the field deck. A run deck that names `FIELDDECK` is describing that field, so its own `SWOF` and `SGOF` are at best a placeholder, and Flow reads Equinor's. Running the two on different tables compares different rock, and every rate, water cut and correlation then describes a model nobody intended. `FIELDSATFUNC` defaults to 1 for this reason, and the run prints the four curve corners at startup (k_{rw} at maximum water, k_{row} at connate water, k_{rg} at maximum gas, k_{rog} at zero gas) each of which must be the large end of its curve. A table read with the wrong column convention, or parsed into the wrong number of rows, produces a smooth curve of the wrong shape rather than anything that looks broken, so this is checked rather than assumed: if any corner fails, the field tables are refused and the reason is printed.

`PROPSARRAYS` and `USEENDSCALE` change the model rather than refining it and both default off. The first hands equilibration the deck's `SWL` and `EQLNUM`; Norne has five equilibration regions, and moving off a single record gives roughly three thousand cells a gas cap they did not previously have. Results obtained with these on and off are not comparable with one another.

Resolved and remaining Norne features

The following items were limitations of the first manual issue and are now handled: the ECLIPSE-default three-phase oil relative permeability, three-point endpoint scaling, Carlson gas hysteresis from `EHYSTR/ISGCR`, per-cell `PVTNUM`, live-oil R_s, VAPOIL R_v, phase-dependent gravity density, producer group allocation, `GCONINJE RATE` ceilings, history-mode well controls, metric gas-rate conversion, completion pressure offsets and active zero-rate well shutdown.

The remaining Norne gaps are narrower and must still be named. `DRVDT` limits are parsed but not enforced. `GCONINJE` reservoir-rate, reinjection and voidage-replacement modes are parsed but not active; the latter two need group production history. `VFPPROD`, `VFPINJ` and tubing-head control are not modelled. `PINCH`, multi-segment wells and surface-network balance are not modelled. Carlson gas hysteresis is supported; other hysteresis models are not. Every keyword each reader did not interpret is listed at startup and by `deckinfo`, which is the fastest way to separate a deck problem from a solver problem.

5.8.9 What a field deck may not contain

Keywords the reader does not implement are stepped over and listed once in the run log, under *not implemented*. That is the right treatment for most of them, which control reporting, formatting and dimensioning and have no bearing on the answer: `RPTRST`, `RPTSCHED`, `NOECHO`, `MESSAGES`, `NSTACK`, `EQLDIMS` and their kind.

It is not the right treatment for a keyword that changes the physics, and the reader does not tell the two apart. A deck containing any of the following will run to completion and give an answer that does not describe the deck:

- `AQUFETP`, `AQUCT`, `AQUANCON` and the other aquifer keywords. There is no aquifer, so the pressure support they provide is absent.
- `WELSEGS`, `COMPSEG`: multisegment wells. Completions are treated as though connected directly to the wellbore.
- `VFPPROD`, `VFPINJ`: no tubing tables, so tubing-head control cannot be honoured.
- `POLYMER`, `SOLVENT`, `THERMAL` and the tracer keywords: different conservation equations entirely.
- `ROCKTAB`, `OVERBURD`: pressure-dependent rock properties.

- PINCH: pinched-out layers are not connected across.
- DRVDT: the rate limit on how fast vaporised oil may change. DRSDT and VAPPARS, its dissolved-gas and saturation-dependent counterparts, are enforced.

The fluid description has its own limits. PVTO, PVTG, PVTW, PVDO, PVDG and DENSITY are read; PVCDO is not. ROCK is read, and its compressibility and reference pressure replace CT and scale pore volume with pressure. Saturation functions are read as SWOF and SGOF only, so a deck using the family II forms SWFN, SGFN and SOF3 has no relative permeability at all.

Read the *not implemented* line in the run log against this list before trusting a result from an unfamiliar deck. A keyword appearing in both means the run answered a different question from the one the deck asked.

5.9 Time stepping

5.9.1 DTREPORT

```
DTREPORT  <real>
```

Default: 100 days.

Values: A positive real in days. Common values are 30, 90 or 100. It controls output frequency and the granularity of the progress table, not the accuracy of the solution, which is governed by the internal timestep.

Length of a report step. Output is written at the end of each report step, and the progress table has one line per report step. The simulator may divide a report step into shorter sub-steps, but never advances past a report boundary in a single step.

5.9.2 TEND

```
TEND  <real>
```

Default: 2000 days.

Values: A positive real in days, ordinarily an exact multiple of DTREPORT.

Total simulated time. The number of report steps is TEND/DTREPORT.

5.9.3 DTMAX

```
DTMAX  <real>
```

Default: 0.

Values: A non-negative real in days. Zero disables the cap. Any positive value is honoured; a value larger than DTREPORT has no effect, since a step never crosses a report boundary.

Upper bound on the internal timestep. A value of zero means that each report step is first attempted as a single step, and shortened only when Newton fails to converge.

Setting DTMAX to a length that is known to converge avoids repeating a search whose answer is already known. On a model that habitually chops four times before succeeding, this removes four discarded attempts per report step, each of which costs a full set of Newton iterations.

Where a model reliably converges at a particular step length, setting it directly avoids rediscovering it at every report boundary. On the one-million-cell benchmark, whose first report step fails at 100 days and chops four times to reach 6.25 days, setting DTMAX to 6.25 reduced the run from 23.0 s to 5.4 s without changing the steps actually taken.

The cap should not be set below the length the model converges at, since the additional steps then cost time without improving the answer, and it should not be used to force a finer discretisation than a comparison case uses.

See also. DTGROW, MAXCHOP.

5.9.4 DTCONTROL

```
DTCONTROL    <0 or 1>
DTRESTART    <real>
DTMAXGROW    <real>
DTDECAYDAMP  <real>
DTGROWDAMP   <real>
DTFAILCAP    <real>
DTCAPGROW    <real>
DTCAPPROBE   <integer>
DTCAPPROBEITERS <integer>
DTCAPPROBEFACTOR <real>
```

Default: `DTCONTROL 0`. When 1: `DTRESTART 0.33`, `DTMAXGROW 3.0`, `DTDECAYDAMP 1.0`, `DTGROWDAMP 3.2`, `DTFAILCAP 0.75`, `DTCAPGROW 1.2`.

Values: `DTCONTROL 0` keeps the dyadic scheme of `DTRELAX`: a report step is planned as $\Delta t_{\text{rep}}/2^c$ and a failure raises c. `DTCONTROL 1` replaces it with a continuous timestep carried across report steps.

How the step length is chosen at all. The two schemes differ in kind rather than in tuning, and the difference is why nothing tuned inside the first one helped.

Under `DTCONTROL 0` the reachable lengths are powers of two. Descent through them is geometric, since one failure can drop six levels at once, and ascent is linear, one level per `DTGROW` clean steps. A run settles far below the length it could carry and does not come back.

Under `DTCONTROL 1` there are no levels. After every converged sub-step the length is rescaled by

$$\Delta t \leftarrow \begin{cases} \Delta t \left(1 + \frac{n_{\text{target}} - n_{\text{used}}}{n_{\text{target}}}\, \delta_{\text{grow}}\right), & n_{\text{used}} \le n_{\text{target}}, \\ \dfrac{\Delta t}{1 + \frac{n_{\text{used}} - n_{\text{target}}}{n_{\text{target}}}\, \delta_{\text{decay}}}, & n_{\text{used}} > n_{\text{target}}. \end{cases}$$

with n_{target} from `DTTARGET`, damping from `DTGROWDAMP` while growing and `DTDECAYDAMP` while shrinking, and growth capped at `DTMAXGROW`. A failure is not a halving either: the sub-step restarts at `DTRESTART` of its length and everything already converged inside the report step stands. These are OPM Flow's own defaults, so `DTCONTROL 1` alone reproduces its scheme rather than a variant of it.

`DTFAILCAP` and `DTCAPGROW` are ours and are not in OPM Flow. The rule above oscillates on its own: a sub-step converging in two Newton iterations against a target of eight is told to grow by 1.94, so two successes carry the length back above one that has just failed. OPM Flow avoids this by taking the smaller of the iteration-count estimate and a PID controller on the relative change in the solution. We approximate that with a ceiling: a failure drops it to `DTFAILCAP` of the length that failed, and each successful sub-step relaxes it by `DTCAPGROW`, so the controller has to earn its way back rather than jumping over the cliff it just fell off. `DTFAILCAP 0` disables the ceiling and gives the unmodified rule.

`DTCAPPROBE` optionally raises a learned ceiling after that many consecutive sub-steps converge in no more than `DTCAPPROBEITERS` iterations. The trial ceiling is multiplied by `DTCAPPROBEFACTOR`; if it fails, the last proven ceiling is restored. Zero disables probes, which is the default. On matched SPE10, probes reduced Newton iterations but increased Krylov work and wall time, so the benchmark leaves them disabled.

`DTCONTROL 1` implies `DTKEEP 1`: a controller that adjusts the length continuously means nothing if a failure discards the interval it was controlling. `MAXCHOP` becomes the restart budget per sub-step rather than a number of halvings.

5.9.5 DTKEEP

```
DTKEEP  <0 or 1>
```

Default: `DTKEEP 0`.

Values: 1 keeps the sub-steps of a report step that have already converged when a later one fails.

Without it a failure anywhere inside a report step discards the whole step and retries all of it at a shorter length, including the sub-steps that had already converged. With it the state is restored to the last converged sub-step boundary, and only the remainder is re-planned.

This is what makes an overshoot cheap, and cheap overshoots are the precondition for letting the step length grow. On the Norne field deck a failure at day 44 kept 29.06 of the report step's 30 days and retried the last 0.94.

Implied by `DTCONTROL 1`.

5.9.6 OPMCONV

```
OPMCONV     <0 or 1>
CNVRELAXPV  <real>
CNVRELAXED  <real>
```

Default: `OPMCONV 0`. When 1: `CNVRELAXPV 0.03`, `CNVRELAXED 1.0`, both of which are Flow's own defaults.

Values: `OPMCONV 1` selects OPM Flow's convergence policy. It is a complete policy, not an extra early exit beside the strict one: under `OPMCONV` the relative L_2 ratio of `NEWTONTOL` no longer ends a sub-step at all.

That last point is not a detail. The combined L_2 residual is dominated by gas, which runs three orders of magnitude above the other two components on Norne, so a water residual can be badly wrong and barely move the norm. While the L_2 exit was allowed to run in parallel it took the step first, and the water error it accepted each time accumulated into a missing water bank by roughly half the history.

The two criteria. Both must pass, and they are tested at every Newton iteration, as Flow tests them.

MB is the *signed* sum of each component residual over the field, as a fraction of that component in place, against `MBTOL`. Signed is the whole point: local residuals that cancel physically (fluid leaving one cell and arriving in its neighbour) must cancel in a field balance too. A per-equation Euclidean norm of the same residual does not cancel, is strictly larger, and is not the quantity Flow tests. Both are reported; the norm is a diagnostic.

CNV is the worst single cell as a fraction of its own contents. The strict test compares it against `CNVTOL`, so one bad cell fails the whole step, and on a 44,431-cell field deck with a gas injector such a cell exists at every step length worth taking. So the tolerance is relaxed to `CNVRELAXED` when the cells that violate the strict bound hold less than `CNVRELAXPV` of the pore volume. Three percent of the field may be badly converged.

The violating cells are counted as a *union* over the three components, and the relaxed tolerance then applies to all three together. Flow classifies a cell by its worst component; three independent per-component fractions would relax three disjoint regions and exempt considerably more of the field than `CNVRELAXPV` names. The `[why]` line prints the union share first, then the three component shares.

Do not tighten these. Setting `CNVRELAXED` to `CNVTOL` makes the relaxed test identical to the strict one, `OPMCONV` then buys nothing, and the run goes back to chopping. Strict CNV everywhere improves the day-268 Norne state and turns pathological immediately after it, chopping into thousands of tiny sub-steps around the first difficult gas front. Screen `DTMAX` instead.

Why there is no stall gate here. MB and CNV both scale with Δt, so on a short enough sub-step the initial guess satisfies both and the step would be declared converged having solved nothing. The strict path guards that with `NEWTONSTALL`; under `OPMCONV` the guard is simply that at least one Newton update must have been taken. Requiring a stall as well wasted

iterations that had already converged and made the L_2 bypass more likely to win the race. NEWTONSTALLMAX still abandons an attempt that has stopped descending, but it watches the same scaled pair, max(MB/MBTOL, CNV/CNVTOL), rather than the raw norm, so a solve whose signed water balance is still improving is not abandoned because the gas-dominated norm has flattened.

5.9.7 NEWTONSTALLMAX

```
NEWTONSTALLMAX  <integer>
```

Default: NEWTONSTALLMAX 0, off.

Values: the number of consecutive Newton iterations that may fail to reduce the residual by NEWTONSTALL, with the convergence tests still unmet, before the attempt is abandoned. 4 is a reasonable setting.

A failed sub-step otherwise costs the full NEWTONMAX. The diagnostics show what those iterations do: material balance parked at 1.3 to 1.8 times its tolerance with the relative residual between 10^{-2} and 3×10^{-1}, unchanged from one iteration to the next. The step is lost by the third or fourth iteration and the rest is spent finding that out.

Ending the attempt early does not change what happens next. The timestep controller retries at a shorter length either way; it simply does so sooner.

A sub-step that ends this way prints

```
[why] step 12 sub 3 abandoned at Newton 4: residual stalled for
      4 iteration(s) at rel 1.786e-01
```

and the [why] line after it names which of the three tests was blocking.

5.9.8 LINESEARCH

```
LINESEARCH  <integer>
```

Default: LINESEARCH 0, off: the fully implicit update is taken at $\alpha = 1$ however large it is.

Values: the greatest number of times α may be halved while looking for an update that actually reduces the residual. 6 is the setting the shipped decks use.

Newton's step is a prediction from a linearisation, and on a strongly nonlinear sub-step (a gas front arriving in a cell, an injector still climbing towards its target), the full step can land somewhere worse than it started. This backtracks until it does not. Each trial restarts from the same reservoir state rather than rescaling an already clipped update, because APPLEYARD and DSMAX make newton_update nonlinear in α, so successive scaling would not describe any single step.

Under OPMCONV the quantity a trial must reduce is Flow's own merit, max(MB/MBTOL, CNV/CNVTOL), rather than the combined L_2 residual, which gas dominates; otherwise it is the L_2 residual.

The search is close to free. A trial that is accepted leaves the assembled residual and Jacobian of exactly the state the next Newton iteration begins from, and that assembly is carried forward instead of being repeated, so a search which accepts $\alpha = 1$, the common case, costs no extra assembly at all. Only genuine backtracking costs anything, which is the case it exists for.

With --verbose or TRACERETRY, a search that halved at least once prints

```
line search: alpha 0.5 after 1 halving(s), |Rtrial| 4.1183e-03
```

5.9.9 DTRELAX

```
DTRELAX   <integer>
DTTARGET  <integer>
```

Default: `DTRELAX 1`, `DTTARGET 8`.

Values: `DTRELAX` is the greatest number of halvings the timestep may recover after one successful report step; 1 is the historical behaviour. `DTTARGET` is the Newton count per sub-step that counts as comfortable.

How far the timestep is allowed to grow back after it has been reduced.

The reduction is geometric and the recovery was not. A report step that fails is retried at half the length, then a quarter, and so on, so one difficult step can drop the step length by six or eight halvings at once, and the controller then holds it. Recovery ran at one halving per `DTGROW` clean steps, so returning from eight halvings took sixteen consecutive report steps that converged at the first attempt.

On a model that chops at all frequently, that run of clean steps never occurs and the timestep never returns. Norne is the case in point: 85 of its 247 report steps chop, the median step length is 0.219 days against a report interval near thirteen, and almost none of that is required. Only the 85 chop; the rest converge at the first attempt, at a length some earlier step imposed on them and which nothing since has been able to undo.

With `DTRELAX` above 1, a step that converged comfortably recovers in proportion to how comfortable it was. If it needed n Newton iterations per sub-step against a target of T, the controller relaxes $1 + \lfloor \log_2(T/n) \rfloor$ halvings, capped at `DTRELAX`. A step that took two iterations against a target of eight is reporting that its length is four times shorter than it needs to be; under the old rule that signal was discarded and half of it acted on two clean steps later.

The setting never shortens a step. Shortening remains the business of `MAXCHOP` and the convergence failure that triggers it.

> **Note.** `DTRELAX 1` reproduces earlier behaviour exactly, and is the default, so no existing result changes without being asked to. Anyone reproducing published numbers should leave it alone; anyone running a new model that chops should try 3 or 4 and compare the Newton totals.

See also. `DTGROW`, `MAXCHOP`, `DTMAX`, `NEWTONMAX`.

5.9.10 DTGROW

```
DTGROW  <integer>
```

Default: 1.

Values: An integer ≥ 1. A value of 1 lengthens the step after every successful report step; 2 requires two consecutive first-attempt successes and is recommended for models that chop; larger values are progressively more conservative.

Number of consecutive report steps that must converge at the first attempt before the timestep is lengthened.

With the default value of 1 the timestep grows after every success, including one that immediately followed a reduction, so the controller can oscillate between a step that is too long and the reduction that repairs it, discarding an attempt each cycle. A value of 2 damps this behaviour and is recommended for models that chop at all.

The behaviour this setting addresses is visible in the log as an alternation between a converged step and a chopped one:

Example.

```
[dt]   step 6 converged at dt = 3.125 days; next step starts at 6.25
[chop] step 7 did not converge; retrying with dt = 3.125 days
```

```
[dt]   step 7 converged at dt = 3.125 days; next step starts at 6.25
[chop] step 8 did not converge; retrying with dt = 3.125 days
```

Each cycle discards a complete set of Newton iterations. With `DTGROW 2` the step is lengthened only after two consecutive first-attempt successes, and the oscillation stops.

See also. `DTMAX`, which fixes the length outright when it is known.

5.9.11 MAXCHOP

```
MAXCHOP  <integer>
```

Default: 6.

Values: An integer ≥ 0. Each unit permits one further halving, so the shortest step attempted is $\texttt{DTREPORT}/2^{\texttt{MAXCHOP}}$. Values of 6 to 8 are usual; a difficult heterogeneous model may need 8.

Maximum number of successive halvings of the timestep before the run is abandoned. Each halving doubles the number of sub-steps covering the report interval, so the shortest timestep attempted is $\texttt{DTREPORT}/2^{\texttt{MAXCHOP}}$.

5.9.12 SUBSTEPS

```
SUBSTEPS  <integer>
```

Default: none.

Values: A positive integer, or omit the keyword to use the adaptive controller.

Fixed number of sub-steps per report step, bypassing the adaptive controller. Intended for reproducing a prescribed time discretisation.

5.10 Non-linear solution control

5.10.1 NEWTONMAX

```
NEWTONMAX  <integer>
```

Default: 20.

Values: A positive integer. Values of 10 to 40 are usual. Raising it helps only when the residual is still decreasing at the limit; when it has stalled, a shorter timestep is the remedy.

Maximum Newton iterations permitted for one sub-step. Exceeding the limit is treated as a failed timestep, and the step is retried at half the length.

Increasing this value is productive only when the residual is still decreasing when the limit is reached. When the residual has stalled or is increasing, a shorter timestep is the appropriate response and the default already provides it.

The question of whether to raise this limit is answered by the residual sequence, which `NVRS_VERBOSE=1` prints. A sequence that is still decreasing at the limit indicates a limit set too low; a sequence that has stalled or is increasing indicates a timestep that is too long, and raising the limit merely delays the reduction that is required.

See also. `NEWTONTOL`, `MAXCHOP`, and Chapter 13.

5.10.2 NEWTONTOL

```
NEWTONTOL  <real>
```

Default: 5×10^{-3}.

Values: A positive real, the relative residual reduction required. Values of 10^{-3} to 10^{-2} are usual for production runs; tighter values increase the Newton count for little change in the answer.

Convergence criterion for a Newton iteration, expressed as the ratio of the current residual norm to the norm at the first iteration of the sub-step.

5.10.3 LINESEARCH

```
LINESEARCH  <integer>
LINEBEST    0 | 1
```

Default: both zero.

`LINESEARCH` is the maximum number of halvings used to find a nonlinear residual-reducing Newton update. Each trial recomputes well controls and the full residual. With `LINEBEST 1`, if no trial strictly descends, the finite trial with the best scaled MB/CNV merit is retained instead of the smallest alpha. The option is experimental and is disabled in supplied decks: it did not recover SPE10's pressure-linear-solver failure.

5.10.4 DPMAX

```
DPMAX  <real>
```

Default: 500 psi.

Values: A positive real in psi. Values of 200 to 1000 are usual. Too small a value slows convergence by truncating legitimate updates; too large a value permits an iterate to leave the range of the property correlations.

Maximum change in cell pressure permitted in a single Newton iteration. The update is truncated at this magnitude, which keeps early iterations from leaving the range where the property correlations are meaningful while retaining the Newton direction.

5.10.5 DSMAX

```
DSMAX  <real>
```

Default: 0.2.

Values: A real in $(0, 1]$. Values of 0.1 to 0.3 are usual. As with `DPMAX` this is a safeguard, not a convergence criterion.

Maximum change in a cell saturation permitted in a single Newton iteration.

5.10.6 PROPSARRAYS

```
PROPSARRAYS  0 | 1
```

Default: 0.

Values:

`0` Per-cell arrays are read from the field deck's `GRID` and `EDIT` sections only.

`1` Arrays defined in `PROPS`, `REGIONS` and `SOLUTION` are read as well.

The reader stops at `PROPS` by default because those sections are full of record-structured keywords it does not model. The cost of stopping is that every per-cell array they define is discarded: the endpoint arrays `SWL`, `SWCR`, `SWU`, `SGL`, `SGCR`, `SGU`, `SOWCR`, `SOGCR` and the maximum relative permeabilities are `PROPS` keywords, and `EQLNUM`, `SATNUM`, `PVTNUM` and `FIPNUM` are `REGIONS` keywords.

Setting this changes more than the name suggests, which is why it is off by default rather than simply fixed. Equilibration reads two of those arrays: `SWL` sets connate water per cell, and `EQLNUM` selects which `EQUIL` record each cell equilibrates against. On Norne, which carries five equilibration regions, enabling this moves the field off a single record onto the deck's own assignment and gives roughly three thousand cells a gas cap they did not previously have. That is faithful to the deck and it is a different model from the one a run with `PROPSARRAYS 0` produces, so results obtained with the two settings are not comparable.

When on, the reader operates in a restricted mode past `PROPS`: arrays and the box operators (`BOX`, `ENDBOX`, `COPY`, `EQUALS`, `MULTIPLY`, `ADD`) are interpreted and every other keyword is stepped

over. An array of the wrong length is fatal in `GRID`, where it means a corrupt deck, but is discarded past `PROPS`, where the likelier cause is an array name appearing as an argument to `RPTPROPS`.

5.10.7 USEENDSCALE

```
USEENDSCALE  0 | 1
```

Default: 0. Requires `PROPSARRAYS` 1 to have any effect.

Values:

`0` One saturation table applies unscaled to every cell.

`1` Each cell's relative permeability curves are scaled onto its own saturation endpoints.

Endpoint scaling maps a cell's endpoints onto the table's, so that a cell whose water becomes mobile at $S_w = 0.3$ reads the table where the table's water becomes mobile, whatever value that is. Each of the four curves has its own pair of endpoints and therefore its own map: k_{rw} is scaled between the critical and maximum water saturations, while k_{row} is scaled between connate water and the saturation at which oil stops moving, so the same S_w reads the table in two different places.

The field deck's `SCALECRS` selects the method. `NO`, or its absence, gives two-point scaling. `YES` gives three-point: the displacing phase is anchored at three saturations rather than two, the middle one being where the displaced phase stops moving, and each interval is mapped separately. The displaced phase is two-point under both settings. The choice is not cosmetic, the middle anchor sits at the residual-oil end of a waterflood, which is the range a front actually sweeps.

Endpoints reach the reported well rates as well as the flow solve, since a productivity computed from a different curve than the reservoir delivers would give a bottom-hole pressure that misses its target.

5.10.8 FIELDSATFUNC

```
FIELDSATFUNC  0 | 1
```

Default: 1.

Values:

`0` `SWOF` and `SGOF` are taken from the run deck.

`1` They are taken from the field deck named by `FIELDDECK`, when it supplies them.

A run deck that names a field deck is describing that field, and its own saturation tables are then at best a placeholder. Leaving them in place is a quiet and serious error: a benchmark run against another simulator reading the same field deck compares two different rocks, and every rate, water cut and correlation computed from it is describing a model nobody intended.

The column order is the hazard, and the conversion is done once at the point of handover. ECLIPSE writes `SWOF` as (S_w, k_{rw}, k_{row}) and `SGOF` as (S_g, k_{rg}, k_{rog}); this simulator's own deck format puts the oil curve second. Both orders yield plausible monotone curves, so a table read with the wrong convention swaps k_{rw} with k_{row} and nothing complains. The run therefore prints the four corner values at startup,

```
[satfunc] krw(Sw_max), krow(Sw_min), krg(Sg_max), krog(Sg_min)
```

all four of which must be the large end of their curve. One near zero means the order is wrong.

5.10.9 APPLEYARD

```
APPLEYARD  0 | 1
```

Default: 0.

Values:

`0` The saturation update is limited by magnitude alone, through `DSMAX`.

`1` In addition, an update that would cross a relative permeability endpoint is truncated to land just beyond that endpoint, so that a cell changes mobility regime over two iterations rather than one. Measured slower on SPE10 model 2; see the entry text.

Enables a modified Appleyard chop: a saturation update that crosses a relative permeability endpoint is truncated so that the iterate lands just beyond the endpoint rather than passing through it in a single iteration. The intent is to prevent oscillation across the point at which a phase becomes mobile.

The option is provided and measured. On SPE10 model 2 it was found to increase run time from 120 s to 143 s over the tuning interval, because the additional truncation increased the Newton count without permitting longer timesteps. It is therefore not enabled by default.

5.10.10 JACOBIAN

```
JACOBIAN  NUMERICAL | ANALYTIC | TENSORCORE
```

Default: `NUMERICAL`.

Values:

`NUMERICAL` Exact-structure finite-difference derivatives. Available for every model, including compositional. Slower, and incompatible with `AIM`.

`ANALYTIC` Hand-derived exact derivatives. Available for two-phase and three-phase black oil. Faster, better conditioned, and required by `AIM`. Recommended, and set by every supplied deck.

`TENSORCORE` The analytic system with its connection blocks formed as a batched operator product. Needs a `TENSORCORE=1` build; see Section 6.5.

Selects how the Jacobian is formed. `ANALYTIC` uses exact derivatives, implemented for two-phase and three-phase black oil, and is both faster and better conditioned. `NUMERICAL` uses exact-structure finite differences and covers every model, including compositional.

`ANALYTIC` is required by `AIM`, because adaptive implicitness modifies the residual and the numerical form would differentiate a residual the simulator does not evaluate. All supplied decks set `ANALYTIC`.

The analytic path is implemented for two-phase and three-phase black oil. Compositional models fall back to the numerical path automatically. The two forms can be compared directly:

Example.

```
mpirun -np 1 ./nvrs_cuda deck.DATA --jac-check
```

which assembles both and reports the maximum relative difference without advancing the simulation. Agreement to the order of the finite-difference truncation error confirms that the residual and its derivatives describe the same system.

See also. `FDEPS` for the perturbation scale; `AIM`, which requires the analytic form.

5.10.11 TCPRECISION

```
TCPRECISION  TF32 | FP32
```

Default: `TF32`. Read only with `JACOBIAN TENSORCORE`.

Arithmetic of the operator product. `TF32` issues a Tensor Core GEMM with FP32 accumulation and needs CUTLASS at build time and compute capability 8.0 or newer at run time; the run stops if either is missing. `FP32` runs the same product on CUDA cores on any GPU.

5.10.12 TCVALIDATE

```
TCVALIDATE  0 | 1
```

Default: `0`. Read only with `JACOBIAN TENSORCORE`.

`1` measures the product against the analytic blocks at every assembly and reports the worst relative Frobenius and block differences at the end of the run.

5.10.13 FDEPS

```
FDEPS  <real>
```

Default: 10^{-7}.

Values: A small positive real. 10^{-6} to 10^{-8} is the useful range: too large a value introduces truncation error, too small a value introduces cancellation. Relevant only when `JACOBIAN` is `NUMERICAL`.

Perturbation scale for the numerical Jacobian.

5.10.14 SCHEME

```
SCHEME  FIM | SEQUENTIAL
```

Default: `FIM`.

Values:

`FIM` Fully coupled Newton on the complete residual. The validated path, and the subject of this manual.

`SEQUENTIAL` Pressure equation with total mobility, followed by an implicit fractional-flow saturation update. Experimental. `SEQ` and `IMPES` are accepted as synonyms.

Selects the solution scheme. `FIM` is the fully coupled Newton formulation described in Chapter 6. `SEQUENTIAL` selects an IMPES-like formulation in which an elliptic pressure equation is solved with total mobility, followed by an implicit fractional-flow saturation update. The sequential path is experimental.

5.10.15 AIM

```
AIM  0 | 1
```

Default: 1.

Values:

`0` Fully implicit. Every cell carries implicit pressure and saturation unknowns, and every face mobility is evaluated at the new time level.

`1` Adaptive implicit. Cells below the `AIMCFL` threshold are explicit in saturation; wells and cells at a front remain fully implicit. Requires `JACOBIAN ANALYTIC`.

Selects adaptive implicit time stepping. When enabled, a cell whose throughput CFL over the coming sub-step is below `AIMCFL` is treated as explicit in saturation: its face mobilities are held at the values from the start of the step, and its saturation couplings are omitted from the Jacobian. Well cells and cells at a saturation front remain fully implicit.

`AIM 0` selects the fully implicit scheme, in which every cell carries implicit pressure and saturation unknowns.

Section 6.7 develops the method and reports its effect. Adaptive implicitness requires `JACOBIAN ANALYTIC` and does not apply to compositional runs; in either case the simulator reverts to the fully implicit scheme.

The scheme is stated in the run banner in both cases, and the proportion of cells treated explicitly is reported for each sub-step on a line beginning `[aim]`. A proportion that does not vary with the timestep indicates that the criterion is not being evaluated as intended and should be investigated.

Example.

```
AIM       1        -- adaptive implicit (the default)
AIMCFL    0.5      -- threshold; 1.0 is the stability limit
JACOBIAN  ANALYTIC -- required by AIM
```

See also. `AIMCFL` for the threshold, `JACOBIAN` for the prerequisite, and Section 6.7 for the formulation and the measured effect.

Common errors. Enabling `AIM` with `JACOBIAN NUMERICAL`, which silently reverts to fully implicit; and comparing a run against an older result without noting that the scheme changed. Set `AIM 0` to reproduce a fully implicit result exactly.

5.10.16 IMPLICIT

```
IMPLICIT  FIM | AIM
```

Default: `AIM` (equivalently `AIM 1`).

Values:

`FIM` Fully implicit. Equivalent to `AIM 0`. `FULL` and `0` are also accepted.

`AIM` Adaptive implicit. Equivalent to `AIM 1`. `1` is also accepted.

Alternative spelling of `AIM`, provided for decks that prefer to name the scheme rather than set a numeric flag. The two keywords are interchangeable; where both appear, the later one in the deck takes effect.

5.10.17 AIMCFL

```
AIMCFL  <real>
```

Default: 0.5.

Values: A positive real. The theoretical stability limit of an explicit saturation update is 1.0, so values above it are not advised. 0.3 to 0.7 is the useful range: smaller values are more conservative and place fewer cells in the explicit set, larger values place more. Setting it very small approaches fully implicit behaviour at the cost of the flag computation.

Threshold of the throughput CFL below which a cell is treated explicitly. The theoretical stability limit of an explicit saturation update is unity; the default provides a factor of two in margin. Larger values place more cells in the explicit set and reduce the assembly and operator cost, at increasing risk that the timestep controller is forced to compensate.

Raising the threshold places more cells in the explicit set, which reduces assembly and operator work per iteration. The risk is that a cell whose stability margin has been consumed forces the timestep controller to compensate, which costs far more than the work saved. The measured behaviour on SPE10 model 2 is that the proportion explicit falls from 86% to 54% as the timestep grows through the run, at a fixed threshold of 0.5.

See also. Section 6.7, which derives the criterion and explains why the mobility derivative rather than the mobility appears in it.

5.11 Linear solver control

5.11.1 KRYLOV

```
KRYLOV  FGMRES | BICGSTAB
```

Default: `FGMRES`.

Values:

`FGMRES` Flexible GMRES. Required when the preconditioner varies between iterations, which is the case here, and the recommended choice.

`BICGSTAB` Stabilised biconjugate gradient. Lower memory, since no Krylov basis is stored, but no flexibility guarantee. `BICG` is accepted as a synonym.

Outer Krylov accelerator. Flexible GMRES is required if the preconditioner varies between iterations, which is the case whenever the multigrid cycle is not a fixed linear operator. BiCGSTAB uses less memory, since it does not store a Krylov basis.

5.11.2 RESTART

```
RESTART  <integer>
```

Default: 30.

Values: A positive integer, the number of basis vectors before a restart. 20 to 90 is the useful range. Memory and orthogonalisation cost grow with this value. The general default is 30; the heterogeneous SPE10 pressure systems favour 60 when paired with CGS1.

Restart length of FGMRES. The Krylov basis requires two vectors of length $b_s N$ per restart level, so memory grows linearly with this value. A longer basis reduces the number of restarts but increases the cost of each iteration through the orthogonalisation; measurements on the benchmark cases favour 30 on the smaller benchmark cases and 60 on matched SPE10.

5.11.3 ORTHOPASSES

```
ORTHOPASSES  1 | 2
```

Default: 2.

Number of classical Gram–Schmidt projection passes used to orthogonalise each FGMRES Arnoldi vector. Two passes (CGS2) are the robust general setting. One pass removes one dense projection, one global reduction, and one basis update per Krylov iteration, but should be selected only after comparing the nonlinear trajectory and solution with CGS2. The matched SPE10 deck uses one pass; its 100-day validation changed pressure by only 0.046 psi RMS and saturation by 4.54×10^{-5} RMS while nearly halving orthogonalisation time.

See also. `KRYLOV` and `RESTART`.

5.11.4 LINTOL

```
LINTOL  <real>
```

Default: 10^{-6}.

Values: A positive real. 10^{-2} is recommended and is used by every benchmark deck; 10^{-1} is too loose for reliable Newton convergence, and values below 10^{-4} waste iterations on accuracy the Newton step cannot use.

Relative reduction of the linear residual required within a Newton iteration. This is an inexact-Newton forcing term, not an accuracy requirement on the final answer, which is governed by `NEWTONTOL`.

A value of 10^{-2} is recommended, and is used by all supplied benchmark decks. Tightening it to 10^{-6} on the verification case increased Krylov iterations from 250 to 992 and the run time

from 82 s to 188 s while changing the Newton count from 43 to 49; the additional linear accuracy purchased nothing.

This is a forcing term in the inexact-Newton sense: it controls how accurately each linear system is solved, not how accurately the non-linear problem is solved. The latter is governed by `NEWTONTOL`. Solving the linear system more accurately than the Newton step can exploit is the most common source of wasted work in a reservoir simulation.

`LINFORCINGMAX` optionally enables a bounded adaptive forcing term. The requested linear tolerance then varies between `LINTOL` and that maximum according to recent nonlinear reduction. Zero disables it. This is an experimental control: a value of 0.08 reduced linear work on matched SPE10 but caused many more short timesteps and increased wall time, so no supplied deck enables it.

The measurement supporting the recommended value, on the $46 \times 112 \times 22$ verification case: at 10^{-2} the first report step required 43 Newton iterations and 250 Krylov iterations and the run took 82 s; at 10^{-6} it required 49 Newton iterations and 992 Krylov iterations and took 188 s.

See also. `LINMAXIT`, `NEWTONTOL`, and Chapter 12.

5.11.5 LINMAXIT

```
LINMAXIT  <integer>
```

Default: 100.

Values: A positive integer. 100 to 300 is usual. Reaching the cap regularly indicates a preconditioner problem rather than a limit that should be raised.

Maximum Krylov iterations per linear solve. When the limit is reached the Newton iteration proceeds with the inexact update and a diagnostic is issued; persistent occurrences indicate that the preconditioner is not performing, and are addressed by the settings in this section rather than by raising the limit.

5.11.6 LINSOLVER

```
LINSOLVER  AMGX | HYPRE | GINKGO
```

Default: `AMGX`. `AMGLIB` is an accepted synonym.

Values:

`AMGX` NVIDIA AmgX (Naumov et al. 2015; Chapter D) supplies the stage-1 multigrid. The default. The hierarchy within AmgX is selected automatically by rank count (Section 6.10).

`HYPRE` HYPRE BoomerAMG (Henson and Yang 2002; Chapter D) supplies the stage-1 multigrid, with classical PMIS coarsening.

`GINKGO` Ginkgo (Anzt et al. 2022; Chapter D) supplies the stage-1 multigrid, with parallel graph matching aggregation, and runs the whole V-cycle in half precision by default (`GINKGOPREC`). Single GPU.

Selects the library providing the stage-1 pressure multigrid. Everything else in the solver is unchanged by this choice: the outer Krylov method, the emulated-precision operator, the quasi-IMPES weights and the second CPR stage are identical, and both libraries run their multigrid in single precision.

The hierarchy used within AmgX is selected automatically according to the rank count; see Section 6.10.

A binary built without AmgX falls back to HYPRE, and one built without Ginkgo falls back from `GINKGO` to AmgX; both report this at start-up. Ginkgo is installed by `benchmark/install_ginkgo.sh` and found by the build through `GINKGO_DIR`.

This keyword selects the library. It does not select the hierarchy used within AmgX, which is chosen by rank count (Section 6.10) and reported in the banner:

Example.

```
AmgX     : classical PMIS hierarchy (automatic: strongest on one rank)
```

Measured on the verification case over the full schedule, one GB200: AmgX completes in 0.89 s against 3.00 s for BoomerAMG, requiring 1,279 Krylov iterations against 6,830 over a comparable nonlinear path (239 Newton against 264), so it is a better preconditioner for this operator rather than merely a faster implementation. On Norne it is 44.95 s against 71.51 s for BoomerAMG and 70.42 s for the fastest Ginkgo arm.

See also. `NVRS_AMGX_ALGO` and `NVRS_AMGX_CONFIG` in Section 3.8.

5.11.7 GINKGOPREC

```
GINKGOPREC  16 | 32
```

Default: `16`. Read only with `LINSOLVER GINKGO`.

Values:

`16` The hierarchy, the smoothers, the coarse solve and the vectors of the V-cycle are IEEE half precision. `HALF` and `FP16` are accepted.

`32` The identical Ginkgo hierarchy in single precision, for measuring the effect of the precision alone. `SINGLE` and `FP32` are accepted.

The precision of the Ginkgo stage-1 multigrid. The outer Krylov method, the operator and the second CPR stage are unchanged. Before narrowing, the pressure matrix is scaled symmetrically by the inverse square root of its diagonal and the right-hand side is normalised by its largest entry, because field-unit coefficients exceed the largest finite half-precision value, 65,504; the correction is scaled back exactly. Both values of this keyword use the same scaling, so the comparison between them isolates the precision. The environment variables `NVRS_GINKGO_SWEEPS`, `NVRS_GINKGO_RELAX`, `NVRS_GINKGO_COARSE_ITERS`, `NVRS_GINKGO_MAXLEVELS`, `NVRS_GINKGO_MINCOARSE` and `NVRS_GINKGO_CYCLE` adjust the hierarchy without a rebuild (Section 3.8).

Example.

```
LINSOLVER   GINKGO
GINKGOPREC  16
```

5.11.8 PRECOND

```
PRECOND  CPR | MGR
```

Default: `CPR`.

Values:

`CPR` The built-in two-stage constrained pressure residual preconditioner. The default and the validated path.

`MGR` HYPRE multigrid reduction, which performs the pressure-saturation reduction inside the library using the block structure directly.

Selects the preconditioner. `CPR` is the two-stage constrained pressure residual method described in Section 6.8. `MGR` selects HYPRE's multigrid reduction, which performs a comparable pressure-saturation reduction inside the library, using the block structure directly rather than a diagonal approximation of it.

5.11.9 STAGE2

```
STAGE2  JACOBI | ZLINE | GINKGO
```

Default: `JACOBI`.

Values:

`JACOBI` Each cell's own $b_s \times b_s$ block is inverted and applied. Inexpensive and fully parallel.

`ZLINE` The block tridiagonal system along each vertical column of cells is factored and solved, coupling the correction in the direction of strongest transmissibility. Improves convergence; see Section 6.11 for the measured trade-off.

`GINKGO` Ginkgo's adaptive-precision block-Jacobi (Flegar et al. 2021; Chapter D) on the same cell blocks, generated from the FP64 operator at every setup. Each block inverse is stored in double, single or half precision according to its conditioning. Requires a build with Ginkgo; single GPU. `ADAPTIVE` is accepted.

Selects the CPR second stage. `JACOBI` applies the inverse of each cell's own $b_s \times b_s$ block. `ZLINE` solves the block tridiagonal system formed along each vertical column of cells, which couples the correction in the direction of strongest transmissibility.

Section 6.11 reports the measured behaviour of `ZLINE`, which improves convergence but is at present limited by the parallelism available to a one-thread-per-column implementation.

The choice matters most when the grid is strongly anisotropic. On a grid with `DZ` $\ll$ `DX`, vertical transmissibility exceeds horizontal by a similar ratio, and the error the pressure stage cannot represent lies along vertical columns, which a per-cell correction does not couple.

Measured on SPE10 model 2, where cells are $20 \times 10 \times 2$ ft: `ZLINE` reduces Krylov iterations per Newton iteration from 61.5 to 45.6, and increases wall-clock time, because the second-stage kernel becomes three times more expensive. The limitation is parallelism rather than arithmetic: one thread per areal column provides 13,200 threads on that grid.

See also. Section 6.11.

5.11.10 MIXED

```
MIXED  0 | 1
```

Default: 1.

Values:

`0` The preconditioner runs in double precision: the launchers start `nvrs_cuda_fp64` (AmgX `dDDI`, FP64 HYPRE).

`1` The preconditioner runs in single precision: the launchers start `nvrs_cuda` (AmgX `dFFI`, FP32 HYPRE).

Selects the precision of the CPR preconditioner. Precision is fixed at build time, so the keyword selects the binary: the launchers read the last `MIXED` line of the deck and start the matching build, and stop with the build command if it is missing. A binary started directly with a deck that disagrees reports the discrepancy, names the binary to use, and proceeds with the compiled behaviour. For an all-FP64 run, combine `MIXED 0` with `EMULATE 0`; the benchmark arm `amgx_fp64amg` does exactly that. (The older arm `amgx_fp64` keeps an FP32 multigrid and changes only the operator.)

5.11.11 EMULATE

```
EMULATE  0 | 1
```

Default: 1.

Values:

`0` The coupled operator is applied in native double precision.

`1` The coupled operator is applied in emulated double precision using a double-single error-free transformation on single-precision units. A pair of singles occupies the eight bytes of one double, so the memory traffic is that of the native operator; measured on Norne the two cost the same per apply to within 5%.

Applies the coupled operator in emulated double precision, using a double-single error-free transformation on single-precision units. Section 6.9 describes the arithmetic and the reason it is used for the operator and not for the preconditioner. Setting 0 selects a native double-precision operator.

5.11.12 BLOCKSPMV

```
BLOCKSPMV  0 | 1
```

Default: 1.

Values:

`0` Scalar CSR: one column index per scalar entry, one thread per row for the emulated operator and cuSPARSE for the native one. Reproduces runs made before this keyword existed bit for bit.

`1` Block storage: one cell-level column index per $b_s \times b_s$ block, four threads per row, for both the emulated and the native operator.

Selects how the Krylov method applies the coupled Jacobian. The Jacobian is assembled with a fixed block layout (every row of a cell has the same member cells, and entry v of member m in row a of cell c sits at $b_s^2\, p_c + (a\, n_c + m)\, b_s + v$), so a block kernel needs one index per block and none of the scalar row pointers or column indices: $4/b_s^2$ bytes of index traffic per nonzero instead of 4. The block index is built from the scalar CSR the first time a sparsity pattern is seen and checked against it entry by entry on the device; a pattern that does not have the block layout keeps the scalar kernels, and the run says so. On a Norne-sized pattern (340,032 rows, 7.0 million nonzeros) the emulated apply is 3.6× faster and the native apply 1.7× faster than the scalar path, with results equal to rounding (10^{-14} relative); `tests/bspmv_bench.cu` reproduces the measurement. The environment variable `NVRS_BLOCKSPMV` overrides the deck. Block sizes 2 to 8 are supported; the scalar kernels serve the rest.

5.11.13 CUDAGRAPH

```
CUDAGRAPH  0 | 1
```

Default: 1.

Values:

`0` Every kernel of the Krylov iteration is launched individually.

`1` The part of each Arnoldi step after the multigrid apply (CPRW, prolongation, the second CPR stage, the Arnoldi operator apply, the Gram–Schmidt projections and the norm) is recorded as three CUDA graphs per basis column and replayed.

On a model whose kernels run for a few microseconds each, launching them costs as much as running them, and the Nsight Systems traces of Norne show the GPU idle between launches. A graph replays the whole recorded sequence from one launch. The multigrid apply itself is not recorded: AmgX, HYPRE and Ginkgo synchronise with the host inside it. Graphs are recorded per basis column, because the vectors differ, and dropped whenever anything they bake in changes (a new sparsity, a change in the number of active well modes); the end of every run prints how many were captured and how many were replayed, and many captures per replay mean the graphs are not paying for themselves. A capture that fails turns graphs off for the rest of the run and says so.

On several ranks the tail communicates: two halo exchanges and up to seven reductions per Arnoldi step. Host-side MPI cannot be recorded in a graph, so in a build that finds NCCL

(NCCL_DIR or NCCL_HOME; the build summary says whether it did) those exchanges and reductions go through NCCL instead, which enqueues them on the stream: the tail is then captured and replayed on every rank, with no host synchronisation between its kernels. The communicator is created at the first solve and the run says so. Everything outside the tail (assembly, the multigrid libraries, host-side norms) keeps MPI. Without NCCL, or with NVRS_NCCL=0, several ranks keep the tail on MPI and do not capture it. Replays are bitwise identical to the individual launches. The environment variable NVRS_CUDA_GRAPH overrides the deck.

Independently of this keyword, each Arnoldi step now reads its results back to the host once rather than three times: both projection columns and the squared norm of the new basis vector are computed on the device and copied together.

5.11.14 AMGREUSE

```
AMGREUSE  <integer>
```

Default: 4.

Values: A non-negative integer. 0 rebuilds the hierarchy at every Newton iteration; 4 is the default; 10 is used by the benchmark decks. Large values are safe because the growth guard (AMGREUSEGROWTH) ends a reuse that stops paying.

Number of consecutive Newton iterations that may reuse a multigrid hierarchy built for an earlier iterate. A value of 0 rebuilds the hierarchy at every Newton iteration.

Within a timestep the pressure coefficients change little between iterates, so a hierarchy built for iterate k remains an effective preconditioner at iterate $k + 1$. The benchmark decks use 10.

The summary at the end of a run reports how the setting behaved:

Example.

```
AMG setup: 538 of 1989 rebuilt (1451 reused, 471 cut short by the
           growth guard), 25.5 ms each, 13.74 s total
```

A large number of guard-triggered rebuilds indicates that the hierarchy is going stale faster than the reuse limit assumes, and that AMGREUSE is set higher than this model supports; the guard is then paying for it in additional Krylov iterations before each rebuild.

See also. AMGREUSEGROWTH for the guard; Section 6.9.

5.11.15 AMGREUSEGROWTH

```
AMGREUSEGROWTH  <real>
```

Default: 1.3.

Values: A real > 1. 1.3 is the default; 2.0 is used by the benchmark decks. Values close to 1 rebuild eagerly and lose the benefit of reuse; very large values permit a stale hierarchy to persist and cost more in Krylov iterations than the rebuild would have cost.

Guard on hierarchy reuse. A reused hierarchy is discarded, and a fresh one built, as soon as the Krylov iteration count exceeds this multiple of the count achieved by the most recent freshly built hierarchy. This prevents reuse from exchanging a saved setup for a larger number of more expensive iterations. The benchmark decks use 2.0.

The run summary reports rebuilds, reuses and the number of reuses ended by this guard, so the trade-off can be inspected rather than assumed.

5.11.16 AMGFAST

```
AMGFAST  0 | 1
```

Default: 0.

Values:

`0` Coarse-level settings are taken individually from the keywords that follow.

`1` Enables the multi-GPU coarse-level bundle: two aggressive coarsening levels, truncated interpolation on them, an increased coarse-grid cutoff and device-resident coarsest-grid relaxation.

Enables, in a single keyword, the set of coarse-level measures that matter on multiple ranks: aggressive coarsening on two levels, truncated interpolation on those levels, an increased coarse-grid cutoff, and relaxation rather than direct solution on the coarsest grid. It alters the preconditioner, so it is not enabled by default.

It applies to HYPRE BoomerAMG only; AmgX and Ginkgo ignore it. It is a trade-off rather than a free saving: the V-cycle becomes cheaper and weaker. On one GB200, with `AMGREUSE` 10, it made Norne's first 336 days faster (4.0 s against 4.6 s) and the 1M case's first 300 days faster (6.3 s against 9.1 s), but SPE10's first 100 days much slower (56.1 s and 20,944 Krylov iterations against 23.4 s and 5,656), because on SPE10's permeability contrast aggressive coarsening loses most of the preconditioner. The SPE10 HYPRE runs therefore use `AMGFAST 0`. `scripts/hypre_screen.sh` repeats the comparison for a case, and the environment variables `NVRS_HYPRE_AMGFAST` and `NVRS_HYPRE_AMGREUSE` override both keywords in every benchmark script's HYPRE runs.

5.11.17 AMGMAXCOARSE

```
AMGMAXCOARSE  <integer>
```

Default: 0.

Values: A non-negative integer. 0 selects the rank-aware default of 64 on one rank and 2000 on several. Explicit values of 1000 to 6000 are useful on multiple ranks, where the cost of an additional level is dominated by communication latency.

Number of rows at which coarsening stops and the coarse system is solved directly. A value of 0 selects a rank-aware default: 64 on a single rank and 2000 on several. Each additional level costs a round of neighbour exchanges during setup and two during each cycle, and those rounds are dominated by latency once a level holds only a few hundred rows per rank.

5.11.18 AMGCOARSERELAX

```
AMGCOARSERELAX  <integer>
```

Default: 9.

Values:

`9` Gaussian elimination on the coarsest grid. Executed on the host in a GPU build, so each cycle incurs a gather across ranks and two transfers.

`18` ℓ_1-Jacobi on the coarsest grid, which keeps it on the device. A weaker solve, usually the better trade when the coarsest grid is small.

Relaxation scheme on the coarsest grid. The default, 9, is Gaussian elimination. On a GPU build this is executed on the host, so each cycle incurs a device-to-host transfer, a gather across ranks and a transfer back. Setting 18 selects ℓ_1-Jacobi, which keeps the coarsest level on the device. This is a weaker coarse solve and is usually the better trade when the coarsest grid is small and the cycle is applied once per Krylov iteration.

5.11.19 AGGLEVELS, AGGTRUNC, AGGPMAX

```
AGGLEVELS  <integer>
AGGTRUNC   <real>
AGGPMAX    <integer>
```

Default: 1, 0.0, 0.

Values: `AGGLEVELS` is a non-negative integer, ordinarily 0 to 2. `AGGTRUNC` is a real in $[0, 1)$, typically 0.2 to 0.4; 0 disables truncation. `AGGPMAX` is a non-negative integer, typically 4; 0 leaves the number of elements per row unlimited.

Number of aggressive coarsening levels, and the truncation factor and maximum number of elements per row of the interpolation operator on those levels. Aggressive coarsening reduces the number of levels at the cost of interpolation quality; truncation limits the fill that results.

5.11.20 AMGSEQTHRESH

```
AMGSEQTHRESH  <integer>
```

Default: 0.

Values: A non-negative integer. 0 leaves the library default in place.

Number of rows below which the remaining levels of the hierarchy are gathered onto a single rank. A value of 0 leaves the library default in place.

5.11.21 AMGNONGALERKIN

```
AMGNONGALERKIN  <real>
```

Default: 0.0.

Values: A real in $[0, 1)$. 0 disables sparsification. Small values such as 0.01 to 0.1 are usual where it is used at all; the operator is altered, so the effect should be measured.

Drop tolerance for non-Galerkin sparsification of the coarse operators. This changes the operator and is disabled by default.

5.11.22 AMGPRINT

```
AMGPRINT  <integer>
```

Default: 0.

Values:

`0` No hierarchy information is printed.

`1` The level table is printed once, giving the size and operator complexity of each level.

Prints the multigrid hierarchy once, giving the size and operator complexity of each level. Useful when comparing configurations or rank counts.

5.11.23 MGRRELAX, MGRSWEEPS, MGRINTERP, MGRSMOOTH, MGRSMOOTHITS

```
MGRRELAX       <integer>
MGRSWEEPS      <integer>
MGRINTERP      <integer>
MGRSMOOTH      <integer>
MGRSMOOTHITS   <integer>
```

Default: 18, 2, 2, 0, 1.

Values: `MGRRELAX` is a HYPRE relaxation index; 18 (ℓ_1-Jacobi) is the parallel choice. `MGRSWEEPS` and `MGRSMOOTHITS` are positive integers. `MGRINTERP` is a HYPRE interpolation index; 2 is diagonal-scaled. `MGRSMOOTH` is 0 for block Jacobi; the incomplete factorisation alternative is not available on the device.

Settings for the multigrid reduction preconditioner, active when `PRECOND` is `MGR`: the relaxation type and sweep count, the interpolation type, and the global smoother and its iteration count. The default relaxation, ℓ_1-Jacobi, is used because it is fully parallel; the default global smoother is block Jacobi, since the incomplete factorisation alternative is not available on the device.

5.12 Output control

5.12.1 OUTDIR

```
OUTDIR  <path>
```

Default: `OUTPUT`.

Values: Any writable path. Created if absent.

Directory for all output. Created if it does not exist.

5.12.2 ECLOUT

```
ECLOUT  0 | 1
```

Default: 0.

Values:

`0` No ECLIPSE-format output.

`1` Writes `.EGRID`, `.INIT`, `.UNRST`, `.SMSPEC`, `.UNSMRY` and `.RSM`. Required for the comparison workflow of Chapter 9.

Writes the ECLIPSE file family: `.EGRID`, `.INIT`, `.UNRST`, `.SMSPEC`, `.UNSMRY` and `.RSM`. These are read by standard post-processing tools and are the basis of the comparison workflow of Chapter 9.

5.12.3 MAKECSV

```
MAKECSV  0 | 1
```

Default: 0.

Values:

`0` No CSV output.

`1` Writes `RSM_FVM.csv` with one row per report step.

Writes `RSM_FVM.csv`, containing one row per report step with the oil, water and gas rates, water cut, gas-oil ratio and bottom-hole pressure for each well.

5.12.4 MAKESUMMARY

```
MAKESUMMARY  0 | 1
```

Default: 0.

Values:

`0` No summary files.

`1` Writes `summary.txt` and `meta.json`. The latter carries the per-phase timings consumed by the reporting scripts.

Writes `summary.txt`, a human-readable account of the run, and `meta.json`, the same information in machine-readable form including the per-phase timings consumed by the reporting scripts.

5.12.5 MAKEVTK

```
MAKEVTK  0 | 1
```

Default: 0.

Values:

`0` No VTK output.

`1` Writes field snapshots in VTK format, one file per report step.

Writes field snapshots in VTK format.

5.12.6 MAKEGASSMANN

```
MAKEGASSMANN  0 | 1
```

Default: 0.

Values:

`0` No acoustic output.

`1` Writes acoustic impedance fields from Gassmann fluid substitution.

Writes acoustic impedance fields obtained by Gassmann fluid substitution, for use in seismic modelling workflows.

5.12.7 AUTOPLOT

```
AUTOPLOT  0 | 1
```

Default: 0.

Values:

`0` No plotting. Recommended inside ensemble and history-matching loops.

`1` Invokes the plotting script when the run completes.

Invokes the plotting script when the run completes. This should be disabled inside ensemble or history-matching loops, where the per-run plotting cost is significant and the figures are not examined.

5.13 Compositional keywords

5.13.1 ZMF

```
ZMF
  <z1> <z2> ... <znc>
/
```

Default: none; required for compositional runs.

Values: One record of n_c non-negative reals summing to 1.

Overall feed mole fractions, one per component. A compositional run without `ZMF` is rejected.

5.13.2 TCRIT, PCRIT, ACF, VMOL

```
TCRIT
  <Tc1> ... <Tcnc>
/
PCRIT
  <Pc1> ... <Pcnc>
/
ACF
  <w1> ... <wnc>
/
VMOL
  <v1> ... <vnc>
/
```

Default: none.

Values: One record of n_c positive reals each. Acentric factors are ordinarily in $[0, 1.5]$.

Critical temperatures in °R, critical pressures in psia, acentric factors, and liquid molar volumes in ft^3/lbmol, one value per component in each case.

5.14 Physical constants

These keywords expose constants that are frequently taken for granted, so that a deck records the convention it assumes rather than depending on the simulator's choice.

5.14.1 DARCY

```
DARCY  <real>
```

Default: 1.127×10^{-3}.

Values: A positive real. The field-unit value is 1.127×10^{-3}; changing it rescales every transmissibility and well index, and is intended only for reproducing another simulator's convention.

Transmissibility constant in field units, with dimensions of bbl·cP/(mD·psi·day·ft).

5.14.2 GRAVGRAD

```
GRAVGRAD  <real>
```

Default: 0.4335.

Values: A positive real in psi/ft per g/cm^3. The field-unit value is 0.4335.

Hydrostatic gradient in psi/ft per unit fluid density in g/cm^3.

5.14.3 REFACTOR

```
REFACTOR  <real>
```

Default: 0.2.

Values: A real in $(0, 1)$. 0.2 is the classical Peaceman result for a square grid block; 0.14 to 0.28 spans the values in common use for other block aspect ratios.

Ratio of the Peaceman equivalent radius to the cell dimension, r_e = `REFACTOR` × `DX`. The value 0.2 corresponds to the classical result for a square grid block; supplying it explicitly allows a deck to match another simulator's well index exactly.

Chapter 6

Numerical formulation

6.1 Governing equations

For each component p present in the model, conservation of mass over a control volume, expressed in surface volumes with $b_p = 1/B_p$, gives

$$\frac{\partial}{\partial t}\left(\phi\, b_p S_p\right) + \nabla \cdot \left(b_p\, \mathbf{u}_p\right) = q_p, \tag{6.1}$$

with the phase velocity given by the multiphase extension of Darcy's law,

$$\mathbf{u}_p = -\frac{k\, k_{rp}(\mathbf{S})}{\mu_p}\left(\nabla p - \rho_p g \nabla z\right) = -k\, \lambda_p \nabla \Phi_p, \tag{6.2}$$

where $\lambda_p = k_{rp}/\mu_p$ is the phase mobility and $\Phi_p = p - \rho_p g z$ the phase potential. Saturations satisfy $\sum_p S_p = 1$, which removes one unknown per cell.

For three-phase black oil the gas component appears both as free gas and dissolved in oil, so the gas equation carries the additional term $R_s\, b_o\, S_o$ in accumulation and $R_s\, b_o\, \mathbf{u}_o$ in the flux.

6.2 Spatial discretisation

Equation (6.1) is integrated over each cell and the flux evaluated with a two-point approximation. For the face between cells i and j,

$$F_p^{ij} = T_{ij}\, \lambda_p^{\mathrm{up}}\, \left(\Phi_{p,j} - \Phi_{p,i}\right), \tag{6.3}$$

where the face transmissibility uses the harmonic mean of the two cell permeabilities in the face direction,

$$T_{ij} = C\, \frac{2 k_i k_j}{k_i + k_j}\, \frac{A_{ij}}{d_{ij}}, \tag{6.4}$$

with C the constant set by `DARCY`, A_{ij} the face area and d_{ij} the cell separation. The harmonic mean is the correct average for flow in series, and it has the desirable property that a single low-permeability cell blocks the connection irrespective of its neighbour.

The mobility λ_p^{up} is evaluated at the *upstream* cell for that phase, determined by the sign of the potential difference in (6.3). Upstream weighting is applied per phase, so different phases across the same face may take their mobility from different cells, which is the physically correct behaviour when phases move in opposite directions under gravity.

6.3 Well model

A grid cell is much larger than a wellbore, so the radial pressure profile around the well cannot be resolved by the grid. The Peaceman model supplies the missing relationship between the well pressure and the pressure of the cell containing it:

$$q_p = \mathrm{WI}\,\lambda_p\,(p_{\mathrm{cell}} - p_{wf})\,, \qquad \mathrm{WI} = \frac{2\pi C k h}{\ln(r_e/r_w) + s}. \tag{6.5}$$

NVRS-CUDA uses a fixed equivalent radius r_e = `REFACTOR` × `DX` rather than deriving an anisotropic radius from the cell dimensions. The consequence is that the well index can be transferred to and from ECLIPSE-format decks without conversion, since it is exactly the quantity that appears in item 8 of `COMPDAT`.

Injectors are rate-controlled: the specified surface rate is imposed directly as a source term. Producers are pressure-controlled: the rate follows from the drawdown and the mobility of each phase in the completed cells, so a producer in a swept region naturally produces water.

6.4 Temporal discretisation and the Newton method

Accumulation is discretised by backward Euler, giving for each cell and each component the residual

$$R_p = \frac{V\phi}{\Delta t}\left(b_p S_p - b_p S_p^{n}\right) - \sum_f b_p F_p^f - q_p = 0, \tag{6.6}$$

which is non-linear in the primary unknowns through the mobilities, the formation volume factors and the well terms. The system is solved by Newton's method: at each iteration the Jacobian $J = \partial\mathbf{R}/\partial\mathbf{u}$ is assembled and the linear system $J\,\delta\mathbf{u} = -\mathbf{R}$ solved for the update.

The Jacobian has a 7-point stencil in cells and is stored as a block compressed sparse row matrix with $b_s \times b_s$ blocks, where b_s is the number of unknowns per cell. It is assembled either analytically or by exact-structure finite differences, selected by `JACOBIAN`; the two are compared by the `--jac-check` option.

6.4.1 Update limiting

The raw Newton update is limited before it is applied. Each component of the update is truncated in magnitude by `DPMAX` for pressure and `DSMAX` for saturations, and the resulting saturations are clamped to their physical bounds. Limiting the update rather than rejecting the iteration preserves the Newton direction while keeping every iterate inside the region where the relative permeability tables and property correlations are meaningful; an unconstrained early iteration can otherwise produce a saturation far outside $[0, 1]$, from which the iteration does not recover.

6.4.2 Convergence and failure

An iteration is converged when the residual norm has fallen below `NEWTONTOL` times its value at the first iteration of the sub-step. Failure to converge within `NEWTONMAX` iterations, and a non-finite residual, are both treated as a failed timestep rather than a fatal error: the state is restored from the beginning of the report step and the step is retried at half the length.

6.5 The Tensor Core Jacobian

This is an option added at compile time. The default build compiles none of it.

```
TENSORCORE=1 ./nvrs.sh install     # fetches CUTLASS, builds nvrs_cuda_tc
```

or, with the libraries already installed, `make TENSORCORE=1 CUTLASS_DIR=<checkout>`. Without `CUTLASS_DIR` the option builds with `TCPRECISION FP32` only. A deck selecting `JACOBIAN TENSORCORE` in a binary built without the option stops with this instruction.

For an oriented connection (i, j) the residual is a product of eleven features $z = (M_w, M_o, M_g, \Delta\Phi_w, \Delta\Phi_o, \Delta\Phi_g, b^u_w, b^u_o, b^u_g, R^u_s, R^u_v)$, with u the upstream cell of the phase and $F_\alpha = T_{ij} M_\alpha \Delta\Phi_\alpha$:

$$\begin{aligned} R_o &= -\big(b^u_o F_o + R^u_v b^u_g F_g\big), \\ R_w &= -b^u_w F_w, \\ R_g &= -\big(b^u_g F_g + R^u_s b^u_o F_o\big). \end{aligned}$$

The chain rule is therefore a matrix product,

$$\frac{\partial R}{\partial x_{ij}} = \underbrace{\frac{\partial R}{\partial z}}_{L_{\text{flux}} \in \mathbb{R}^{b_s \times 11}} \underbrace{\frac{\partial z}{\partial x_{ij}}}_{L_{\text{props}} \in \mathbb{R}^{11 \times 2b_s}},$$

with x_{ij} the primaries of both cells. Both factors are padded to 16×16 tiles and all connections are issued as one strided batched GEMM. Column v of the product is the coupling to primary v of cell i, and column $b_s + v$ the coupling to primary v of cell j. The path covers $b_s = 2$ and $b_s = 3$ on the connection list and the fixed stencil, including AIM, THPRES, capillary pressure and variable substitution.

The analytic kernel still runs first and supplies the residual, the accumulation and well terms, the norms and the CPR weights. Neighbour blocks are then replaced by the product, and the self block is corrected,

$$J_{ii} \leftarrow J_{ii} + \sum_{\text{conn}} \big(J^{\text{product}}_{\text{conn},i} - J^{\text{FP64}}_{\text{conn},i}\big),$$

so accumulation and wells keep FP64. The stage-2 inverse and the pressure row are rebuilt from the result. Because the analytic kernel still runs, this path adds to the assembly time rather than replacing it. The run ends with the seconds of the analytic kernel, fill, product and scatter reported separately, so the product can be compared with the scalar arithmetic it stands in for.

Each connection uses three tiles of 1 KB, so a field that does not fit in a quarter of free GPU memory runs in several batches. TF32 rounds its inputs to 11 significant bits, so the Jacobian is inexact while the residual stays FP64: Newton still converges to the same solution, but it may need more iterations.

```
cd benchmark
./run_tensorcore.sh                  # SMOKE and RESERVOIR, analytic then Tensor Core
./run_tensorcore.sh NORNE            # or any deck in decks/
cd ..
make TENSORCORE=1 test_tc_jacobian   # factorisation against all analytic kernels
./test_tc_jacobian tf32              # omit tf32 on a GPU older than 8.0
```

6.6 Timestep control

Each report step is attempted as a whole and subdivided on failure. The controller maintains a hint, the number of halvings that succeeded at the previous report step, and begins there rather than from a single step, so that a model which requires short steps does not rediscover this at every report boundary.

On failure the state is restored, the multigrid hierarchy is invalidated, and the number of sub-steps is doubled. Invalidating the hierarchy matters: the retry begins from a different state, and preconditioning it with a hierarchy built for the state that failed would start the retry from the preconditioner that had just proved inadequate.

On success the step length is increased only after `DTGROW` consecutive report steps have converged at the first attempt. Without this condition a step that has just been reduced is immediately doubled again, and the controller oscillates between a step that is too long and the reduction that repairs it, discarding a set of Newton iterations on each cycle.

6.7 Adaptive implicit

6.7.1 Motivation

In a fully implicit step every cell carries an implicit saturation unknown. In most reservoir models the great majority of cells experience very little saturation change over a timestep: they are far from any front, their mobilities are effectively constant, and the implicit treatment of their saturation adds unknowns and couplings without adding accuracy.

Adaptive implicit retains implicit pressure everywhere, because pressure is elliptic and its influence is global, while treating saturation explicitly in cells where doing so is stable. This is a standard technique in reservoir simulation; the description here documents the specific criterion and implementation used by NVRS-CUDA.

6.7.2 Formulation

For a cell marked explicit, the face mobilities in (6.3) are evaluated at the saturations from the start of the timestep whenever that cell is the upstream cell, and no derivative of those mobilities with respect to saturation is contributed to the Jacobian. The flux is then a linear function of the implicit unknowns.

The accumulation term remains implicit in every cell. This is what keeps each row well posed: the diagonal of an explicit cell's saturation row retains $V\phi\, b_p/\Delta t$, so the row determines the new saturation directly given the fluxes. The sparsity pattern of the matrix is unchanged, which means the linear solver, the preconditioner and every downstream component operate exactly as in the fully implicit case.

6.7.3 The switching criterion

A cell is marked explicit when the dimensionless quantity

$$\mathrm{CFL}_c \;=\; \frac{\Delta t}{V\phi} \sum_f T_f \; |\Delta\Phi_p| \left| \frac{\partial \lambda_p}{\partial S} \right| \tag{6.7}$$

is below `AIMCFL`. This is the Courant-Friedrichs-Lewy condition written in reservoir variables: volumetric flux divided by pore volume is the reciprocal residence time of the cell, so (6.7) is the familiar $v\,\Delta t/\Delta x$. The stability limit of an explicit update is unity; the default threshold of 0.5 provides a factor of two in margin.

Two features of (6.7) deserve emphasis, because both were established by measurement during development and an implementation that omits either does not work.

The derivative, not the mobility. The stability of an explicit saturation update is governed by the sensitivity of the flux to saturation, $\partial\lambda/\partial S$, and not by the mobility itself. For Corey-type relative permeability curves the derivative near a front is several times the mobility, so a criterion built on λ underestimates the CFL number and marks cells adjacent to a front as stable when they are not. The observable consequence is that the timestep controller collapses to whatever those cells tolerate: in development this appeared as a timestep pinned at 0.39 days and a Newton count more than doubled relative to the fully implicit run.

Both directions of flow. The sum in (6.7) runs over all faces with the donor cell's mobility for each phase, counting flow into the cell as well as out of it. A criterion using only outflow marks the cell immediately downstream of a front as explicit, because its own water is still immobile and it therefore exports very little; meanwhile an implicit upstream neighbour continues to fill it at a rate frozen for the sub-step. The receiving cell then overfills beyond its physical bounds, the saturation clamp of Section 6.4.1 prevents the residual from closing, and the Newton iteration cannot converge at any timestep.

6.7.4 When the criterion is evaluated

The flux-dependent part of (6.7) is measured from a *converged* state and applied to the following sub-step. It cannot be evaluated at the start of a sub-step, because at that point the pressure field is whatever the previous solve produced; on the first sub-step of a run that is the uniform initial condition, in which every potential difference is zero. The measured throughput is then zero everywhere and the entire field is marked explicit, including the cells about to receive the full injection rate.

Accordingly the simulator is fully implicit until a converged measurement exists. The first sub-step of each report step is therefore an ordinary fully implicit step, and a timestep reduction resets to fully implicit, because the rates measured during the discarded attempt describe a state that has been rolled back.

6.7.5 Measured behaviour

On SPE10 model 2 over 2000 days on a single GB200, with all other settings identical:

Scheme	Wall clock	Newton iterations	Krylov iterations
Fully implicit	402.6 s	1984	203,414
Adaptive implicit	**342.1 s**	**1789**	**175,728**

The proportion of cells treated explicitly falls from 86% early in the run, when the timestep is short, to 54% at the end, when it has grown to 12.5 days. This is the expected behaviour: (6.7) is proportional to Δt.

The improvement is smaller than adaptive implicitness delivers in simulators that solve the coupled system directly, and the reason is worth recording. The principal benefit of the method is conventionally a reduction in the size of the coupled linear system. Here the linear solver already reduces the problem to a pressure system through CPR, and that system is unaffected by the treatment of saturation; the saturation couplings that adaptive implicitness removes were being handled by the inexpensive second stage. What remains is the reduction in assembly and operator work, which is what the measurement above reflects.

6.8 The linear solver

6.8.1 The character of the system

The linearised system inherits the mixed character of the equations. The pressure part is near-elliptic: its influence is global, its condition number grows like $\mathcal{O}(h^{-2})$ under grid refinement, and it responds to multigrid. The saturation part is hyperbolic: its influence is local and directional, and it responds to local smoothing but not to coarsening.

A single preconditioner cannot serve both. An incomplete factorisation addresses the local coupling and leaves the global pressure error untouched, so its iteration count grows with the grid. Multigrid on the full system coarsens the saturation equations, for which coarsening has no meaning.

6.8.2 Constrained pressure residual

CPR applies a different method to each character, in two multiplicative stages. Given a residual $\mathbf{r}$:

1. **Pressure stage.** Form the scalar pressure system $A_p = WJC$, where C restricts to the pressure unknown and W holds the quasi-IMPES weights, taken as the first row of each inverted cell block. Restrict the residual with the same weights, apply one multigrid V-cycle, and prolong the correction.

2. **Local stage.** Compute the residual remaining after the first stage and apply a local smoother to it, by default the inverse of each cell's own $b_s \times b_s$ block.

It is essential that the same weights W that build A_p also restrict the residual. Restricting with a simple selection of the pressure row while building A_p with W solves a different problem, and the outer Krylov method stalls.

The complete two-stage operator is used as a preconditioner within flexible GMRES. Flexibility is required because the multigrid V-cycle is not a fixed linear operator across iterations, particularly when the hierarchy is reused and rebuilt on the schedule described in Section 6.9.

6.8.3 Hierarchy reuse

Rebuilding the multigrid hierarchy at every Newton iteration is the single largest cost in a direct implementation. Within a timestep, however, the pressure coefficients change little between iterates, so the hierarchy built for one iterate remains an effective preconditioner for the next.

`AMGREUSE` sets how many rebuilds may be skipped. Because a stale hierarchy is not always harmless, `AMGREUSEGROWTH` provides a guard: as soon as the Krylov count on a reused hierarchy exceeds that multiple of the count achieved by the last freshly built one, the hierarchy is discarded and rebuilt. The run summary reports rebuilds, reuses and guard-triggered rebuilds separately, so the trade-off is visible.

6.9 Mixed precision

6.9.1 Principle

Every kernel in a reservoir simulator is limited by memory bandwidth rather than by arithmetic throughput: the arithmetic intensity of a sparse matrix-vector product is far below the machine balance of any modern GPU. Precision therefore matters where it changes the number of bytes moved, and is close to free where it only changes the number of floating-point operations.

This leads to an asymmetric assignment:

Component	Precision	Reason
State and residual	FP64	Determines the answer.
Krylov recurrences	FP64	Orthogonality is sensitive to rounding.
Operator apply	df64	FP64 semantics at FP32 bandwidth.
Stage-1 multigrid	FP32	An approximation by construction.
Stage-2 smoother	FP32	An approximation by construction.
CPR weights	FP64	Requires inverting an ill-conditioned block.

The justification for running the preconditioner in single precision is that a preconditioner is an approximation by definition. Error introduced within it changes the *rate* at which the Krylov method converges, not the *limit* to which it converges, which is fixed by the residual and the outer recurrences in double precision.

6.9.2 Emulated double precision

The operator apply requires double-precision semantics, because it forms the residual that defines the answer, but it is also the largest consumer of memory bandwidth in the solve. NVRS-CUDA therefore applies it in a double-single representation: each double is carried as an unevaluated pair of single-precision numbers,

$$a = a_{\mathrm{hi}} + a_{\mathrm{lo}}, \qquad a_{\mathrm{hi}} = \mathrm{fl}_{32}(a), \qquad a_{\mathrm{lo}} = \mathrm{fl}_{32}(a - a_{\mathrm{hi}}),$$

and products and sums are accumulated with error-free transformations, so that the result carries about 48 significant bits, close to double precision, using only single-precision arithmetic units.

This is double-word arithmetic in the sense of Dekker and of Bailey's QD library. It is not the Ozaki scheme, which splits whole matrices so that low-precision products are exact and which cuBLAS provides for dense matrix multiplication. A pair of single-precision numbers occupies the same eight bytes as one double, so the representation does not reduce memory traffic; its purpose is to deliver double-precision-quality products from single-precision units. Setting `EMULATE 0` selects the native double-precision operator.

6.9.3 Measured budget

On the verification case, the distribution of wall-clock time by arithmetic precision is 61% single precision, 32% emulated double precision and 7% native double precision. This distribution is unchanged when the multigrid library is replaced, which indicates that it is a property of the method rather than of a particular implementation.

6.9.4 Mixed precision against all-FP64

The comparison that isolates the preconditioner's precision runs the same solver twice: `nvrs_cuda` (FP32 preconditioner, AmgX `dFFI`) and `nvrs_cuda_fp64` (FP64 preconditioner, AmgX `dDDI`, native FP64 operator). The FP64 run replays the accepted time steps of the mixed run (`DTSEQFILE`), so both follow the same schedule and differ only in precision. `benchmark/precision_fp64_vs_mixed.sh` runs Norne, SPE10 and BENCH3P on one GPU and the MILLION family (1M to 5M cells) on 1, 2 and 4 GPUs, logs every residual (`NVRS_RESLOG`), and writes tables, plots and the field and well report of `compare.py` with the FP64 run in the reference position.

Case	GPUs	Mixed, s	FP64, s	FP64/mixed	V-cycle FP64/mixed	Newton mixed/FP64
SPE10	1	52.05	49.90	0.96	1.17	352/317
BENCH3P	1	0.83	0.76	0.92	1.05	239/212
MILLION 1M	1	20.65	20.87	1.01	1.23	693/683
MILLION 2M	1	38.47	39.16	1.02	1.27	694/684
MILLION 4M	1	74.18	75.24	1.01	1.32	693/683
MILLION 5M	1	90.96	94.36	1.04	1.33	695/685
MILLION 5M	2	67.02	67.31	1.00	1.10	362/362
MILLION 5M	4	63.73	61.73	0.97	1.07	362/363

Measured on GB200, one run per configuration. The FP32 V-cycle is 1.05 to 1.33 times cheaper than the FP64 one, more so on larger grids, but the V-cycle is only 11 to 27% of the run, so the end-to-end difference is within ±8%: on GB200 the precision split is runtime-neutral. Where FP64 is faster (SPE10, BENCH3P) the mixed run took more Newton iterations. Krylov iterations per solve and the accuracy each solve reached are the same in both builds, and the answers agree: on SPE10 at day 1000 the pressure differs by 0.005 psi RMS (0.04 psi at most) and water saturation by 2.3×10^{-5} RMS; on BENCH3P at day 2000 by 0.0002 psi RMS. Every producer curve agrees with $R^2 \geq 0.8$. On Norne the two runs separate after about day 316 even on the same time steps, and are under investigation.

The case for the split is therefore not speed on GB200 but independence from FP64 throughput: the part of the solve that dominates the work does not need FP64, and the emulated operator removes the remaining dependence on native FP64 units at a measured cost of 1.28× (GB200) and 1.21× (VR72) per apply.

6.10 Multigrid hierarchies

6.10.1 The two hierarchies

Algebraic multigrid constructs its coarse levels from the matrix alone. Two families of construction are relevant here.

Classical coarsening partitions unknowns into coarse and fine sets using a strength-of-connection threshold, then builds an interpolation operator from the strong connections. The PMIS variant used here performs the partitioning by a parallel maximal independent set computation. Classical hierarchies are more expensive to construct and generally represent heterogeneous problems better.

Aggregation groups nearby unknowns into aggregates and defines interpolation by injection from the aggregate. Construction is cheaper and maps well onto the device, at the cost of a coarse representation that is less faithful when coefficients vary sharply.

6.10.2 Automatic selection

NVRS-CUDA selects between them according to the number of ranks:

Ranks	Hierarchy	Reason
1	Classical PMIS	The stronger preconditioner on heterogeneous permeability. On SPE10 model 2 it approximately halves both the Krylov count and the run time relative to aggregation.
≥ 2	Aggregation	The construction that functions under a domain decomposition.

AmgX itself supports classical coarsening on several GPUs; the restriction comes from what was observed in this integration. Here the distributed classical setup exhausted device memory while constructing its communication maps, within seconds of the first setup, at two, three and four ranks alike, with either the D1 or the D2 interpolation operator, and on problems of which each rank holds a smaller share than the single-rank configuration that completes successfully. This may be a matter of configuration or library version rather than a limitation of AmgX. Aggregation is unaffected.

The selection is reported in the run banner together with its reason, and may be overridden with `NVRS_AMGX_ALGO`. Forcing classical coarsening on several ranks remains available deliberately, so that a later version of the library can be evaluated by setting one environment variable.

> **Note.** A strong-scaling study should pin one hierarchy with `NVRS_AMGX_ALGO`. A speed-up computed from a single-rank classical run against a multi-rank aggregation run measures the change of hierarchy as well as the change of rank count, and understates the parallel efficiency.

6.11 The ZLINE second stage

Cells in reservoir models are frequently much thinner than they are wide. When `DZ` $\ll$ `DX`, vertical transmissibility exceeds horizontal by the same ratio, and the error that the pressure stage cannot represent lies predominantly along vertical columns. A per-cell block-Jacobi second stage never couples along those columns.

`STAGE2 ZLINE` replaces it with a block-tridiagonal solve along each vertical column: the diagonal and the two vertical coupling blocks are extracted from the assembled Jacobian, the tridiagonal system is factored once per Newton iterate by a block Thomas recursion, and the correction is obtained by forward and backward substitution. Columns are mutually independent, so no colouring is required and the sweep is a single kernel launch.

On SPE10 model 2, where the cells are $20\times10\times2$ ft, this reduces Krylov iterations per Newton iteration from 61.5 to 45.6, a reduction of 27%, and the multigrid, operator and orthogonalisation costs fall in proportion. The wall-clock time nevertheless increases, because the second-stage kernel itself becomes three times more expensive: one thread per areal column provides only 13,200 threads on this grid, which is insufficient to occupy a large GPU.

The option is therefore documented, measured and disabled by default. Realising its convergence advantage requires a parallel cyclic reduction within each column, which would expose parallelism proportional to the number of cells rather than the number of columns.

Chapter 7

Parallel execution

7.1 Domain decomposition

The global grid is partitioned into a $p_x \times p_y \times p_z$ array of rectangular blocks, one per MPI rank, with one GPU bound to each rank. The factorisation is chosen automatically to minimise the total number of ghost cells, subject to no axis being divided more finely than it has cells.

A three-dimensional partition is materially better than a one-dimensional one. On a $200 \times 200 \times 25$ grid across four ranks, slabs cut perpendicular to k give each rank $200 \times 200 \times 6$ cells with two 200×200 ghost faces: 80,000 ghost cells against 250,000 owned, a surface-to-volume ratio of 32%. A $2 \times 2 \times 1$ partition gives each rank $100 \times 100 \times 25$ cells with four 100×25 faces: 10,000 ghost cells, or 4%. The advantage widens on every coarse level of the multigrid hierarchy, because a one-dimensional cut exhausts the layers available to divide while a three-dimensional cut continues to shrink in all directions.

`NVRS_DECOMP` overrides the automatic choice when a specific partition is required, for example to reproduce a previous measurement.

7.2 The halo exchange

The two-point flux stencil reaches one cell across each face, so each rank stores a one-cell ghost layer for every face on which it has a neighbour. The layer is refreshed at every Newton iteration, before assembly, since the residual of a boundary cell depends on its neighbour's current primary variables.

When the MPI implementation is CUDA-aware, device pointers are passed directly to the exchange. Otherwise the data is staged through host memory, which adds two transfers per exchange per face. The banner reports which path is in use.

Ghost cells are stored after the owned cells in the local index space, so that owned cells remain contiguous. This is required by the linear algebra: the pressure system must present each rank's rows as a contiguous global range.

7.3 Scaling behaviour

Iteration counts are invariant to the rank count to within about one per cent. This is an important diagnostic: it establishes that the preconditioner is not degrading when the grid is partitioned, and therefore that any time lost to parallelism is overhead rather than a change in the algorithm.

The overhead is concentrated in the multigrid setup. Parallel coarsening is a sequence of small neighbour exchanges and global reductions whose cost depends on the number of levels and ranks rather than on the number of cells each rank holds. Splitting the grid therefore adds this cost without dividing it.

The practical consequence is a threshold below which decomposition does not pay. As a guideline, maintain at least 5×10^5 cells per GPU. Below that the coarse levels hold a few hundred

rows per rank and the ranks spend their time in collective operations rather than in arithmetic. On the 113,344-cell verification case, four GPUs are slower than one; this is a statement about the size of the model relative to the machine, not a deficiency of the implementation.

7.4 Multi-node execution

The decomposition is not aware of node boundaries; it minimises ghost cells irrespective of where the ranks are placed. On a multi-node run the placement of ranks therefore matters, and ranks that share a node should hold adjacent blocks. `scripts/slurm_multinode.sh` provides a starting point for batch submission.

Confirm that the MPI in use is CUDA-aware before running on more than one node: the cost of host staging grows with the number of exchanges, and is most damaging exactly where the interconnect is slowest.

Chapter 8

Output files and post-processing

8.1 ECLIPSE-format output

With `ECLOUT` enabled, the simulator writes the standard file family under `OUTDIR`, named from `CASENAME`:

File	Contents
`.EGRID`	Grid geometry and active cell map
`.INIT`	Static properties: permeability, porosity, pore volume
`.UNRST`	Field snapshots at each report step
`.SMSPEC`	Summary vector definitions
`.UNSMRY`	Summary vector values
`.RSM`	Summary in text form

These files are read by standard post-processing tools and by the `resdata` and `ecl` Python packages, which is what makes a direct comparison against another simulator possible without any intermediate conversion.

8.2 Native output

`RSM_FVM.csv` Written when `MAKECSV` is set. One row per report step, with oil, water and gas rates, water cut, gas-oil ratio and bottom-hole pressure for each well.

`summary.txt` Written when `MAKESUMMARY` is set. A human-readable account of the run: configuration, per-phase timings, iteration counts and material balance.

`meta.json` The same information in machine-readable form. The reporting scripts consume this file.

`state_*.bin`, `rock.bin` Raw field snapshots and the static fields, in the simulator's own binary layout.

8.3 Reporting scripts

`scripts/plot_results.py` renders well curves, field slices and animations from a completed run and assembles them into a single self-contained HTML page.

`benchmark/compare.py` takes a directory containing a deck and two result trees in ECLIPSE format and produces a comparison report: well curves overlaid for both simulators with per-well differences, material balance, per-report-step field statistics, and three-dimensional renderings of pressure and each saturation for both simulators and their difference.

`scripts/backend_report.py` converts a tree of `scaling.json` files into an HTML performance report with wall-clock and scaling charts per case and configuration.

8.4 Locating the output of a benchmark study

A study driven by `scripts/backend_study.sh` writes:

Path	Contents
`backend_study/index.html`	The performance report
`backend_study/study.log`	Complete session log, all cases in order
`backend_study/<case>/<cfg>/scaling.json`	Timings and counts
`backend_study/<case>/<cfg>/console.log`	That arm's transcript
`runs_scaling/<stamp>/<n>gpu/run.log`	Individual run output
`backend_study/bench3p/opm/flow.log`	Reference simulator run
`backend_study/bench3p/physics.html`	Physical comparison report

Chapter 9

Verification and benchmarks

9.1 Approach

Two questions must be answered separately about any simulator: whether it produces the correct answer, and how quickly. Both are addressed by comparison against an independent implementation on identical problems, and the value of such a comparison depends entirely on the two inputs describing the same physical situation.

`benchmark/make_decks.py` therefore generates both decks from a single parameter set. Everything physical is declared once and emitted twice, in the NVRS-CUDA deck format and in ECLIPSE format.

9.2 Matching the formulations

Five adjustments are required for the two simulators to solve the same discrete system, each applied explicitly by the deck generator.

1. **Constant PVT.** `PVTCONST` pins the fluid to the deck constants, which can then be transcribed into an ECLIPSE PVT table with the same meaning.
2. **Zero fluid compressibility.** With ϕ and b_p fixed the accumulation term reduces to $V\phi/\Delta t$ exactly, matching a `PVTW` and `PVCDO` specification with zero compressibility.
3. **Non-zero rock compressibility.** A small sandstone value is retained in the ECLIPSE deck alone. The reference simulator's CPR implementation builds its pressure equation from the derivative of accumulation with respect to pressure; with every compressibility zero that derivative vanishes and the run fails at the first timestep. The effect on pore volume over the schedule is of order 10^{-3}.
4. **No dissolved gas.** `RS` zero, and `DISGAS` omitted on the other side, make both formulations immiscible.
5. **Matched gravity and well index.** Surface densities are chosen so that ρ_{surface}/B equals the reservoir density used by NVRS-CUDA, and the connection transmissibility factor is written explicitly into `COMPDAT` item 8.

9.3 The former three-phase residual difference

Early verification runs used a Baker product in NVRS-CUDA while OPM Flow used the ECLIPSE saturation-weighted default:

$$\text{NVRS-CUDA:}\quad k_{ro} = \frac{S_o}{1 - S_{wi} - S_{wr}}\, k_{row}(S_w)\, k_{rog}(S_g),$$

$$\text{ECLIPSE default:}\quad k_{ro} = \frac{S_g k_{rog} + (S_w - S_{wco}) k_{row}}{S_g + S_w - S_{wco}}.$$

That discrepancy has been removed. NVRS-CUDA now evaluates the second expression, with its analytic derivatives with respect to both transported saturations. The old Baker expression is shown because reports made before the three-phase correction used it and must not be compared directly with this release. Stone I and Stone II remain unsupported.

The benchmark still provides two useful cases. `BENCH2P` is the term-by-term two-phase check. `BENCH3P` exercises mobile gas and the shared ECLIPSE-default three-phase path. A difference in a current `BENCH3P` run is therefore no longer excused as a relative-permeability model difference; it belongs to properties, controls, nonlinear stepping or the linear solve and should be diagnosed as such.

9.4 Performance on the verification case

`BENCH3P`, $46 \times 112 \times 22$ or 113,344 cells, twenty report steps covering 2000 days, both simulators using their GPU linear solver on one GB200:

	Reference	NVRS-CUDA (HYPRE)	NVRS-CUDA (AmgX)
Wall clock	160.5 s	37.1 s	**8.3 s**
Linear solve	70.4 s	34.4 s	5.8 s
Assembly, wells, properties	85.6 s	2.7 s	2.5 s
Newton iterations	630	571	571
Krylov iterations	21,776	38,431	**7,171**

Both simulators conserve water to better than one per cent of the injected volume, which is the check that establishes they are solving the same system rather than two that merely resemble each other.

9.5 The size ladder

`MILLION.DATA` at $200 \times 200 \times 25$ and its refinements to two, four and five million cells measure how the configurations scale with problem size and rank count. Aggregation improves with problem size, reaching 1.36× on four GPUs at five million cells and still rising at the top of the measured range, while a classical hierarchy under this library remains close to flat because its setup cost does not divide with the grid.

9.6 SPE10 model 2

The Tenth SPE Comparative Solution Project, model 2, is the standard test of a solver's tolerance of heterogeneity: $60 \times 220 \times 85$, or 1,122,000 cells at $20 \times 10 \times 2$ ft, with a permeability contrast of 3×10^7, two phases, one central injector and four corner producers, over 2000 days. It was constructed as an upscaling exercise, and most participants in the original study were unable to run the fine grid.

`benchmark/make_spe10.py` retrieves the public dataset into `dataset/SPE10/` on first use and generates the deck. Two properties make the resulting case faithful rather than approximate. The quadratic Corey curves specified by the project, with $S_{wc} = S_{or} = 0.2$, are analytically identical to the two-phase kernel's built-in curves at `SWI` = `SWR` = 0.2, so no saturation table is involved on either side. And the generator verifies that $K_y = K_x$ in the published data before writing a deck, rather than assuming the isotropy that NVRS-CUDA requires.

Reading the dataset requires the per-cell `POROFILE` and `PERMZFILE` keywords, since the project supplies porosity as an independent measurement and a vertical permeability that is not a scalar multiple of the horizontal.

9.6.1 Results

Each configuration below was measured against the one above it, on a single GB200:

Configuration	Wall clock	Speed-up
Aggregation hierarchy, fully implicit	861.3 s	1.00×
Classical PMIS hierarchy	402.6 s	2.14×
Classical PMIS with adaptive implicit	**342.1 s**	**2.52**×

Both improvements are defaults, so a single-GPU run with no additional settings produces the final row.

9.6.2 Configurations that were rejected

The following were measured and not adopted. They are recorded because a negative result is information, and because each represents an avenue that would otherwise be re-explored.

Configuration	Outcome
Two smoothing sweeps	235 s against 287 s for the baseline: cheaper cycles, convergence unchanged.
`SIZE_4` aggregates	197 s: as above.
`MULTICOLOR_DILU` smoother	Substantially better convergence per iteration, but the smoother is approximately three times more expensive; net loss.
Two V-cycles per application	The timestep controller failed to recover; the case did not complete.
`RESTART` 60	Orthogonalisation more expensive per iteration at one third of the iteration count.
`APPLEYARD` 1	143 s against 120 s: more Newton iterations without longer timesteps.
`STAGE2 ZLINE`	Krylov per Newton reduced from 61.5 to 45.6; kernel cost three times block-Jacobi. See Section 6.11.

Each can be reproduced with `scripts/spe10_tune.sh`.

9.6.3 Why the case is reported on one GPU

SPE10 is quoted single-GPU in the published literature, and at 1.12×10^6 cells it is below the threshold at which decomposition becomes profitable (Chapter 7). Running it on several ranks would additionally force the aggregation hierarchy, which on this field is the weaker preconditioner by a factor of approximately two. The size ladder is the appropriate vehicle for scaling measurements.

9.7 Reproducing the benchmarks

```
GPUS="1 2 3 4" ./scripts/backend_study.sh all
```

runs every case and regenerates the report. Individual studies are named `bench3p`, `million`, `2m`, `4m`, `5m` and `spe10`.

The precision study of Section 6.9.4:

```
cd benchmark
./precision_fp64_vs_mixed.sh                          # NORNE SPE10 BENCH3P MILLION
REPEATS=3 ./precision_fp64_vs_mixed.sh SPE10 BENCH3P  # medians of three
SIZES="1m 2m" GPUS="1 2" ./precision_fp64_vs_mixed.sh MILLION
```

Results go to `runs/precision_<stamp>/`: `RESULTS.txt`, `results.csv`, `fields.csv`, `plots/` (wall clock, scaling, residuals) and `compare/compare_<CASE>.html`, and one archive, `runs/precision_<stamp>.tar.gz`, without the field output. `PREC_SKIP="NORNE"` leaves a case out when the tables and plots are redrawn with `precision_report.py`.

Chapter 10

Norne validation and OPM parity

10.1 Purpose and acceptance boundary

This chapter records the field-scale validation state of version 2. Its purpose is not to claim that two independently implemented simulators are bitwise identical. It establishes what was made equivalent, how that equivalence is tested, which differences remain, and what a new result must show before it is accepted.

The reference is OPM Flow reading the original metric Norne deck. NVRS-CUDA reads the same corner-point model and writes the same ECLIPSE result family. Both result trees are then read by the same Python ECL reader. This last point matters: using one reader removes differences caused by a plotting package, summary interpolation convention, or unit label. The report is generated by `benchmark/compare.py` as `compare_NORNE.html`.

10.2 Well responses against OPM Flow

Figure 10.1 is what the acceptance boundary below is ultimately about. It is the oil production rate for every Norne producer over the full 3312-day history, taken from `compare_NORNE.html`, with OPM Flow in red and NVRS-CUDA under its five solver arms (AmgX, BoomerAMG, and Ginkgo in FP16, FP32 and FP16 with adaptive block-Jacobi) in the colours of the legend.

Over the whole field, cumulative oil is 425.07 M STB against Flow's 422.78, the recovery factor is 41.5% against 41.8%, and the oil balance closes: what the saturation field gives up accounts for 100% of what the well rates report. **20 of the 22 producers reach $R^2 \geq 0.70$ on oil rate and 19 on all four rate quantities at once**, 21 on water cut and all 22 on gas rate; every producer's score is in Table 10.1.

The release criterion used during this work has four parts:

1. the complete 247-step schedule finishes with AmgX, HYPRE and every Ginkgo arm;

2. the solver arms produce essentially the same reservoir and well response, showing that a remaining discrepancy is not being introduced by one linear solver;

3. oil and gas producer profiles follow the reference over the intervals in which the wells are active, and derived ratios contain no unit-induced thousand-fold error or zero-target cross-flow spike;

4. the faster AmgX path remains faster than the OPM reference and no new full-field host scan is placed inside every Newton iteration.

The current result satisfies this engineering boundary. Every score is retained in the report as computed, rather than clipped.

10.3 Reference case and reproducibility identity

The comparison is meaningful only if a run can be identified. Record at least the following with every Norne result:

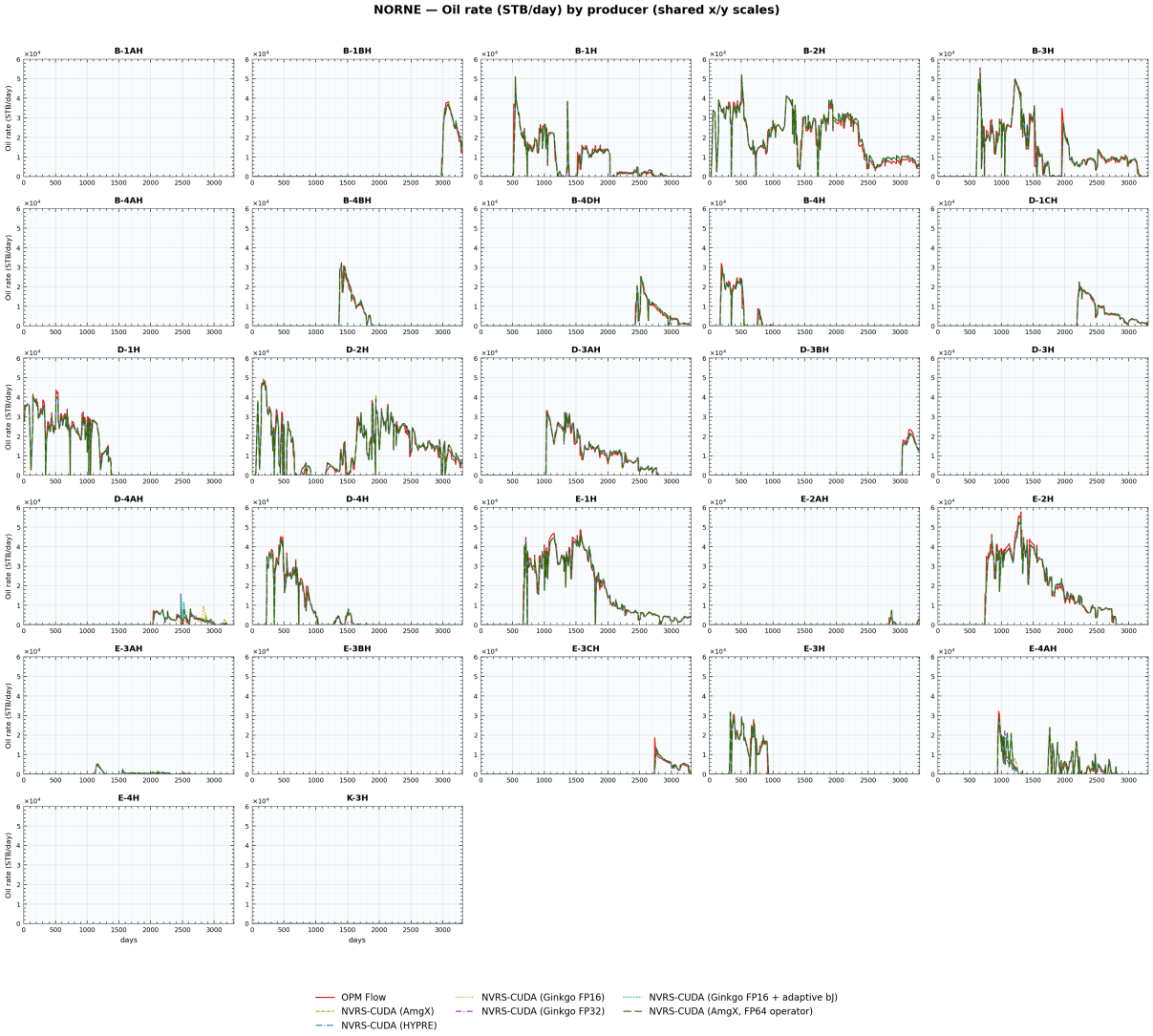


Figure 10.1: Norne oil production rate, one panel per producer.

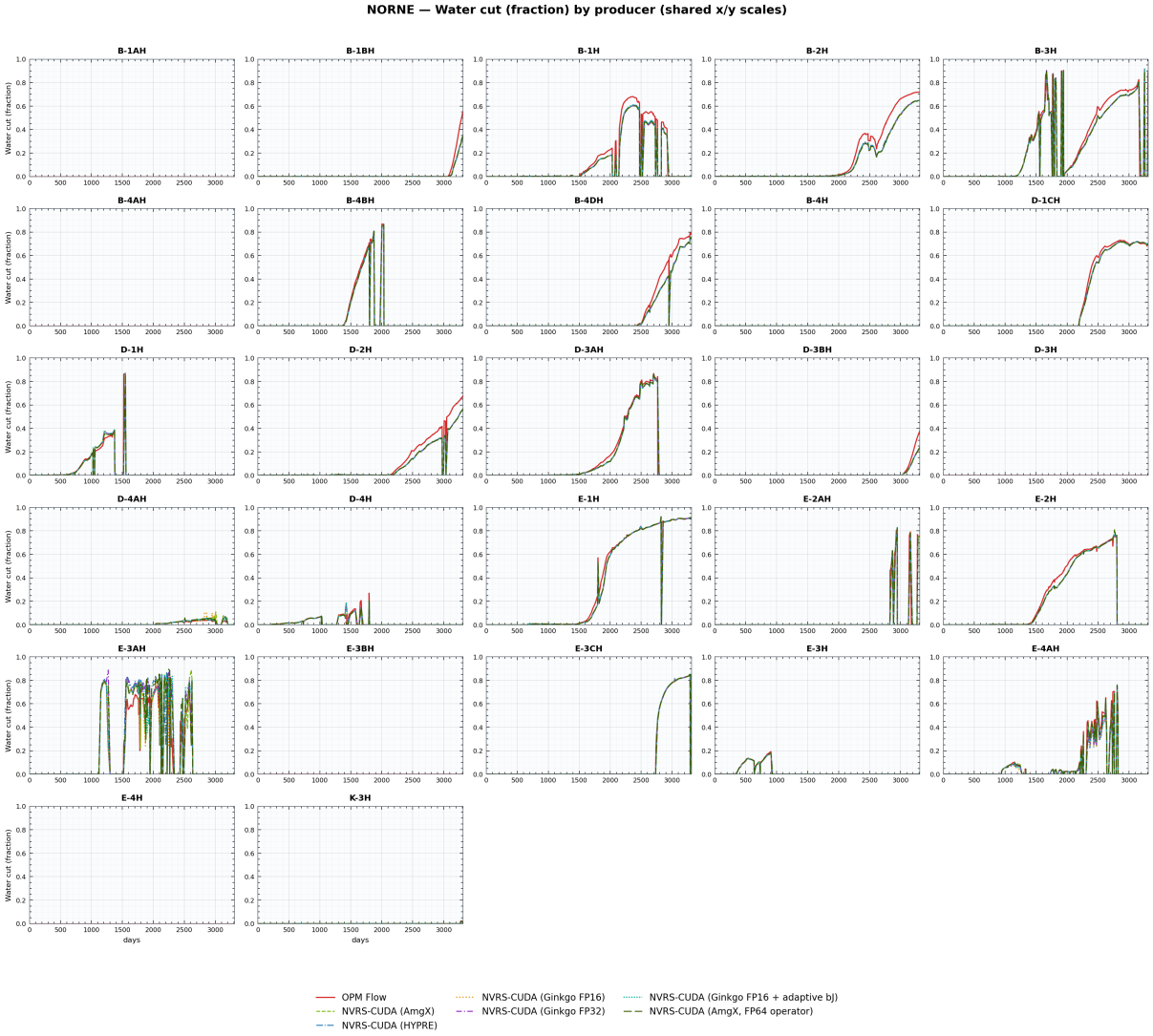


Figure 10.2: Norne water cut, one panel per producer: all 22 producers at $R^2 \geq 0.70$.

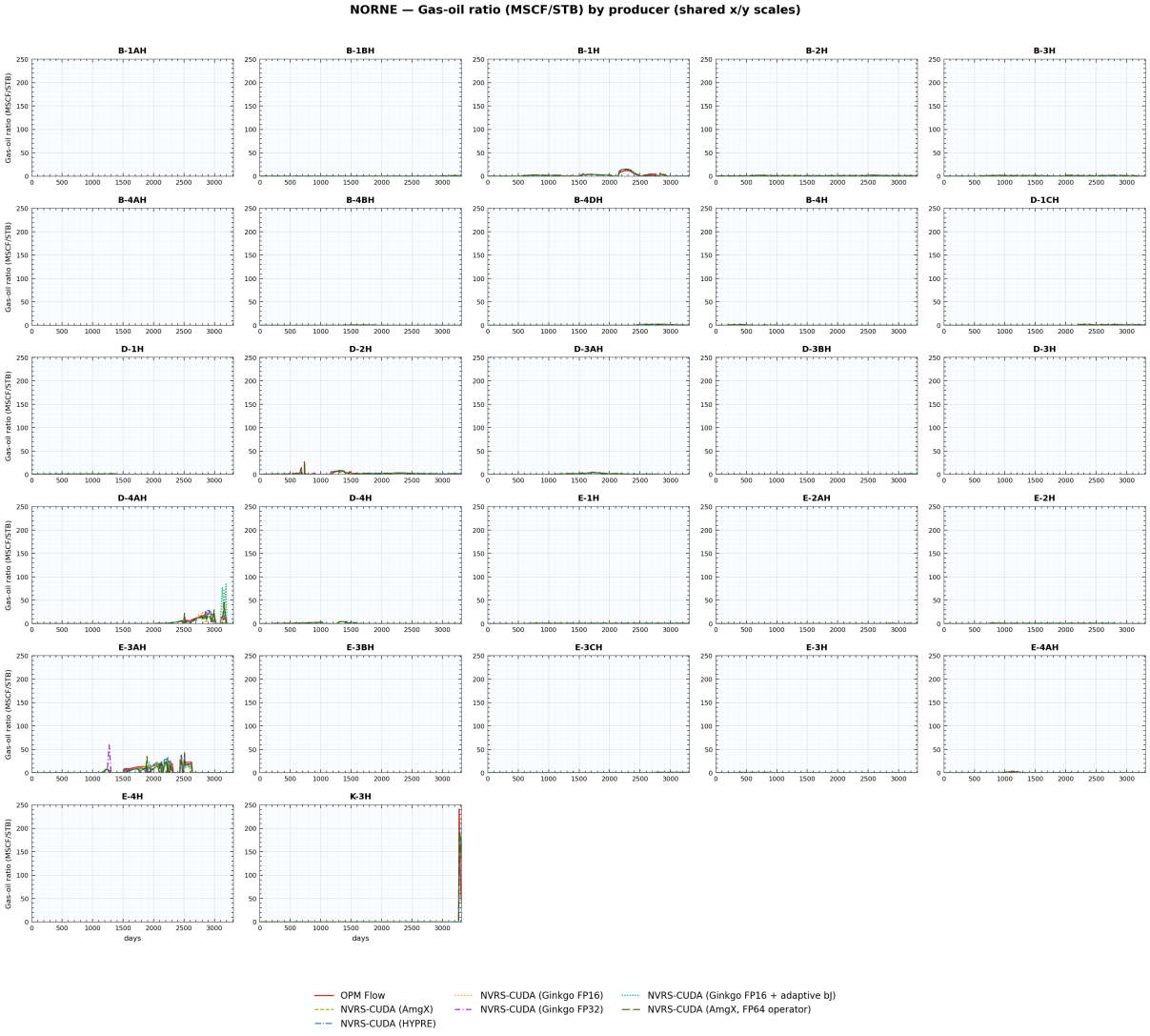


Figure 10.3: Norne gas-oil ratio, one panel per producer.

Item	Required record
Source revision	Full Git commit and branch. The result in this edition was produced from the merged source at commit `d72244c`.
Deck identity	Path to `NORNE_ATW2013.DATA`, the include tree, and whether any include was edited.
Units	The field deck must report `METRIC`; internal and output units are described in Section 10.6.
Backends	`LINSOLVER AMGX` and `LINSOLVER HYPRE`, each on one GPU for the published comparison.
Architecture	CUDA architecture and GPU model; the reported run used a GB200 build with `sm_100`.
Libraries	HYPRE and AmgX prefixes printed in the run banner.
Result time	The timestamp generated into `compare_NORNE.html`; the result quoted here is the version 2 GB200 run.

Do not label a report only by a directory name such as `latest`. The history of this case contains physically different runs: endpoint arrays were off in some, the older three-phase k_{ro} convention was active in others, and early well reports read only PVT region zero. A commit and banner turn an apparently visual comparison into reproducible evidence.

10.4 Cluster installation runbook

The following is the complete cluster procedure from a clean checkout. It is written out because three directories are easily confused: the Git worktree, the two HYPRE installation prefixes, and the AmgX installation prefix.

10.4.1 Get the code

```
git clone --branch feature/norne-examples --single-branch \
  https://github.com/clementetienam/physicsnemo.git
cd physicsnemo/examples/reservoir_simulation/reservoir_simulator
```

If the worktree already exists, preserve local changes before pulling:

```
git status --short
git stash push -u -m "local-before-update"
git pull --ff-only origin feature/norne-examples
```

The untracked AmgX and HYPRE source/build directories may be excluded by the repository ignore rules. A stash does not delete an installation. Never use a recursive clean command at the repository root to solve a pull conflict.

10.4.2 Build or restore HYPRE

The simulator's mixed path links the FP32 HYPRE installation. The repository helper writes two prefixes and an environment file:

```
cd /workspace/project/full_reservoir_simulator/Mixed_precision
CUDA_ARCH=100 ./scripts/docker/install_hypre_sm100.sh
source hypre_env.sh
test -f "$HYPRE_FP32_DIR/include/HYPRE.h"
test -f "$HYPRE_FP64_DIR/include/HYPRE.h"
```

When those two tests succeed, a later `make clean` in the CUDA simulator does not affect HYPRE. It removes simulator objects only. If the Makefile reports "No HYPRE.h", source `hypre_env.sh` again and inspect the two tests before rebuilding anything.

10.4.3 Build or restore AmgX

```
cd /workspace/project/full_reservoir_simulator/GPU_reservoir_simulator/cuda
./benchmark/install_amgx.sh
source ../../amgx-install/amgx_env.sh
test -f "$AMGX_DIR/include/amgx_c.h"
```

The exact installed header may differ by AmgX revision; the installation script prints the prefix it staged. The simulator can build HYPRE-only, but a published AmgX result must show AmgX in the startup banner rather than a fallback warning.

10.4.4 Configure and compile

With the two environment files sourced, either build path is valid:

```
cd /workspace/project/full_reservoir_simulator/GPU_reservoir_simulator/cuda

# Direct nvcc build. Incremental: recompiles only changed translation units.
make -j"$(nproc)"

# Or CMake.
cmake -S . -B build \
  -DHYPRE_DIR="$HYPRE_FP32_DIR" \
  -DAMGX_DIR="$AMGX_DIR" \
  -DCMAKE_CUDA_ARCHITECTURES=100
cmake --build build -j
```

Do not run an additional `make` after `cmake --build`; those are alternative build trees. After pulling a change to `well_report.cu`, plain `make -j` is enough: Make rebuilds that object and relinks. A full `make clean` is needed only when changing compilers, architectures, or compile definitions.

10.4.5 Fetch, run and compare Norne

```
cd /workspace/project/full_reservoir_simulator/GPU_reservoir_simulator/cuda
./benchmark/get_norne.sh
cd benchmark

# Reuse an existing OPM reference tree.
SKIP_OPM=1 ./run_norne.sh

# Or regenerate both sides.
./run_norne.sh
```

The scripts create separate dated logs for AmgX and HYPRE and rebuild `compare_NORNE.html`. Preserve all three files. A screenshot is useful for discussion, but it cannot replace the HTML because the HTML carries the full field table, timing breakdown and all producer panels.

10.5 Schedule time semantics

An ECLIPSE `DATES` record closes the preceding interval. The controls read after a date apply to the interval that follows it. Applying the new snapshot to the interval that just ended shifts every opening, shutdown and target change by one report step. The resulting curves can look plausible because the schedule is dense, yet every well transition is displaced.

Let t_n be the nth report time and $\mathcal{C}_n$ the schedule state active on $[t_n, t_{n+1})$. The update order is

$$U_n \xrightarrow[\mathcal{C}_n]{\Delta t \le t_{n+1}-t_n} U_{n+1}, \qquad \mathcal{C}_n \longrightarrow \mathcal{C}_{n+1} \quad \text{only after accepting } U_{n+1}.$$

Internal timestep chops do not advance the schedule. A failed substep retries with the same $\mathcal{C}_n$ and a smaller Δt. This ordering is tested by the schedule unit suite and is one of the reasons a report-step plot should be compared on reference times, not merely by array index.

The Norne reader retains the observed oil, water and gas rates in `WCONHIST` separately from the control target. `WHISTCTL` overrides the record's control mode until it returns to `NONE`. An observed BHP is not silently made a pressure limit for a RESV-controlled history well. These distinctions match OPM's schedule state and prevent a record of what happened from being treated as a prediction constraint of a different type.

10.6 Metric Norne and the internal unit contract

Norne is a metric deck. NVRS-CUDA converts it once when the field model is constructed and solves in the field-unit convention used by the CUDA kernels. The conversion must be phase-aware:

$$q_\ell^{\mathrm{STB/d}} = q_\ell^{\mathrm{sm^3/d}}\, 6.28981077, \qquad \ell \in \{o, w\},$$

$$q_g^{\mathrm{scf/d}} = q_g^{\mathrm{sm^3/d}}\, 35.3146667.$$

Gas is stored internally in scf/day, not Mscf/day. The report divides by 10^3 when it labels a plotted value MSCF/day. Applying that factor on input as well would make every gas target one thousand times too small.

Pressure, length and transmissibility follow

$$p_{\mathrm{psi}} = 14.5037738\, p_{\mathrm{bar}}, \qquad z_{\mathrm{ft}} = 3.280839895\, z_{\mathrm{m}},$$

$$T_{\mathrm{field}} = T_{\mathrm{metric}} \frac{6.28981077}{14.5037738}.$$

Formation-volume factors remain numerically in reservoir volume per stock-tank volume; permeability remains millidarcy, viscosity centipoise, and time days.

For a producer report the ratios are dimensionless after consistent surface units:

$$\mathrm{WWCT} = \frac{q_w}{q_o + q_w}, \qquad \mathrm{WGOR} = \frac{q_g}{q_o}.$$

The software guards a zero denominator. A ratio is not evaluated from raw metric gas and converted oil, nor from plotted MSCF/day and internal scf/day. Such mixed bases create a factor of 10^3 or 5.615 that can masquerade as a physical gas mismatch.

10.7 Connection model and reported phase rates

For completion k, let WI_k be the deck or Peaceman connection factor, p_k the oil pressure in the cell, p_w the well reference BHP and Δp_k the hydrostatic correction from the reference depth to the completion. Define the signed drawdown

$$d_k = p_k - p_w - \Delta p_k.$$

Positive d_k is production under the simulator's reporting sign. With phase mobilities $\lambda_\alpha = k_{r\alpha}/\mu_\alpha$ and shrinkage factors $b_\alpha = 1/B_\alpha$, the surface component rates are

$$q_{o,k}^s = WI_k \left(b_o \lambda_o + R_v b_g \lambda_g\right) d_k,$$

$$q_{w,k}^s = WI_k b_w \lambda_w d_k,$$

$$q_{g,k}^s = WI_k \left(b_g \lambda_g + R_s b_o \lambda_o\right) d_k.$$

The R_s term is dissolved gas produced with oil. The R_v term is vaporised stock-tank oil produced with wet gas. Omitting either term changes both the phase report and the coefficient used to hold a surface target.

The total well rate is a sum over live completions. A completion may have a negative contribution when cross-flow is allowed; that signed value must be retained until the well sum is formed. Clipping each completion before the sum destroys conservation and changes the BHP that solves a target.

10.7.1 Hydrostatic completion pressure

The reference BHP belongs to the datum in `WELSPECS`, not separately to each completion. The correction is accumulated through the well column with a mobility-weighted mixture density,

$$\Delta p_k = g_f \int_{z_{\mathrm{ref}}}^{z_k} \rho_{\mathrm{mix}}(z)\, dz, \qquad \rho_{\mathrm{mix}} = \frac{\lambda_o\rho_o + \lambda_w\rho_w + \lambda_g\rho_g}{\lambda_o + \lambda_w + \lambda_g},$$

where g_f is the field-unit gravity factor. Oil and gas densities include the current dissolved/vaporised component content. Setting every completion directly to one BHP without Δp_k forces a horizontal datum onto a vertical well and generates artificial inter-completion flow.

10.7.2 Closed-form BHP for a rate target

For any linearised target, collect the applicable connection coefficient into A_k. Examples are the oil coefficient in the first rate equation, the water coefficient in the second, or their sum for liquid rate. Then

$$q^{\mathrm{target}} = \sum_k A_k(p_k - p_w - \Delta p_k).$$

The host control solve is the scalar closed form

$$p_w = \frac{\sum_k A_k(p_k - \Delta p_k) - q^{\mathrm{target}}}{\sum_k A_k}.$$

All ranks contribute their owned completions to the two sums before this equation is evaluated. Consequently a well split across MPI ranks receives one BHP and one global target; it is not independently rate-controlled on every rank.

The mobilities are frozen for this scalar update and are refreshed after the Newton state changes. This is a well-control outer update, not a modification to the reservoir Jacobian or either CPR backend.

10.8 History RESV conversion: derivation from OPM

A `WCONHIST` RESV record supplies observed surface phase rates, not a single reservoir-rate column. OPM first constructs a representative field state and then converts the observed surface vector to reservoir voidage. NVRS-CUDA now follows that procedure.

10.8.1 Representative region state

For a property x_c in cell c, define the hydrocarbon-pore-volume average

$$\langle x\rangle_{\mathrm{HCPV}} = \frac{\sum_c PV_c(1 - S_{w,c})x_c}{\sum_c PV_c(1 - S_{w,c})}.$$

It is used for pressure, saturated dissolved ratio and saturated vaporised ratio. If the region contains no hydrocarbon pore volume, the fallback is the full pore-volume average

$$\langle x\rangle_{PV} = \frac{\sum_c PV_c x_c}{\sum_c PV_c}.$$

The previous implementation averaged only the well's completions and weighted them by mobility. That made a field-region conversion depend on which cells a particular well happened to perforate. It also repeated table flattening in a hot path. The current region average is reduced across MPI once per report step and its immutable flattened PVT tables are cached for the run.

10.8.2 Effective dissolved and vaporised ratios

Let the non-negative observed surface rates be q_o^s, q_w^s and q_g^s. OPM infers the mixture ratios from those observations and caps them at the representative reservoir state:

$$R_s^* = \min\left(\langle R_s\rangle, \max\left(0, \frac{q_g^s}{q_o^s + \epsilon_o}\right)\right),$$

$$R_v^* = \min\left(\langle R_v\rangle, \max\left(0, \frac{q_o^s}{q_g^s + \epsilon_g}\right)\right).$$

The signed epsilons protect zero observations without changing the sign of a non-zero rate. In the production-only form above the rates are non-negative.

10.8.3 Surface-to-reservoir matrix

Oil and gas surface components are coupled by dissolution and vaporisation. With formation-volume factors B_o, B_w, B_g evaluated at the regional pressure and effective ratios, the determinant is

$$D = 1 - R_s^* R_v^*.$$

The reservoir phase voidages are

$$q_o^r = \frac{B_o}{D}\left(q_o^s - R_v^* q_g^s\right), \qquad q_w^r = B_w q_w^s,$$

$$q_g^r = \frac{B_g}{D}\left(q_g^s - R_s^* q_o^s\right).$$

Therefore the scalar total RESV target is

$$q_{\mathrm{RESV}} = \frac{B_o - R_s^* B_g}{D} q_o^s + B_w q_w^s + \frac{B_g - R_v^* B_o}{D} q_g^s.$$

This is the algebra used by OPM's `RateConverter`. A common error is to insert shrinkage factors $b = 1/B$ into the coefficients as though they were formation-volume factors. The direction of the conversion then reverses and gas-dominated targets become especially wrong.

Each well evaluates B_o, B_w and B_g from its `PVTNUM` region at the common field-region pressure. This matches OPM's use of a field-region state together with the well's PVT region.

10.9 Why a zero phase target stops the whole well

Several Norne history intervals contain an open producer under ORAT with a target of zero. It is tempting to solve only

$$\sum_k q_{o,k}^s = 0.$$

For a multi-completion well this equation permits large positive oil flow in one completion and negative flow in another. The oil contributions cancel, but water and gas generally do not. The apparent result is a zero oil target with enormous water/gas cross-flow, a water cut that jumps between zero and one, and severe WGOR spikes.

OPM's well primary variables do not use that interpretation. An active zero ORAT, WRAT, GRAT or LRAT target sets total well flow to zero. A history RESV record with all observed rates zero behaves the same way. NVRS-CUDA therefore marks the producer inactive for that schedule interval. Both its source and its well Jacobian disappear from the existing assembly path. At the next schedule interval the well array is rebuilt from the deck state, so a later non-zero control reopens it normally.

This correction is deliberately implemented in the host control state. It does not add a conditional or an extra branch to the CUDA reservoir kernels. It is the principal reason the new WWCT curves no longer consist solely of completion-cross-flow steps during zero-rate history intervals.

10.10 PVTNUM parity across physics, controls and reporting

Norne contains multiple PVT regions. Correct reservoir assembly is not enough: the well-control coefficients, reported phase rates and oil-in-place summary must select the same region as the cell being evaluated.

Let $r(c) = \texttt{PVTNUM}_c - 1$ be the zero-based region of active cell c. The contract is

$$\left(B_\alpha, \mu_\alpha, R_s, R_v\right)_c = \mathcal{P}_{r(c)}(p_c, R_{s,c}, R_{v,c}).$$

The mapping is performed through the grid's global index because region arrays are stored in global ECLIPSE order while the nonlinear state is stored in active/local order. Missing, invalid or single-region data falls back to region zero. That fallback is explicit and should be printed by diagnostics; it must not result from indexing an active cell directly into a global array.

The flattened host PVT representation is immutable after deck construction. It is cached by the PVT table object and reused in all well and summary paths. Before this change, rebuilding PVTO/PVTG branches during every rate-control update was both unnecessary and a possible source of the slowdown observed in intermediate Norne runs.

10.11 Group injection ceilings

For a group/phase pair, let Q_i be the surface target of injector i and Q_G the `GCONINJE` RATE ceiling. If

$$\sum_{i \in G} Q_i > Q_G,$$

the individual injector targets are scaled by

$$\gamma_G = \frac{Q_G}{\sum_{i \in G} Q_i}, \qquad Q_i^{\text{applied}} = \gamma_G Q_i.$$

If the sum is already below the ceiling, $\gamma_G = 1$. Water, gas and oil injection records are accumulated separately. A RESV ceiling requires a consistent reservoir-volume conversion before it can share this path; the current release therefore enforces RATE only. Reinjection and voidage-replacement fractions remain unsupported because they require the group's produced volumes.

Without this ceiling, early Norne runs injected the sum of all individual `WCONINJE` rates even where the group limited them. The reservoir then stored and produced far too much water. The defect was not a relative permeability issue: it was a schedule aggregation issue, and the water balance made that distinction visible.

10.12 Derived well quantities and denominator policy

The comparison reads the primary summary quantities WOPR, WWPR, WGPR and WBHP. WWCT and WGOR are derived consistently when a writer does not supply them. For production magnitudes,

$$\text{WWCT} = \frac{\max(q_w, 0)}{\max(q_o, 0) + \max(q_w, 0)},$$

$$\text{WGOR} = \frac{q_g^{\text{scf/d}}}{q_o^{\text{STB/d}}}.$$

If the denominator is below the report tolerance the ratio is recorded as zero/undefined according to the ECL writer's convention rather than divided by machine epsilon. This avoids assigning a physically impressive GOR to a well producing numerically zero oil.

The distinction between a primary and derived quantity is useful during diagnosis:

- WOPR, WWPR and WGPR diagnose component rate equations and controls;
- WWCT diagnoses the relative split between oil and water and is sensitive to water breakthrough timing;
- WGOR diagnoses both gas/oil rate equations and the near-zero-oil denominator;
- WBHP diagnoses the scalar control solve, completion pressure offsets and the mode that is binding.

A poor WWCT with good WOPR does not imply that the oil equation is wrong. It usually means the water front or water allocation differs. A poor WGOR with good WGPR may be caused by the oil denominator rather than the gas numerator.

10.13 How the report computes and displays R^2

For reference samples y_j and interpolated simulator samples $\hat{y}_j$ at the same OPM report times,

$$R^2 = 1 - \frac{\sum_j (y_j - \hat{y}_j)^2}{\sum_j (y_j - \bar{y})^2}, \qquad \bar{y} = \frac{1}{N} \sum_j y_j.$$

One is exact. Zero is no better than predicting the reference mean. A negative score is worse than that mean and is intentionally shown rather than clipped.

For well schedules, R^2 requires context. If a well is shut for most of its life, the denominator can be extremely small. One small timing difference at opening or shutdown then produces a score of -10^3 or -10^8 even when the two visible non-zero segments nearly overlap. This is mathematically correct but not a useful percentage similarity.

The report therefore uses a cumulative-by-time trace in each panel. The final number is useful for wells with sustained variance; the trace identifies the date interval that destroyed it. Read it together with:

1. the overlaid physical well curve;
2. active-day count and the magnitude of the reference signal;
3. normalised RMSE or integral error over active intervals;
4. opening, control-switch and shutdown timing;
5. cumulative oil, water and gas where available.

An engineering "70% similar" statement should therefore not be made by counting all final R^2 values above 0.70. For a balanced scorecard, use variance-weighted or active-time metrics and state the thresholds explicitly.

10.14 Measured Norne result, version 2

The current comparison was generated from the complete 247-step history on one GB200 GPU and repeated on one VR72 GPU (the reports under `results/GB200/` and `results/VR72/`), with all six solver arms run back to back on each machine. Version 2 stores the coupled operator in 3×3 blocks (`BLOCKSPMV`) and replays the Krylov tail from CUDA graphs (`CUDAGRAPH`); version 1 is the same code with scalar CSR and direct launches.

	GB200			VR72	
Run	**Wall clock**	**Linear solve**	**AMG**	**Wall clock**	**Over OPM**
OPM Flow	488.51 s	97.22 s (21%)	n/a	243.88 s	
NVRS-CUDA AmgX	**44.95 s**	23.60 s (54%)	14.56 s	**35.73 s**	10.87× / 6.83×
NVRS-CUDA AmgX, FP64 operator	43.19 s	22.48 s (53%)	13.82 s	34.91 s	11.31× / 6.99×
NVRS-CUDA HYPRE	71.51 s	49.91 s (70%)	34.12 s	73.54 s	6.83× / 3.32×
NVRS-CUDA Ginkgo FP16	80.55 s	56.30 s (70%)	41.52 s	50.02 s	6.06× / 4.88×
NVRS-CUDA Ginkgo FP32	70.42 s	49.94 s (70%)	34.89 s	49.31 s	6.94× / 4.95×
NVRS-CUDA Ginkgo FP16 + abJ	76.61 s	53.37 s (72%)	34.59 s	57.27 s	6.38× / 4.26×

The last column is the speed-up over OPM Flow on GB200 and on VR72. The GB200 AmgX run takes 1,497 Newton iterations against OPM's 1,985, rebuilds the AMG hierarchy 172 times at 10.1 ms each, retries 3 of the 247 report steps and discards 47 Newton iterations; HYPRE takes 1,506 and retries 6. The arms overlay in the well panels; five of the six agree to within 0.015 in mean per-well R^2 on each machine (AmgX 0.901 on GB200, 0.910 on VR72), and the sixth on each machine moved the water breakthrough of one producer, D-4AH, so the differences between the arms are solver performance. The fastest Ginkgo arm is faster than HYPRE on both machines and slower than AmgX, by 1.56× on GB200 and 1.38× on VR72; its FP16 V-cycle costs the same as its FP32 one.

10.14.1 What version 2 changed

Neither optimisation touches the multigrid V-cycle, which costs 0.31 ms on GB200 in both versions; both make the rest of the Krylov iteration cheaper.

Norne, GB200, AmgX	**Version 1**	**Version 2**
Operator apply per Krylov iteration	64.0 μs	**24.1 μs**
Orthogonalisation per Krylov iteration	91.0 μs	**44.8 μs**
Kernel launches, first 336 days (Nsight)	173,083	**129,499**
Wall clock, GB200	46.52 s	**44.95 s**
Wall clock, VR72	44.65 s	**35.73 s**

The run summary reports how often the graphs were reused (`Operator : block storage, ...; CUDA graphs on (N captured, M replays)`). On SPE10 each captured graph is replayed about 160 times; on Norne about twice, because hierarchy rebuilds, restarts and well-control refreshes change what the Krylov tail touches. `EMULATE 0` (native FP64 operator) and `EMULATE 1` cost the same per apply, 11.2 against 11.8 μs on GB200 and 11.7 μs each on VR72; the wall-clock differences between those two arms come from the Newton path they take.

10.14.2 Water inventory

At the common $t = 3260$ day output the comparison reports million stock-tank barrels, with injection integrated from the reported injector rates:

Run	W_0	W_{3260}	gained	produced	implied inj.	closure
OPM Flow	2596	3115	518.4	146.4	664.8	+3.8%
NVRS-CUDA (GB200)	2596	3119	523.0	133.7	656.7	+6.2%
NVRS-CUDA (VR72)	2596	3119	522.5	132.9	655.5	+6.6%

Both begin at the same water inventory and gain the same water to within 0.9%. The oil balance closes: what the saturation field gives up accounts for 100% of what the well rates report.

10.14.3 Field-state agreement

At day zero the pressure RMS difference is 2.055 psi (0.271% of the plotted range), water-saturation RMS is 0.001888 (0.205%), and oil/gas RMS differences are about one per cent of range. At day 3260 they are 327 psi (5.3% of the range) and 0.0150, with oil and gas RMS 0.019 and 0.017; VR72 gives 338 psi. Cumulative oil over the history is 425.07 M STB against OPM's 422.78 (0.54%), and the recovery factor 41.5% against 41.8%.

10.15 Per-well agreement

Each of the 22 producers is scored separately, on six quantities, by the ordinary R^2 of its whole history against OPM Flow: the number printed in each panel of the report's per-producer R^2 charts. The threshold is 0.70. Table 10.1 gives every value for the GB200 AmgX run, and Figure 10.4 the same scores as bars.

Table 10.1: Norne, R^2 against OPM Flow per producer and quantity, GB200, AmgX. Green $R^2 \geq 0.80$, amber 0.70 to 0.80, red below 0.70. Where OPM's value is constant over the history the error is measured against 5% of that level, so a shut-in period no longer yields a negative score.

Well	WOPR	WWPR	WWCT	WGPR	WGOR	WBHP	Well	WOPR	WWPR	WWCT	WGPR	WGOR	WBHP
B-1BH	0.99	0.76	0.84	0.97	0.98	0.98	D-3BH	0.99	0.78	0.86	0.99	0.98	0.90
B-1H	0.95	0.83	0.93	0.97	0.85	0.92	D-4AH	**0.67**	**−0.25**	**0.53**	0.92	0.72	0.97
B-2H	0.96	0.92	0.94	0.92	0.83	**0.28**	D-4H	0.98	0.95	0.89	0.96	0.97	0.99
B-3H	0.93	0.94	0.94	0.93	0.93	0.92	E-1H	0.97	0.98	0.98	0.94	0.93	0.91
B-4BH	0.98	0.99	0.99	0.96	0.93	0.99	E-2AH	0.86	0.96	0.90	0.81	0.74	0.99
B-4DH	0.95	0.92	0.95	0.97	0.95	0.95	E-2H	0.98	0.96	0.99	0.94	0.96	0.94
B-4H	0.98	0.98	0.96	0.98	0.97	1.00	E-3AH	0.93	**0.68**	0.84	0.86	**0.69**	0.99
D-1CH	0.99	0.98	1.00	0.97	0.97	0.96	E-3CH	0.84	0.97	0.99	0.82	0.97	0.78
D-1H	0.98	0.97	0.86	0.96	0.97	0.95	E-3H	0.98	0.98	0.98	0.97	0.98	0.98
D-2H	0.97	0.90	0.93	0.95	0.80	**0.53**	E-4AH	0.90	0.88	0.92	0.77	**0.52**	1.00
D-3AH	0.98	0.97	0.97	0.98	0.97	0.97	K-3H	**0.64**	**0.65**	0.93	0.97	**0.67**	0.95

Wells at $R^2 \geq 0.70$ of 22: WOPR **20**, WWPR **19**, WWCT **21**, WGPR **22**, WGOR **19**, WBHP **20**

- Oil rate: 20 of 22 producers at $R^2 \geq 0.70$; below are D-4AH (0.67) and K-3H (0.64), which opens in the last months of the history.
- Water rate: 19 of 22; below are D-4AH (−0.25), E-3AH (0.68) and K-3H (0.65).
- Water cut: 21 of 22; below is D-4AH (0.53).
- Gas rate: all 22; the lowest is E-4AH (0.77).
- Gas-oil ratio: 19 of 22; below are E-3AH (0.69), E-4AH (0.52) and K-3H (0.67).
- Bottom-hole pressure: 20 of 22; below are B-2H (0.28) and D-2H (0.53).

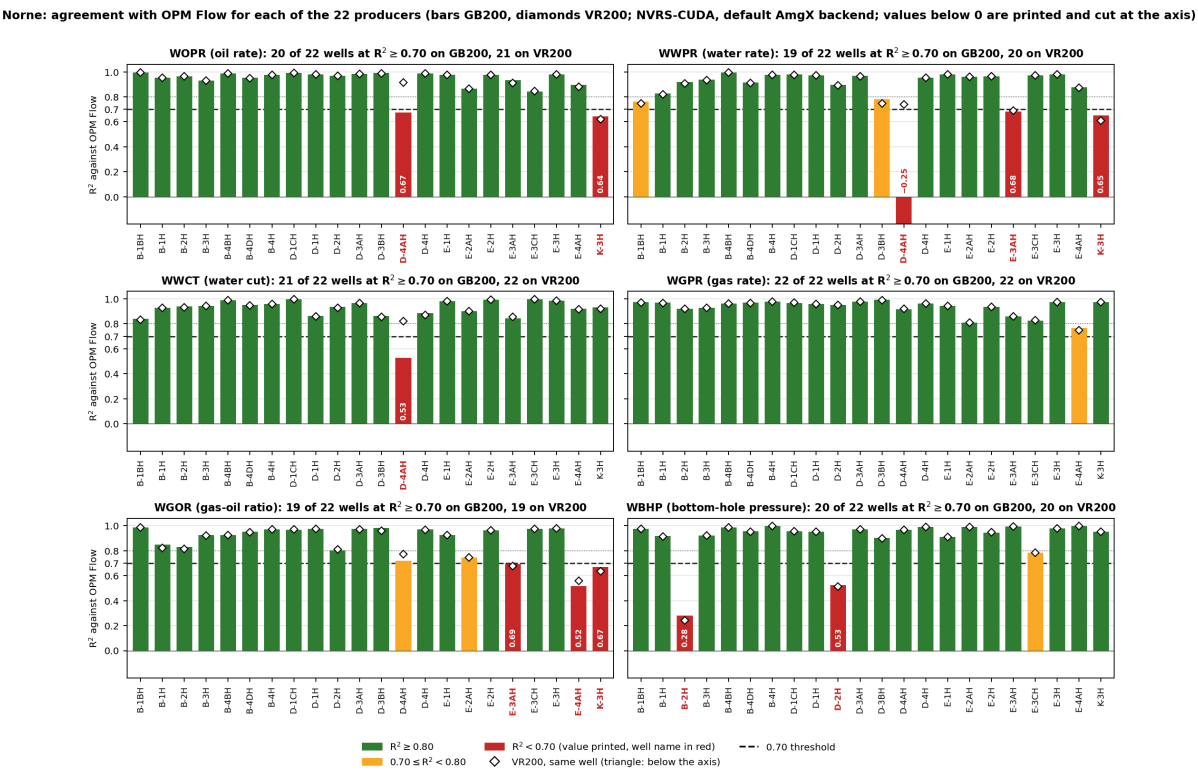


Figure 10.4: The scores of Table 10.1 as bars, GB200 bars and VR72 diamonds. A well below 0.70 has its name in red and its value printed.

Nineteen of the 22 producers reach $R^2 \geq 0.70$ on all four rate quantities at once, and 16 on all six; the VR72 run gives 20 of 22 on all four rates and 17 on all six. The mean over producers and quantities is 0.901 on GB200 and 0.910 on VR72. A field average can hide one high-rate producer, which is why the table names every well: each value can be checked against its panel in the HTML report.

10.16 Three-phase saturation mathematics used in Norne

The Norne comparison is sensitive to saturation-function details because a well rate is the product of connection factor, mobility and drawdown. Matching well controls while evaluating a different k_r surface can hold one primary rate and still produce a different BHP, water cut and gas-oil ratio.

10.16.1 ECLIPSE-default oil relative permeability

Let $k_{row}(S_w)$ be the oil curve from SWOF and $k_{rog}(S_g)$ the oil curve from SGOF. With connate water S_{wco}, the ECLIPSE-default three-phase oil curve is

$$k_{ro}(S_w, S_g) = \frac{S_g k_{rog}(S_g) + (\max(S_w, S_{wco}) - S_{wco}) k_{row}(S_w)}{S_g + \max(S_w, S_{wco}) - S_{wco}}.$$

At the point where the denominator tends to zero, the implementation uses the continuous average limit of the two oil curves. The result is bounded by the physical endpoint and its analytic derivatives are passed to the Jacobian.

Writing

$$w = \max(S_w, S_{wco}) - S_{wco}, \qquad A = S_g k_{rog} + w k_{row}, \qquad D = S_g + w,$$

the interior derivatives are

$$\frac{\partial k_{ro}}{\partial S_w} = \frac{(k_{row} + w k'_{row})D - A}{D^2},$$

$$\frac{\partial k_{ro}}{\partial S_g} = \frac{(k_{rog} + S_g k'_{rog})D - A}{D^2}.$$

The derivative of w is zero below connate water and one above it. The same piecewise condition is used by the residual and Jacobian, so Newton does not receive a slope for a different function.

The earlier Baker-product path multiplied k_{row} and k_{rog} and scaled by oil saturation. It agrees on two-phase edges but differs in the mobile-gas interior. Results produced with that path are a different model and are not a baseline for this edition.

10.16.2 Endpoint scaling

Field tables define a regional reference curve, while ENDSCALE arrays define cell endpoints. In two-point scaling, a cell saturation interval $[S_{\min,c}, S_{\max,c}]$ is mapped to the table interval $[S_{\min,t}, S_{\max,t}]$:

$$S_t = S_{\min,t} + \frac{S_c - S_{\min,c}}{S_{\max,c} - S_{\min,c}}(S_{\max,t} - S_{\min,t}).$$

The Jacobian multiplier is

$$\frac{dS_t}{dS_c} = \frac{S_{\max,t} - S_{\min,t}}{S_{\max,c} - S_{\min,c}}.$$

Three-point scaling, selected by Norne's `SCALECRS YES`, adds the critical/residual interior endpoint and applies the same affine construction on each side. The map and derivative remain continuous at the pinned point when the endpoint data are consistent.

Rate endpoint multipliers are applied after the saturation-axis mapping. The water, oil and gas endpoint arrays must be selected through the active/global cell map in the same way as `PVTNUM`. A plausible curve evaluated for the wrong cell is harder to detect than an out-of-range value, which is why `test_endscale` checks both mapping and scaling.

10.16.3 Carlson gas hysteresis

For every local cell, $S_g^{\max}$ stores the maximum accepted gas saturation. While gas saturation increases, the drainage curve is used and the maximum is updated only after a nonlinear state is accepted. On reversal, a scanning curve is constructed so that

$$k_{rg}^{\text{scan}}(S_g^{\max}) = k_{rg}^{\text{drain}}(S_g^{\max}),$$

and its critical endpoint is shifted toward the cell's imbibition critical gas saturation, `ISGCR`. This gives continuity at reversal while trapping gas as saturation falls.

Updating $S_g^{\max}$ on a rejected Newton trial would make the constitutive law depend on a state that the timestep never accepted. The history variable is therefore part of the accepted reservoir state. This is the same reason a schedule snapshot is not advanced on a failed substep.

Carlson model 0 moves the non-wetting gas curve. It does not create a general oil/water hysteresis framework. A case requiring Killough curves or oil scanning must be declared outside the release scope.

10.17 Control modes and their connection coefficients

The scalar target solve of Section 10.7 changes only the coefficient A_k and the target definition. With

$$C_{o,k} = WI_k(b_o\lambda_o + R_v b_g \lambda_g),$$

$$C_{w,k} = WI_k b_w \lambda_w, \qquad C_{g,k} = WI_k(b_g\lambda_g + R_s b_o \lambda_o),$$

the control table is:

Mode	A_k	Target basis	Limit interaction
ORAT	$C_{o,k}$	surface oil	BHP floor may become binding.
WRAT	$C_{w,k}$	surface water	BHP floor may become binding.
GRAT	$C_{g,k}$	surface gas	BHP floor may become binding.
LRAT	$C_{o,k} + C_{w,k}$	surface liquid	BHP floor may bind.
RESV	reservoir mobility coefficient	reservoir voidage	History RESV is derived by Section 10.8; prediction RESV uses its stated target.
BHP	,	pressure	Rate ceilings can take control when explicitly stated.
THP	,	tubing-head pressure	Parsed as a mode, but unsupported without VFP; must not be silently treated as BHP.
GRUP	inherited	group allocation	Producer or injector group target is distributed before the well solve.

For a producer the BHP limit is a floor. For an injector it is a ceiling. Choosing the opposite inequality makes a well switch controls in the direction that worsens the violation. The schedule unit test exercises both directions.

10.17.1 Worked ORAT example

Consider two completions with frozen oil coefficients $A_1 = 2.0$ and $A_2 = 1.0$ STB/day/psi, cell pressures 3000 and 2800 psi, and hydrostatic corrections 50 and -10 psi. For a 600 STB/day oil target,

$$p_w = \frac{2(3000 - 50) + 1(2800 + 10) - 600}{2 + 1} = 2703.33 \text{ psi}.$$

The connection rates are

$$q_1 = 2(3000 - 2703.33 - 50) = 493.34,$$

$$q_2 = 1(2800 - 2703.33 + 10) = 106.67,$$

which sum to the target to roundoff. Omitting the completion pressure terms would give a different BHP and a different allocation even though the total could still be forced to 600.

If the BHP floor were 2750 psi, the computed rate-control BHP would violate it. The well would switch to 2750 psi and deliver the rate supported by that pressure. A comparison that shows the OPM rate target and a lower NVRS rate is not necessarily wrong; first determine whether both simulators are on the same binding mode.

10.17.2 Worked zero-target example

Take two oil coefficients equal to one, completion effective pressures 3100 and 2900 psi, and solve only a zero oil target. The scalar equation gives $p_w = 3000$ psi, hence $q_{o,1} = +100$ and $q_{o,2} = -100$. Net oil is zero, but the well is flowing internally. If the water coefficients are 0.5 and 2.0, the net water is

$$q_w = 0.5(100) + 2(-100) = -150,$$

not zero. This is precisely the mechanism that created a phase-rate spike while satisfying the wrong mathematical statement. Whole-well shutdown makes all four connection contributions zero.

10.17.3 Worked history RESV example

Suppose the observed surface rates are $q_o^s = 1000$ STB/day, $q_w^s = 200$ STB/day and $q_g^s = 600{,}000$ scf/day. Let the regional caps be $\langle R_s \rangle = 500$ scf/STB and $\langle R_v \rangle = 10^{-4}$ STB/scf. Then

$$R_s^* = \min(500, 600) = 500, \qquad R_v^* = \min(10^{-4}, 1/600) = 10^{-4}.$$

For $B_o = 1.2$, $B_w = 1.03$, $B_g = 8.0 \times 10^{-4}$ rb/scf,

$$D = 1 - 500(10^{-4}) = 0.95.$$

The coefficients are

$$c_o = \frac{1.2 - 500(8 \times 10^{-4})}{0.95} = 0.842105,$$

$$c_w = 1.03, \qquad c_g = \frac{8 \times 10^{-4} - 10^{-4}(1.2)}{0.95} = 7.1579 \times 10^{-4}.$$

Thus

$$q_{\mathrm{RESV}} = 0.842105(1000) + 1.03(200) + 7.1579 \times 10^{-4}(600000) \approx 1477.6 \text{ rb/day}.$$

The example shows why gas units and B_g matter. Treating 600,000 scf/day as 600 Mscf/day would remove about 429 reservoir barrels/day from the target.

10.18 A staged decomposition of simulator disagreement

Changing a well formula because a field curve differs is unsafe. Use the following decomposition, stopping at the first level that does not agree.

10.18.1 Level 0: identity and static model

Confirm grid dimensions, active-cell count, pore volume, fault/NNC count, region counts, well/completion count and initial state. The new comparison has the same initial water inventory and small initial RMS differences, so a gross grid or unit mismatch is unlikely. A future change that breaks those values must be fixed before running nine years.

10.18.2 Level 1: prescribed schedule volumes

For each report interval, integrate injector targets after group scaling:

$$\Delta V_{\mathrm{inj},n} = \sum_{i \in \mathrm{open}} q_{i,n}^{\mathrm{applied}} (t_{n+1} - t_n).$$

Compare cumulative water and gas injection with OPM. This test does not depend on relative permeability. A discrepancy here belongs to units, schedule state, group control or injector allocation.

10.18.3 Level 2: field conservation

For water, form

$$\mathcal{E}_w(t) = W(t) - W(0) + W_{\mathrm{prod}}(t) - W_{\mathrm{inj}}(t).$$

In an incompressible diagnostic this should be near zero; in the full model, the comparison must use the same surface/reservoir basis and include the defined compressible terms. The HTML's implied-injection column is a screening quantity, not a replacement for the component residual.

10.18.4 Level 3: field state

Compare pressure and saturation RMS on matching active cells and common report times. A state difference preceding a well-rate difference indicates transport or accumulation. A well-rate difference preceding a field-state difference indicates controls or reporting.

10.18.5 Level 4: well mode and total rate

For one well and date, print control mode, requested target, applied group target, calculated BHP, binding BHP limit and total primary rates. This level usually explains a profile discontinuity without examining every completion.

10.18.6 Level 5: completion allocation

Only after the total mode is aligned, compare per-completion WI, PVT region, relative mobilities, hydrostatic offset, drawdown and phase contributions. Summing the printed values must reconstruct the well total. This catches an active/global index mismatch and a wrong reference depth.

10.18.7 Level 6: residual and Jacobian

Use a small extracted case or the IR Newton sandbox. Compare residual blocks, analytic derivatives and a finite-difference directional derivative. Editing the CUDA assembly is justified only here. The full Norne plot is too far from the local equation to prove which derivative is wrong.

This hierarchy saves time: most issues resolved in the parity work were found at Levels 1, 4 and 5, not in the linear solver or reservoir Jacobian.

10.19 A robust well-validation scorecard

The HTML keeps raw R^2 because it is standard and transparent. For release decisions, supplement it with the following quantities over OPM report times.

10.19.1 Normalised RMSE

$$\mathrm{NRMSE} = \frac{\sqrt{N^{-1}\sum_j (\hat{y}_j - y_j)^2}}{y_{95} - y_5 + \epsilon},$$

where y_5 and y_{95} are robust reference percentiles. This remains finite for a mostly shut well and expresses the error relative to its active range.

10.19.2 Integral and cumulative error

For a rate,

$$E_{\mathrm{int}} = \frac{\left|\int \hat{q}(t)dt - \int q(t)dt\right|}{\int |q(t)|dt + \epsilon}.$$

This penalises a persistent bias and is insensitive to a small pointwise phase shift when the produced volume is correct. Both pointwise and integral scores are needed: matching cumulative oil alone can hide wrong timing.

10.19.3 Active-time mask

Define

$$\mathcal{A} = \{j : |y_j| > \tau_y \text{ or } |\hat{y}_j| > \tau_y\}.$$

Report metrics on all times and on $\mathcal{A}$. The threshold τ_y must be stated in physical units, not selected separately for each simulator. For WGOR also require $q_o^{\mathrm{OPM}} > \tau_o$ so the denominator is meaningful.

10.19.4 Event timing

Record the errors in first production, first water breakthrough, first gas production, control changes and shutdown:

$$E_{\mathrm{event}} = \hat{t}_{\mathrm{event}} - t_{\mathrm{event}}.$$

A water-cut shape can have a poor pointwise score because breakthrough is 30 days late; the event metric says exactly what must improve.

10.19.5 Field-weighted aggregation

For per-well scores s_i, use a transparent production weight

$$\bar{s} = \frac{\sum_i V_i s_i}{\sum_i V_i}, \qquad V_i = \int |q_i^{\mathrm{OPM}}(t)|dt.$$

Publish the unweighted distribution beside it so a high-rate well cannot hide many failures. Never replace negative scores with zero before averaging.

The proposed release dashboard is therefore: median and production-weighted NRMSE, active-time R^2, cumulative-volume error, event-timing errors, water inventory error, field-state RMS and wall clock. A single "70%" number is a summary label, not the evidence.

10.20 Reading nonlinear and runtime diagnostics

The progress line reports residual norm, relative norm, mass-balance/CNV measures and linear iterations. They answer different questions.

Let $R(U)$ be the scaled nonlinear residual. A Newton update solves

$$J(U^m)\delta U^m = -R(U^m), \qquad U^{m+1} = U^m + \lambda_m \delta U^m,$$

where λ_m may be reduced by the line search. The raw norm can remain large because field-unit gas equations have large magnitudes; convergence uses the scaled residual, mass balance and CNV criteria. Comparing a raw 10^7 gas residual with a desired 10^2 without its scaling is not a convergence proof.

A repeated message that line search failed after six halvings means none of $\lambda = 1, 1/2, \ldots, 1/64$ improved the merit sufficiently. The timestep controller may recover by chopping Δt. If the same report

interval requires many chops, inspect control changes and constitutive kinks at that date before tightening the linear tolerance.

The timing decomposition is

$$t_{\text{wall}} = t_{\text{assembly}} + t_{\text{linear}} + t_{\text{well/control}} + t_{\text{I/O}} + t_{\text{other}}.$$

For the current Norne AmgX run, 78% of wall clock is in the linear solve and 67% of that solve is AMG. Printing less cannot recover the majority of time; reducing nonlinear systems or multigrid cost can. Conversely, an accidental full-field host PVT scan in every Newton step may not appear in the linear summary and must be found by phase timing or profiling.

When a fidelity change slows the run, compare:

- accepted internal timesteps per report step;
- Newton iterations per accepted and rejected substep;
- line-search halvings and timestep chops;
- Krylov iterations per Newton;
- AMG setup/reuse and V-cycle time;
- well-control and reporting time;
- bytes/log lines emitted inside the nonlinear loop.

This separation prevents reverting correct physics because the expensive effect was actually a debug print, repeated table construction or altered timestep policy.

10.21 Release promotion and branch discipline

The release workflow keeps the validated baseline and the next experiment separate:

1. complete source, tests, READMEs and this manual on `ir-develop`;
2. regenerate `manual.pdf` from `manual.tex` and verify that Git LFS records the binary;
3. commit and push `ir-develop`;
4. fast-forward or merge that validated commit into `main`, run the documentation/build checks again, and push `main`;
5. switch back to `ir-develop` before any further accuracy change;
6. never force-push either branch as part of this workflow.

The branch pointer does not preserve uncommitted cluster work. Before pulling, users with local changes should commit to a local branch or stash with a named entry. After a successful fast-forward, leave the stash in place until the new binary has run; it is a recoverable copy of the old experiment.

Because `manual.pdf` is an LFS object, a normal commit stores a pointer in Git and uploads the PDF content to LFS during push. Verify with

```
git check-attr filter -- \
  full_reservoir_simulator/GPU_reservoir_simulator/cuda/docs/manual.pdf
git lfs ls-files | grep manual.pdf
```

The first command must report `filter: lfs`; the second must list the manual. Do not add a second PDF outside the tracked path merely to bypass LFS.

10.22 Provenance of the parity implementation

The stable branch contains a sequence of small commits rather than one opaque rewrite. Their subjects identify the evidence they introduced: nonlinear cycling and three-phase k_{ro}, history controls, dissolved/vaporised gas, metric and primary-state scaling, endpoint handling, completion pressure, injector group allocation, schedule interval timing and history RESV/zero-rate semantics. The final parity correction is commit `39df30c`, “Match OPM history well rate controls.”

The source of the RESV algebra is OPM's `RateConverter`; the source of the regional averaging is OPM's `SurfaceToReservoirVoidage::defineState`; and the source of zero-target semantics is OPM's stopped-or-zero-rate primary-variable path. The equations are reproduced in Section 10.8 so the implementation can be reviewed without accepting a library call as authority.

No OPM code is copied into the CUDA kernels. The implementation reproduces the published algebra and observable control semantics in the host well-control layer. This distinction matters for licensing, maintainability and the user's requirement that the mixed-precision linear solver remain the subject of the work rather than being changed to fit one field profile.

10.23 Per-well triage matrix for the current report

The following matrix is a reading guide for the 24 August report. "Strong" means the primary rate shape is useful as a guard; "investigate" identifies a sustained signal whose discrepancy can teach us something; "sparse" warns that the well is shut or nearly constant for much of the scoring interval and that raw R^2 is ill-conditioned. It is not a replacement for the HTML.

Well	WOPR	WGPR	Water	Primary use in the next diagnosis
B-1BH	moderate, late	strong, late	investigate	Late-life opening and shutdown timing; do not score the long zero interval alone.
B-1H	useful	strong	investigate	Repeated control changes; compare requested/applied target and BHP at each transition.
B-2H	useful	strong	useful	Gas guard and late water-front timing.
B-3H	strong	strong	investigate	High-value sustained producer for simultaneous WOPR, WGPR, WWCT and WBHP diagnosis.
B-4BH	sparse	mixed	sparse	Event timing only; raw ratio scores are dominated by a short active interval.
B-4DH	useful	strong	useful	Late-life completion pressure and water allocation; one of the best BHP guards.
B-4H	sparse	sparse	sparse	Zero-target/shutdown regression guard; exclude from aggregate ratio claims unless active-time masked.
D-1CH	moderate	useful	investigate	Late reactivation and water breakthrough; BHP remains a useful control check.
D-1H	strong	strong	useful	Early rate-control reference and group schedule alignment.
D-2H	useful	strong	useful	Strong phase rates but weaker BHP; inspect mobility-weighted completion pressure rather than gas units.
D-3AH	strong	strong	investigate	Sustained high-value well; late water decline makes it a field-front diagnostic.
D-3BH	sparse late	mixed late	sparse	Shutdown/reactivation timing; use cumulative volume and event metrics.
D-4AH	investigate	investigate	sparse	Late opening; compare completion list and control state at the first active report.
D-4H	sparse	sparse	sparse	Zero-target and control-interval regression guard, not a global R^2 contributor.

Well	WOPR	WGPR	Water	Primary use in the next diagnosis
E-1H	investigate	useful	investigate	Long active interval with a clear late divergence; valuable for pressure/transport separation.
E-2AH	useful late	useful late	sparse	Late event timing and denominator masking for WGOR/WWCT.
E-2H	useful	strong	strong	One of the best water-cut and gas guards; retain before and after every change.
E-3AH	investigate	strong	investigate	Multiple transitions; plot schedule mode behind the physical curve.
E-3CH	useful late	useful late	sparse	Late-life rate and event guard.
E-3H	sparse early	strong early	sparse	Gas-rate guard; oil-denominator masking is mandatory for WGOR.
E-4AH	investigate	useful	investigate	Repeated active intervals; compare event timing and cumulative phases separately.
K-3H	sparse late	useful	sparse	Final shutdown timing and BHP guard; not representative of field-average water.

Two rules follow from this matrix. First, tune on informative wells and retain sparse wells as event/zero-target regressions. Second, require a change to improve more than one related observable: for example, water allocation should improve WWPR and inventory without destroying WOPR or BHP.

10.24 Planned experiments beyond the stable release

The stable result is promoted before these experiments. Each experiment is performed on `ir-develop`, one hypothesis at a time, and accepted only after the complete matrix in Section 10.27.

10.24.1 Experiment A: active-period metrics only

Add active masks, NRMSE, cumulative-volume error and event times to `compare.py`. This changes no simulator output. It should identify whether the apparent 70% level is limited by a small number of high-volume wells or by ill-conditioned scores from shut wells. Acceptance requires the old raw R^2 panels to remain present.

10.24.2 Experiment B: injection ledger

Write one row per group/phase/report interval containing individual requested rates, group ceiling, scale factor, applied total and integrated volume. Compare the cumulative ledger with OPM's field injection summaries. If the 82 million barrel inventory gap is already present in the prescribed ledger, the defect is schedule/group semantics. If not, it belongs to conservation, surface to reservoir conversion or producer allocation.

10.24.3 Experiment C: selected completion ledger

For B-3H, D-3AH, E-1H and E-2H, emit a debug file only at report boundaries: completion cell, WI, PVTNUM, pressure, saturations, mobilities, density, hydrostatic offset, drawdown and phase component rates. The file is disabled by default and contains no Newton-iteration print. The completion sums must recover the normal well summary exactly.

10.24.4 Experiment D: reference-depth audit

Compare the well reference depth and completion ordering with OPM. Repeat the scalar BHP calculation from the ledger offline. A correction is accepted only if it improves WBHP for rate-controlled wells and the delivered total rate remains at target. No reservoir kernel change is needed for this experiment.

10.24.5 Experiment E: group replacement modes

Quantify how often Norne activates `GCONINJE REIN` or `VREP` and the volume those intervals represent. Implement only if material. The required state is the group's produced surface/reservoir volumes from

the previous accepted report interval; a trial Newton state must not update it. Test with a small synthetic schedule before Norne.

10.24.6 Experiment F: PINCH and transmissibility

Use `deckinfo` to count cells/connections affected by `PINCH`. Compare static connection graphs before implementing a rule. This is a grid preprocessing change and must be validated by connection count and transmissibility, not tuned against one well curve.

10.24.7 Experiment G: timestep parity as a diagnostic

Record OPM and NVRS-CUDA accepted internal timestep sequences around the first material divergence. The goal is not to force identical timesteps; it is to determine whether different nonlinear paths precede the state difference. Run with reduced maximum step on both simulators as a convergence study. If the solutions approach one another under refinement, the gap is temporal error; if not, it is model semantics.

10.24.8 Experiment H: component mass-balance audit

At each report step, write component inventory, cumulative boundary source and cumulative well source in one unit basis. Check

$$M_c(t) - M_c(0) - \int_0^t Q_c(\tau)d\tau = \mathcal{E}_c(t)$$

for water, stock-tank oil and surface gas components. This separates a report conversion problem from a residual conservation problem. The audit should use accepted states only and MPI-reduce owned cells once per report step.

10.25 Published-result record template

Use the following template in a merge request, run note or benchmark README. It is deliberately concise enough to fill for every experiment and complete enough to reproduce one.

```
Norne result ID:
Date/time:
Git branch and commit:
Worktree status:
GPU / CUDA architecture:
HYPRE prefix and revision:
AmgX prefix and revision:
Deck path and checksum:
Norne switches (FIELDSATFUNC/PROPSARRAYS/USEENDSCALE):
Backend(s), ranks:
Build command:
Run command:
OPM command/version:
Wall / linear / AMG seconds:
Report steps completed:
Newton / Krylov / chops / line-search failures:
Initial and final water inventory:
Primary active-well metrics:
Known unsupported keywords active in this interval:
HTML and log paths:
Conclusion and rollback commit:
```

The result record should distinguish facts, interpretation and decision. “AmgX completed in 251.96 s” is a fact. “The remaining difference is shared physics because HYPRE overlays it” is an interpretation supported by two runs. “Promote this commit to main” is a decision. Keeping these separate makes later re-evaluation possible when a newly implemented deck feature changes the reference.

For a negative experiment, record the same template. A failed hypothesis is valuable when it states what was held fixed, what changed and which evidence rejected it. Do not delete the stable branch or overwrite its logs to make room for the next attempt.

10.26 One-page Norne diagnosis decision tree

Use this page before editing source. Start at the first observation that is false and follow only that row.

Observation	First place to diagnose
Static identity agrees	Otherwise check deck/LFS, grid mapping, units, PORV, regions and initialisation; do not run nine years.
Prescribed injector volumes agree	Otherwise check schedule interval, metric conversion, group hierarchy and `GCONINJE` scaling.
AmgX and HYPRE physical curves agree	Otherwise check convergence/tolerance; if they overlay, hold the linear solver fixed.
Opening and shutdown dates agree	Otherwise check `DATES`, WCONHIST/WHISTCTL, WELOPEN and snapshot order.
Mode and total target agree	Otherwise check ORAT/WRAT/GRAT/LRAT/RESV/BHP, group allocation and zero-target shutdown.
Total matches but phases differ	Check PVTNUM, R_s/R_v, mobility, gas-unit basis and ratio denominator.
Mode matches but BHP differs	Check reference depth, completion hydrostatic offsets, WI, mobility and the binding pressure limit.
Wells match but state diverges	Check conservation, saturation functions, hysteresis, PVT, accumulation and timestep refinement.
Physics matches but runtime regresses	Compare nonlinear systems, chops, Krylov/Newton, AMG reuse, host-control timing and log volume.
Residual-level mismatch is proven	Only then inspect CUDA residual/Jacobian with directional derivatives; preserve mixed precision and the stable branch.

The decision tree is intentionally ordered from cheapest and most global to most invasive. A full-field plot is evidence of a discrepancy, not evidence of which CUDA equation caused it.

10.27 Performance safeguards for parity work

Accuracy fixes can be computationally correct and operationally unusable. The following safeguards were adopted after intermediate runs exceeded eleven minutes:

No diagnostic output in the Newton hot loop Retry and line-search diagnostics are rate-limited. Per-cell or per-completion printing is not permitted in a production benchmark.

No repeated immutable PVT flattening PVTO/PVTG branches are flattened once per PVT object and selected by `PVTNUM` through lightweight views.

No full-field RESV average per Newton The HCPV regional state is computed once when the report step changes.

No kernel change for zero wells The host control record clears the existing well flag; residual and Jacobian kernels retain their known fast path.

No solver retuning during physics validation AmgX/HYPRE configurations, mixed-precision boundaries and Krylov tolerances remain fixed. A physical improvement must not be obtained by weakening the solve.

Measure both backends Coincident physical results and different wall clocks distinguish a shared-model change from backend-specific numerical behaviour.

After any change, compare total wall time, linear-solve time, AMG time, Newton iterations, chops and line-search failures. A curve improvement accompanied by an order-of-magnitude timestep increase is not accepted without explaining the trade.

10.28 Regression and review matrix

The Norne work touches several layers. The minimum review matrix is:

Layer	Automated evidence	Field evidence
Schedule parser	`test_schedule`: dates, WCONHIST, WHISTCTL, completions and controls	Opening/shutdown transitions align with OPM.
Metric units	`test_units`	Gas targets and WGPR have the correct order of magnitude; no 10^3 correction is applied in plotting.
PVT	`test_pvt`	PVTNUM regions, R_s, R_v and densities print plausible values and AmgX/HYPRE overlay.
Endpoint/hysteresis	`test_endscale`	Water/gas front timing and saturation bounds remain physical.
Well controls	Schedule tests plus CUDA translation-unit build	WOPR, WWPR, WGPR, WBHP and zero-target intervals.
Nonlinear path	Existing Newton tests and complete run	247 steps finish; chops, Newton count and runtime do not regress unexpectedly.
Linear solver	Existing CPR tests; unchanged source diff	Both backends converge to the same physical result.
Output	ECL reader round trip	HTML field table, curves and ratios use one unit convention.

The parity commit was checked by compiling `well_report.cu` with NVCC and running the Norne schedule suite: 83 checks, zero failures. The complete field run reported in this chapter supplies the GPU and MPI evidence that a host-only unit test cannot.

10.29 Code map for the parity implementation

The implementation is intentionally localised:

`include/fluid.hpp` ECLIPSE-default three-phase oil relative permeability, endpoint scaling and Carlson gas hysteresis.

`include/pvt_tables.hpp` PVTO/PVTG evaluation, R_s, R_v, shrinkage factors and region-capable flattened table views.

`src/assemble_blackoil.cu` Device residual/Jacobian use of regional PVT, live oil/wet gas and phase gravity. This file was not changed by the final history-control correction.

`src/schedule.cpp` Dated controls, WCONHIST observations, WHISTCTL overrides, group producer/injector records and completion state.

`src/field_model.cu` Schedule snapshot application, completion allocation and group injection ceilings.

`src/well_report.cu` PVTNUM-aware surface rates, completion pressure correction, scalar producer target solve, OPM history RESV conversion, zero-target whole-well shutdown and well summary quantities.

`benchmark/compare.py` Common ECL reader, interpolation to OPM report times, well panels, R^2, state RMS and water inventory.

This map is also a change-control boundary. A proposed fix for a plotted ratio should begin in reporting and units. A proposed fix for a control transition should begin in the schedule. A change to `assemble_blackoil.cu` needs residual-level evidence and analytic derivative tests; it is not the first place to tune a visual well curve.

10.30 Limitations after the parity release

The following are explicitly not claimed by this edition:

- exact OPM equivalence for every ECLIPSE keyword or network/well model;
- THP/VFP control, multi-segment wells or surface network balance;
- `GCONINJE` reinjection and voidage-replacement feedback;
- enforcement of `DRVDT` limits;
- `PINCH` transmissibility processing;
- Stone I/II oil relative permeability;
- all ECLIPSE hysteresis models beyond Carlson `HYSTMOD` 0;
- identical adaptive timestep sequences or bitwise state equality;
- a universal percentage similarity represented by unweighted R^2.

These limitations are compatible with a useful simulator when they are stated, measured and excluded from claims. They are not compatible with silently altering the reference deck or clipping an inconvenient score.

10.31 Checklist before publishing a new Norne result

This checklist is the reproducible definition of the result in this manual. If one item is missing, the report may still be useful for development, but it is not the published parity result.

1. Record `git rev-parse --short HEAD` and a clean/status listing.
2. Run `git lfs pull`; confirm the Norne deck and manual are not LFS pointer text.
3. Source HYPRE and AmgX environment files; record both prefixes.
4. Run incremental `make -j`; do not mix a stale Make binary with a new CMake source tree.
5. Confirm the banner says metric field deck, 44,431 active cells, 36 wells, 247 report steps, and the intended linear backend.
6. Preserve the dated AmgX and HYPRE logs.
7. Verify both runs finish and compare their well curves before comparing either with OPM.
8. Inspect water inventory, state RMS and timings, not only WOPR.

9. Inspect active sustained wells before interpreting an extreme R^2 from a mostly shut well.
10. State the remaining unsupported deck features alongside conclusions.

Release sign-off record

Complete this record beside the generated HTML rather than relying on a directory named `latest`. Blank entries mean the run is developmental, not the parity release.

Evidence	**Recorded value or artefact**
Source and working tree	Commit, branch and clean/status output
Deck identity	Norne source revision, top DATA file and checksum
Units and architecture	METRIC; CUDA architecture and GPU model
Dependency identity	HYPRE and AmgX installation prefixes/configurations
Build identity	Build system, command, executable path and timestamp
Run evidence	Dated AmgX and HYPRE logs; 247/247 accepted report steps
Comparison evidence	Dated `compare_NORNE.html`; OPM version and log
Acceptance	Backend overlay, water ledger, active-well metrics and limitations

Chapter 11

Profiling

11.1 Two tools, two questions

Nsight Systems and Nsight Compute answer different questions and are not interchangeable.

Nsight Systems (`nsys`) answers **where the time goes**. It traces the whole run at low overhead, roughly 1.2× runtime, so the split the simulator claims for itself can be checked from outside it. It is cheap enough to ride along with any benchmark.

Nsight Compute (`ncu`) answers **why a kernel is slow**. It replays kernels with the hardware counters enabled and places each against the roofline. Replay costs roughly 100× per profiled kernel, which is why it is pointed at a bounded number of launches and never at a whole run, and why it is always requested explicitly.

11.2 Running a profile

```
NVRS_PROFILE=1    ./run_norne.sh   # nsys: the timeline
NVRS_PROFILE=both ./run_norne.sh   # add the kernel counters
./profile_norne.sh ncu             # counters only, no benchmark
```

The profiled run is deliberately shortened. A full history under `ncu` takes days, and under `nsys` produces a trace no viewer will open. The kernel mix does not change over a history (the same assembly, V-cycle and sparse products run at every step) so a couple of dozen steps carries the same mix. What it does not carry is the wall-clock total, and nothing downstream quotes a profiled runtime as one.

Both tools are located where they are actually installed, which is frequently not on `PATH`: under `/opt/nvidia/nsight-systems/*/target-linux-*/`, in the CUDA toolkit's `bin`, and in the HPC SDK. The architecture directory differs by host (`target-linux-sbsa-armv8` on Grace and Vera, `target-linux-x64` elsewhere) so it is searched rather than assumed. `NSYS` and `NCU` name a particular copy.

11.3 What reaches the report

Four figures are folded into the comparison HTML. The first needs no profiler at all: the solver times its own phases and writes them out. That is the only way to separate a multigrid V-cycle from an operator apply, because both are the same sparse kernel and an external trace cannot tell them apart. The other three come from `nsys` (measured share of GPU time by phase and by kernel) and from `ncu` (arithmetic intensity against achieved throughput, and DRAM against SM throughput so the limiter can be read off directly).

When a Nsight figure is absent the report says so and why. It does not imply that Nsight measured the self-timed figure.

11.4 When profiling does not work

Both tools depend on the driver and on what a container exposes, so either can fail on a machine where the simulator itself runs correctly. Both fail quietly, and each failure resembles something it is not.

11.4.1 nsys runs and collects no CUDA

The report is written, `nsys` exits successfully, and the only sign is in its own diagnostics:

```
CUDA profiling might have not been started correctly.
No CUDA events collected. Does the process use CUDA?
```

There are two causes and they are distinguished by the session duration. The profiled process may have exited before reaching the GPU, a session lasting two seconds did not simulate anything, and the usual reason is a launch that failed, historically a launch without `mpirun`. Or `nsys` is older than the GPU: a version that does not know the architecture cannot trace CUDA on it and says nothing clearer than the above. Compare `nsys --version` against the age of the hardware.

11.4.2 ncu fails to initialise

```
Failed to initialize the profiler: LibraryNotLoaded.
Check that a compatible driver library is loaded.
```

This is *not* a permissions problem. That one reports `ERR_NVGPUCTRPERM` and is lifted by an administrator setting `NVreg_RestrictProfilingToAdminUsers=0`. `LibraryNotLoaded` means the driver's profiling components are not reachable inside the container, which is a matter of what the container runtime was told to expose: `NVIDIA_DRIVER_CAPABILITIES` must include `compute,utility`, and profiling generally wants `CAP_SYS_ADMIN`.

To separate "`ncu` is broken here" from "`ncu` is broken on this application", profile something trivial:

```
printf '__global__ void k(float*a){a[threadIdx.x]=1;}\n'\
       'int main(){float*d;cudaMalloc(&d,1024);'\
       'k<<<1,64>>>(d);cudaDeviceSynchronize();}\n' > t.cu
nvcc t.cu -o t && ncu --metrics gpu__time_duration.sum ./t
```

If that fails in the same way, nothing in the simulator is involved.

11.4.3 Reading the diagnostics

`ncu` writes its CSV to standard output, so its error messages land in `ncu_kernels.csv` rather than `ncu.log`, which often contains nothing but MPI observing the exit code afterwards. A profile that comes back with a handful of rows has failed rather than succeeded quietly: the requested launch count contributes several rows each. Both streams are printed when that happens, because reading only the log points at the wrong cause.

Chapter 12

Tuning guide

12.1 Method

Solver settings interact, and intuition about which of them dominates is frequently wrong. The procedure that produced the results in Chapter 9 was, in every case, to change one setting at a time against a fixed baseline, on a shortened but representative interval, and to accept only changes that improved the measured wall clock. `scripts/spe10_tune.sh` implements this procedure and can be adapted to other models.

Two measurements should precede any change: the phase breakdown printed at the end of a run, and the Krylov iterations per Newton iteration.

12.2 Interpreting the phase breakdown

Multigrid V-cycles dominate The preconditioner is being applied many times. Either the iteration count is high, addressed in the next section, or each cycle is expensive, addressed by `AMGMAXCOARSE` and `AMGFAST`.

Multigrid setup dominates The hierarchy is being rebuilt too often. Raise `AMGREUSE`; the summary reports rebuilds against reuses. On several ranks, also consider `AMGFAST`.

Orthogonalisation dominates The Krylov basis is long or the iteration count is high. `RESTART` controls the former; note that reducing it increases the number of restarts.

Operator and halo dominate The linear iteration count is high relative to the work per iteration, or halo exchange is staging through host memory. Confirm the MPI is CUDA-aware.

Assembly and properties dominate Unusual in a well-configured run. Confirm `JACOBIAN ANALYTIC`.

12.3 Reducing Krylov iterations per Newton

This ratio is the primary indicator of preconditioner quality. On the synthetic benchmark cases it is between 10 and 20; on SPE10 model 2 with the recommended settings it is approximately 55, and the difference is entirely attributable to the heterogeneity of the field.

1. Confirm from the banner that the classical hierarchy is in use. On one rank it is selected automatically; on several it is unavailable, and the question becomes whether the model requires several ranks at all.
2. Confirm `LINTOL` is 10^{-2}. Tighter values increase the count without improving the Newton trajectory.
3. Consider `STAGE2 ZLINE` for models with strong vertical anisotropy, remembering the kernel cost documented in Section 6.11.

12.4 Reducing the number of timesteps

A run that chops frequently spends its time on discarded attempts.

1. Set `DTGROW` to 2 to prevent the controller oscillating between a step that is too long and the reduction that repairs it.
2. If the model reliably settles at a particular step length, set `DTMAX` to that length so the search is not repeated at every report boundary.
3. Do not use a report-boundary remainder as the controller's next-step baseline. The physical step is shortened only to land exactly on the report date; the controller must retain the untruncated proposal. Otherwise each report interval restarts a sawtooth of unnecessarily small steps.
4. Examine whether the failures are non-linear or linear. If the log reports that the linear solver did not reach `LINTOL`, the preconditioner is the limitation and shortening the timestep is treating a symptom.

12.5 A worked example

The following sequence is the one that produced the SPE10 results, and illustrates the method.

1. **Baseline.** The default configuration completed the case in 861 s. The phase breakdown showed the run to be almost entirely linear solver, at 120 Krylov iterations per Newton iteration.
2. **Hypothesis.** A ratio of 120 against the 15 observed on synthetic cases suggested the hierarchy, not the settings, was the limitation.
3. **Measurement.** Five configurations were run on a 400-day interval, each differing from the baseline in one respect. Smoothing sweeps and aggregate size changed the cost per cycle but not the convergence. The classical hierarchy halved both the iteration count and the run time.
4. **Second round.** With the classical hierarchy established as the baseline, a further set of candidates was measured. Interpolation threshold and additional V-cycles were rejected; relaxing the Newton iteration limit was retained.
5. **Third round.** Adaptive implicitness was measured against the established configuration and adopted.
6. **Confirmation.** The full 2000-day case was re-run with the adopted settings, giving 342 s.

At no point was a setting adopted on the grounds that it should help.

When a caller has explicitly cleared the Krylov solution vector, it passes the zero-initial-guess flag to the solver. FGMRES and BiCGSTAB then copy b into the initial residual instead of evaluating the mathematically redundant $b - A0$. Restart residuals and nonzero Woodbury correction guesses still use a true operator application. This removes one full sparse operator application per Newton solve without changing the nonlinear or linear iteration path.

Chapter 13

Troubleshooting

13.1 The run stops

13.1.1 Report step failed to converge after n timestep reductions

Newton did not converge even at the shortest timestep permitted by `MAXCHOP`. In order of likelihood:

1. **Well controls.** Confirm the voidage balance is sensible. An injection rate substantially exceeding what the producers can remove will eventually fill the model, and no timestep is short enough to make that converge.
2. **Saturation tables.** Confirm the column order (Section 5.5.3) and the monotonicity requirements. A table in ECLIPSE column order is accepted and produces a water relative permeability that decreases with water saturation.
3. **Linear solution.** If the log also reports that `LINTOL` was not reached, the preconditioner is failing and the timestep reduction is treating a symptom. See Chapter 12.
4. **Iteration limit.** Run with `NVRS_VERBOSE=1` and examine the residual sequence. If it is still decreasing at `NEWTONMAX`, raising the limit will help; if it has stalled, it will not.
5. **Chop budget.** Raise `MAXCHOP` to 8 for a strongly heterogeneous model.

13.1.2 Non-finite residual

Treated as a failed timestep and retried. Persistent occurrences indicate an unphysical model: zero pore volume in some cell, a negative well index, or a saturation table that does not span the full range.

13.1.3 PERMFILE has fewer values than grid cells

The file must contain one value per cell of the *global* grid, in natural order. The usual cause is a file written for a different grid size.

13.1.4 Compositional model requires a ZMF feed record

A compositional run was requested without `ZMF`. See Section 5.12.

13.2 The results are not as expected

13.2.1 Water does not move

Check the saturation table column order. This is the single most common error when adapting an ECLIPSE deck.

13.2.2 Disagreement with another simulator in three phases

Expected when gas is mobile, for the reason given in Section 9.3. Compare in two phases, where no such difference exists.

13.2.3 Results changed after upgrading

Adaptive implicitness and automatic hierarchy selection are both defaults. To reproduce earlier behaviour exactly, set `AIM 0` and `NVRS_AMGX_ALGO=AGGREGATION`, then reintroduce them individually to identify which is responsible for the difference.

13.2.4 Material balance error grows

Examine whether the timestep is being chopped repeatedly, and whether the Newton tolerance is being reached or the iteration limit is being hit. A run that regularly exits Newton on the iteration limit accumulates error at every step.

13.3 Performance problems

13.3.1 Many Krylov iterations per Newton iteration

See Section 7.3. Confirm the hierarchy from the banner first.

13.3.2 Several GPUs are slower than one

Expected below approximately 5×10^5 cells per GPU (Chapter 7). Confirm also that the MPI is CUDA-aware, since host staging affects only multi-rank runs and is therefore easy to overlook.

13.3.3 The multigrid setup dominates

Raise `AMGREUSE`. Confirm from the summary that reuse is actually occurring and that the growth guard is not ending it immediately, which would indicate `AMGREUSEGROWTH` is set too low for the model.

13.3.4 The timestep collapses

The controller is reporting that the non-linear problem is difficult at that point in the schedule. `DTGROW` 2 prevents oscillation; `DTMAX` avoids repeating a known search. If the collapse coincides with the arrival of a front at a producer, it is physical and should be accepted.

13.4 Library errors

13.4.1 CUDA failure: out of memory during coarse point selection

The distributed classical setup described in Section 6.10. This should not occur under the automatic selection; it indicates that `NVRS_AMGX_ALGO=CLASSICAL` has been set on more than one rank.

13.4.2 Incorrect AmgX configuration provided

The configuration file is in a format the installed AmgX does not accept. The files supplied under `benchmark/amgx_configs/` use the plain key-value form, which is accepted by all versions tested; the JSON form is rejected by some builds.

13.4.3 Invalid device ordinal from within the multigrid library

A device binding problem. Each rank must bind to its device before any library initialises a CUDA context. This is handled internally; if it appears, confirm that `CUDA_VISIBLE_DEVICES` is not being set inconsistently across ranks by the batch system.

13.5 Build problems

13.5.1 HYPRE_DIR is empty

The environment script has not been sourced in this shell.

13.5.2 No HYPRE.h under the given prefix

`HYPRE_FP32_DIR` points at a location without headers; the usual cause is a HYPRE that was built but not installed.

13.5.3 Deck sets MIXED but the binary disagrees

Precision is a build-time property. Rebuild with `-DNVRS_MIXED_PRECISION` matching the HYPRE library being linked.

13.6 Every ablation and study

This section measures each of the solver options this manual describes against the default, on Norne with one GPU unless a row says otherwise. The table is the quickest guide to what each keyword is worth. GB200 walls are medians of three runs with [min, max] unless marked as a single run; Newton and Krylov counts are GB200.

Study	Change from the default	GB200 wall, s	Newton / Krylov	VR72 wall, s	Finding
Default					
Default configuration	block CSR, captured Krylov tail, df64 operator, quasi-IMPES, block-Jacobi stage 2, AMG reused up to 10 Newton	46.75 [46.45, 46.99]	1,497 / 41,114	35.76 [35.64, 36.62]	reference point (repeated session); 11.6 s of it is output
Storage and launch (2 × 2)					
Scalar CSR, direct launches	version 1 operator	51.24 [51.22, 51.55]	2,004 / 45,125	40.08	operator 64.1 μs per Krylov
Block CSR, direct launches	no graph capture	45.25 [44.74, 45.47]	1,497 / 41,114	34.89	52.0 μs: storage alone 1.23× (model 1.31×); fastest
Scalar CSR, captured tail	no block storage	52.55 [52.44, 52.56]	2,004 / 45,125	40.86	capture costs 2.9 s; not repaid
Block CSR, captured, reuse guard	CUDAGRAPHMINREUSE 2	45.21 [44.91, 45.33]	1,497 / 41,114	34.91	guard stops after 192 captures (0.22 reuses each)
Operator arithmetic					
Native FP64 operator	EMULATE 0	44.04 [43.82, 44.21]	1,400 / 38,869	34.56	11.37 μs per apply; path differs, not arithmetic
FP32-stored operator, FP64 master	EMULATE 2	47.62 [47.52, 47.81]	1,654 / 42,608	34.36	apply 7% cheaper, run 8% slower; 9 of 1,663 solves continued on the true residual
Preconditioner					
Rebuild AMG every Newton	AMGREUSE 0 (single run)	54.90	1,532 / 31,479	59.73	reuse is worth 1.17× (1.67× on VR72)
Dynamic row-sum decoupling	CPRDECOUPLE DRS (single run)	45.21	1,440 / 40,149	37.11	Krylov per Newton 27.9 vs 27.5
Plain row-sum decoupling	CPRDECOUPLE SUM (single run)	47.41	1,596 / 42,864	37.41	26.9 per Newton: decoupling within 5%
Ginkgo FP16 vs FP32 multigrid	GINKGOPREC 16 (comparison session, single runs)	80.55 vs 70.42	1,826 / 58,439 vs 1,437 / 48,209	50.02 vs 49.31	Krylov per Newton 32.0 vs 33.5: no loss per Newton; longer nonlinear path
Preconditioner applications per Newton vs Flow (P)	FGMRES applies once, BiCGSTAB twice per iteration		27.5 vs 30.2		linear tolerance 1e-4 vs 1e-2: raw Krylov counts are not comparable
Vertical line stage 2	STAGE2 ZLINE	n/a	n/a	n/a	needs the Cartesian neighbour map; falls back to block-Jacobi on Norne
BoomerAMG, coarse grid 20,000 rows	AMGMAXCOARSE 20000, vs BoomerAMG 71.51 s (single run)	87.35	1,723 / 136,274	73.39	V-cycle 0.24 vs 0.44 ms but 1.87× Krylov
Time integration					
Flow's accepted substeps imposed	DTSEQFILE	47.68 [47.23, 47.82]	1,580 / 42,867	32.35	Newton ratio to Flow 1.26 (1.33 unconstrained)
Reference's material balance	MBTOL 1e-7, LINTOL 1e-2, no stall limit (single run)			2,901	Newton stagnates; main limitation of the nonlinear solver
Reference (Flow 2024.10)					
(P) GPU ILU(0), 1 rank × 2 threads	two-thread configuration	488.51	1,985 / 30,072	243.88	NVRS 10.45× faster (VR72 6.82×)
(G) GPU ILU(0), every host thread	1 × 144 (VR72 1 × 352)	348.95	1,985	543.49	7.46×; threads slow VR72 down
(C) CPU CPR, best of 8/16/32 ranks	16 ranks on both	165.62	1,263	88.48	3.54× (VR72 2.47×)
Scaling, 1 to 4 GPUs					
1 million cells	aggregation, NCCL, one imposed step sequence	24.92 → 27.22	683 → 687	16.84 → 25.67	0.92× (VR72 0.66×): too small to scale
5 million cells	as above	104.68 → 80.60	683 → 685	69.16 → 56.73	1.30×, 33% efficiency (VR72 1.22×)
Kernels in isolation					
Block FP64 vs cuSPARSE CSR / BSR	Norne-sized box, μs per apply	14.4 vs 34.8 / 39.0		11.1 vs 29.7 / 32.3	2.42× / 2.71× (VR72 2.69× / 2.92×)
df64 vs native FP64	same box	18.5 vs 14.4		13.3 vs 11.1	emulation costs 1.28× (VR72 1.21×)
Graph capture break-even r^*	capture cost / saving per replay	2.3		30	Norne reuses each graph 1.16 times
Accuracy					
Pressure-matrix conditioning	$\mathrm{cond}_2(A_p) = 5.2\mathrm{e}3$	u_{32}: 3e-4, u_{16}: 2.5		same	FP32 hierarchy safe; FP16 outside Proposition 1
Exact discrete water ledger	NVRS_SUBSTEP_BALANCE=1	-0.68% (trapezoid 1.75%)		-0.70% (1.15%)	bias due to the MB tolerance; the remainder is integration error in the balance

Appendix A

Example decks

A.1 Two-phase waterflood

A five-spot waterflood on a synthetic field, with every solver setting at its recommended value.

```
TITLE    EXAMPLE WATERFLOOD

NCOMP    2                -- 2 = oil/water.  Must precede MODEL.
MODEL    BLACKOIL
OUTDIR   OUTPUT_EXAMPLE
CASENAME EXAMPLE

--, - GRID, , , , , , , , , , , , , , , , , , , , , , , , , , , , , , , , , -
NX       100
NY       100
NZ       20
DX       50
DY       50
DZ       10
KZMULT   0.1
PERMMIN  10
PERMMAX  800
POROMIN  0.08
POROMAX  0.28

--, - PROPS, , , , , , , , , , , , , , , , , , , , , , , , , , , , , , , , ,
PVTCONST 1
UW       0.5
UO       2.5
BW       1.0
BO       1.0
RHOW     1.02
RHOO     0.85
CFW      0.0
CFO      0.0
SWI      0.2
SWR      0.2
PINIT    4000
SWINIT   0.2
SOINIT   0.8
GRAVITY  1

--, - SOLVER, , , , , , , , , , , , , , , , , , , , , , , , , , , , , , , , --
KRYLOV   FGMRES
```

```
LINTOL   1e-2           -- inexact-Newton forcing term
LINMAXIT 200
RESTART  30
AMGREUSE 10
AMGREUSEGROWTH 2.0
MIXED    1
EMULATE  1
JACOBIAN ANALYTIC       -- required by AIM, and faster regardless

--, - TIME, , , , , , , , , , , , , , , , , , , , -
DTREPORT   100
TEND       2000
DTGROW     2
NEWTONMAX  20
NEWTONTOL  5e-3
DPMAX      500
DSMAX      0.2
MAXCHOP    8

--, - OUTPUT, , , , , , , , , , , , , , , , , , , --
MAKECSV      1
MAKESUMMARY  1
ECLOUT       1

--, - WELLS:  I  J  TYPE  RATE  RW  SKIN  PWF  PERF  NAME, , , ,
WELLS
   50  50   1   8000   0.5   0.0  6000   all INJ
    5   5  -1     -1   0.5   0.0  3500   all P1
   95   5  -1     -1   0.5   0.0  3500   all P2
    5  95  -1     -1   0.5   0.0  3500   all P3
   95  95  -1     -1   0.5   0.0  3500   all P4
/
```

A.2 Three-phase with tabulated saturation functions

The additions required for a three-phase model. Grid, solver and schedule sections are as above.

```
NCOMP    3
MODEL    BLACKOIL

UG       0.02
BG       0.9
RS       0.0            -- immiscible; omit for dissolved gas
RHOG     0.85
SGINIT   0.02
SOINIT   0.78

--, - SWOF: (Sw, Krow, Krw) -- note the column order, , , , , --
SWOF
  0.00  1.0000  0.0000
  0.30  1.0000  0.0000
  0.40  0.3200  0.0200
  0.50  0.0800  0.0460
  0.60  0.0100  0.0900
  0.70  0.0001  0.1660
  0.80  0.0000  0.2400
  1.00  0.0000  0.2400
```

```
/

--, - SGOF: (Sg, Krog, Krg), , , , , , , , , , , , , --
SGOF
  0.00  1.0000  0.0000
  0.05  0.5900  0.0200
  0.15  0.1800  0.1600
  0.25  0.0300  0.3300
  0.35  0.0010  0.5500
  0.50  0.0000  1.0000
  1.00  0.0000  1.0000
/
```

A.3 Reading an external rock model

The keywords required to read per-cell properties from files, as used by the SPE10 case.

```
NX        60
NY        220
NZ        85
DX        20
DY        10
DZ        2
PERMFILE  dataset/SPE10/perm.txt      -- one value per global cell
PERMZFILE dataset/SPE10/permz.txt     -- vertical permeability
POROFILE  dataset/SPE10/poro.txt      -- independent porosity
POROMIN   1e-4                        -- floor under the file values
```

Appendix B

Keyword index

A ACF 67, AGGLEVELS 64, AGGPMAX 64, AGGTRUNC 64, AIM 56, AIMCFL 57, AMGCOARSERELAX 64, AMGFAST 63, AMGLIB 59, AMGMAXCOARSE 64, AMGNONGALERKIN 65, AMGPRINT 65, AMGREUSE 63, AMGREUSEGROWTH 63, AMGSEQTHRESH 65, APPLEYARD 54, AUTOPLOT 67.

B BG, BO, BW 35.

C CASENAME 32, CFO, CFW, CT 36.

D DARCY 68, DPMAX 53, DSMAX 53, DTGROW 51, DTMAX 47, DTREPORT 47, DX, DY, DZ 32.

E ECLOUT 66, EMULATE 61.

F FDEPS 56.

G GINKGOPREC 60, GRAVGRAD 68, GRAVITY 40.

I IMPLICIT 57.

J JACOBIAN 55.

K KRYLOV 58, KZMULT 34.

L LINMAXIT 59, LINSOLVER 59, LINTOL 58.

M MAKECSV 66, MAKEGASSMANN 67, MAKESUMMARY 66, MAKEVTK 66, MAXCHOP 52, MGRINTERP, MGRRELAX, MGRSMOOTH, MGRSMOOTHITS, MGRSWEEPS 65, MIXED 61, MODEL 31.

N NCOMP 31, NEWTONMAX 52, NEWTONTOL 52, NX, NY, NZ 32.

O ORTHOPASSES 58, OUTDIR 66.

P PATM 37, PBUB 37, PCRIT 67, PERMFILE 33, PERMMAX, PERMMIN 35, PERMZFILE 34, PINIT 39, POROFILE 34, POROMAX, POROMIN 35, PRECOND 60, PVTCONST 37.

R REFACTOR 68, RESTART 58, RHOG, RHOO, RHOW 35, RS 36.

S SCHEME 56, SGINIT 39, SGOF 39, SOINIT 39, STAGE2 61, SUBSTEPS 52, SWI 38, SWINIT 39, SWOF 38, SWR 38.

T TCRIT 67, TEMP 37, TEND 47, TITLE 32.

U UG, UO, UW 35.

V VMOL 67.

W WELLS 40, WELSPECS 40.

Z ZFACTOR 38, ZMF 67.

Appendix C

Glossary

Adaptive implicit
: A time-stepping scheme in which the implicitness of each cell is chosen individually, typically implicit in pressure everywhere and implicit in saturation only where required for stability.

Aggregation
: A multigrid coarsening strategy that groups unknowns into aggregates and interpolates by injection.

Black oil
: A fluid model with three components, in which gas may dissolve in oil but the composition of each phase is otherwise fixed.

Classical coarsening
: A multigrid coarsening strategy that partitions unknowns into coarse and fine sets using strength of connection.

CPR
: Constrained pressure residual: a two-stage preconditioner combining a multigrid solve of an extracted pressure system with a local smoother on the full system.

df64
: Double-single arithmetic: representing a double-precision value as an unevaluated sum of two single-precision values, so that double-precision accuracy is obtained from single-precision storage and units.

Active-time mask
: The subset of report times for which the reference well is open and the diagnostic denominator is physically meaningful. It prevents long shut periods from dominating a well-profile score.

Carlson hysteresis
: A non-wetting-phase relative-permeability model that stores the largest attained gas saturation and shifts the gas curve continuously after flow reversal.

Completion
: A connection between a well and one reservoir cell, carrying its own well index, depth, open/shut state and phase rates.

Endpoint scaling
: A cell-wise transformation of a saturation-function table so that critical saturations and endpoint permeabilities agree with deck arrays. Its derivative includes the transform's chain rule.

FGMRES
: Flexible generalised minimum residual: a Krylov method that permits the preconditioner to vary between iterations.

FIM Fully implicit method: all unknowns in all cells are solved implicitly.

Halo The layer of cells owned by a neighbouring rank whose values are required to evaluate the residual of a boundary cell.

HCPV average A hydrocarbon-pore-volume-weighted regional average state, used when converting history surface rates to reservoir voidage.

History RESV A producer control obtained by converting observed surface oil, water and gas rates to one reservoir-volume target using regional PVT; it is not the sum of the unconverted surface rates.

NRMSE Root-mean-square error divided by a documented reference scale. Active-time NRMSE remains interpretable when a nearly constant reference makes raw R^2 ill-conditioned.

Peaceman model The relationship between wellbore pressure and the pressure of the containing grid cell, based on the analytic radial solution.

Quasi-IMPES weights The weights used to extract a scalar pressure equation from the coupled system, taken here from the first row of each inverted cell block.

PVT region A per-cell selection of pressure–volume–temperature tables. Well reporting and reservoir assembly must use the same region mapping.

Reference depth The datum at which bottom-hole pressure is reported. Completion pressures include the phase hydrostatic offset between their depth and this datum.

Report step The interval at which output is written. The simulator may subdivide it but does not step across it.

TPFA Two-point flux approximation: the flux across a face is computed from the potentials of the two cells sharing it.

Upstream weighting Evaluation of mobility at the cell from which flow originates, required for stability of the discretised transport.

Voidage balance The reservoir-volume accounting identity relating injection, production and change in in-place phase volume. It is the first check when saturation and pressure drift together.

Water cut Produced water divided by produced oil plus water, evaluated only when the liquid-rate denominator is valid.

WGOR Produced gas divided by produced oil. A shut or negligible oil denominator is masked rather than converted into a numerical spike.

Zero-rate control An explicitly scheduled target of zero. It shuts the whole well for that control interval; it is not treated as a missing value and does not permit cancelling cross-flow between completions.

Appendix D

References

The simulator stands on three external multigrid libraries and on a published body of work in reservoir simulation, algebraic multigrid and mixed-precision numerical linear algebra. Everything cited here is used by the code or by the measurements in this manual.

D.1 Solver libraries

AmgX

M. Naumov, M. Arsaev, P. Castonguay, J. Cohen, J. Demouth, J. Eaton, S. Layton, N. Markovskiy, I. Reguly, N. Sakharnykh, V. Sellappan and R. Strzodka. *AmgX: a library for GPU accelerated algebraic multigrid and preconditioned iterative methods.* SIAM Journal on Scientific Computing 37(5), S602–S626, 2015. The default stage-1 multigrid, selected by `LINSOLVER AMGX`.

HYPRE BoomerAMG

V. E. Henson and U. M. Yang. *BoomerAMG: a parallel algebraic multigrid solver and preconditioner.* Applied Numerical Mathematics 41(1), 155–177, 2002. R. D. Falgout and U. M. Yang. *hypre: a library of high performance preconditioners.* ICCS, 632–641, 2002. Selected by `LINSOLVER HYPRE`.

Ginkgo

H. Anzt, T. Cojean, G. Flegar, F. Göbel, T. Grützmacher, P. Nayak, T. Ribizel, Y.-H. M. Tsai and E. S. Quintana-Ortí. *Ginkgo: a modern linear operator algebra framework for high performance computing.* ACM Transactions on Mathematical Software 48(1), 1–33, 2022. Selected by `LINSOLVER GINKGO`; the precision of its hierarchy is set by `GINKGOPREC`.

Adaptive-precision block-Jacobi

G. Flegar, H. Anzt, T. Cojean and E. S. Quintana-Ortí. *Adaptive precision block-Jacobi for high performance preconditioning in the Ginkgo linear algebra software.* ACM Transactions on Mathematical Software 47(2), 1–28, 2021. The second CPR stage under `STAGE2 GINKGO`.

OPM Flow

A. F. Rasmussen et al. *The Open Porous Media Flow reservoir simulator.* Computers & Mathematics with Applications 81, 159–185, 2021. The reference simulator in every comparison.

D.2 Method

CPR preconditioning

J. R. Wallis. *Incomplete Gaussian elimination as a preconditioning for generalized conjugate gradient acceleration.* SPE 12265, 1983. H. Cao, H. A. Tchelepi, J. R. Wallis and H. E. Yardumian. *Parallel scalable unstructured CPR-type linear solver for reservoir simulation.* SPE 96809, 2005.

Algebraic multigrid
J. W. Ruge and K. Stüben. *Algebraic multigrid.* In Multigrid Methods, SIAM Frontiers in Applied Mathematics, 73–130, 1987.

Flexible GMRES
Y. Saad. *A flexible inner-outer preconditioned GMRES algorithm.* SIAM Journal on Scientific Computing 14(2), 461–469, 1993.

Inexact Newton
S. C. Eisenstat and H. F. Walker. *Choosing the forcing terms in an inexact Newton method.* SIAM Journal on Scientific Computing 17(1), 16–32, 1996.

Well model
D. W. Peaceman. *Interpretation of well-block pressures in numerical reservoir simulation.* SPE Journal 18(3), 183–194, 1978.

D.3 Mixed precision and emulation

Double-word arithmetic
T. J. Dekker. *A floating-point technique for extending the available precision.* Numerische Mathematik 18, 224–242, 1971.

Error-free transformations
K. Ozaki, T. Ogita, S. Oishi and S. M. Rump. *Error-free transformations of matrix multiplication by using fast routines of matrix multiplication and its applications.* Numerical Algorithms 59, 95–118, 2012. H. Ootomo, K. Ozaki and R. Yokota. *DGEMM on integer matrix multiplication unit.* International Journal of High Performance Computing Applications, 2024.

Mixed-precision analysis
N. J. Higham and T. Mary. *Mixed precision algorithms in numerical linear algebra.* Acta Numerica 31, 347–414, 2022.

Roofline
S. Williams, A. Waterman and D. Patterson. *Roofline: an insightful visual performance model for multicore architectures.* Communications of the ACM 52(4), 65–76, 2009.

D.4 Cases and inversion

SPE10 model 2
M. A. Christie and M. J. Blunt. *Tenth SPE comparative solution project: a comparison of upscaling techniques.* SPE Reservoir Evaluation & Engineering 4(4), 308–317, 2001.

Norne
Open Porous Media initiative. *The Norne field data set*, 2012.

Ensemble Kalman inversion
M. A. Iglesias, K. J. H. Law and A. M. Stuart. *Ensemble Kalman methods for inverse problems.* Inverse Problems 29(4), 045001, 2013.

Neural-operator surrogates
Z. Li, N. Kovachki, K. Azizzadenesheli, B. Liu, K. Bhattacharya, A. Stuart and A. Anandkumar. *Fourier neural operator for parametric partial differential equations.* ICLR, 2021.